

RAPPORT DE PROJET

Conception et réalisation d'une solution
intelligente de création et de correction
automatique de QCM basée sur
l'intelligence artificielle

QCM Corrector

Projet réalisé par :

JORIO Lina
LAMRABET Khalil
KABINDA Gaston
SELESSON Japhet

Membres du jury :

MOULINE Salma - Examineur
GUENDOUZ Dalil - Examineur
HDAFA Simohamed - Encadrant
LYAZIDI Achraf - Examineur
NAQOS Hatim - Encadrant

Introduction et remerciements

Dans le cadre de notre formation, ce projet a représenté une étape importante de mise en pratique de nos acquis. Il nous a permis de passer de la théorie à une réalisation concrète, en abordant les différentes phases du projet, depuis l'analyse du besoin jusqu'à la conception et au développement de la solution.

Travailler sur une plateforme comme QCM Corrector demande bien plus que des compétences en programmation. Ce projet nous a amenés à mobiliser plusieurs compétences techniques complémentaires et à explorer différents aspects du développement informatique. Il nous a également appris à mieux organiser notre travail, à collaborer efficacement, à gérer les difficultés techniques et à avancer progressivement malgré les imprévus.

Nous tenons à adresser nos sincères remerciements à M. NAQOS Hatim et M. HDAFA Simohamed, nos encadrants pédagogiques, pour leur accompagnement, leurs conseils et leur suivi tout au long de ce projet. Leur disponibilité et leurs remarques nous ont permis de mieux orienter notre travail, de corriger certaines erreurs et d'améliorer la qualité de notre solution.

Nous souhaitons également remercier Mme MOULINE Salma pour l'aide qu'elle nous a apportée concernant la partie base de données. Ses explications et ses orientations nous ont permis de mieux structurer notre modèle de données, de clarifier certains points et d'améliorer la cohérence générale de notre conception.

Nous remercions également Junia Maroc pour le cadre de travail, les ressources pédagogiques et les moyens mis à notre disposition. Ces éléments ont contribué au bon déroulement du projet et nous ont permis de travailler dans de meilleures conditions.

Cette expérience nous a permis non seulement de consolider nos connaissances techniques, mais aussi de mieux comprendre les exigences d'un projet complet : analyse du besoin, conception, développement, tests, organisation et amélioration continue. Elle constitue pour nous une expérience formatrice et une base solide pour aborder de futurs projets plus ambitieux dans le domaine du développement logiciel et de l'intelligence artificielle.

SOMMAIRE

1. Table des figures	5
2. Présentation générale du projet	7
2.1 Contexte du projet.....	7
2.2 Problématique	7
2.3 Objectifs du projet	8
2.4 Solution proposée	9
3. Architecture générale de la solution	11
3.1 Vue d'ensemble	11
3.2 Application Web	12
3.3 Application Mobile iOS	14
3.4 API Backend	14
3.5 Service IA et traitement d'image	16
3.6 Base de données	17
3.7 Service de notification par mail	17
4. Conception de la base de données	19
4.1 Analyse des besoins.....	19
4.2 Modélisation conceptuelle.....	19
• 4.2.1 Présentation générale	
• 4.2.2 Principales associations	
• 4.2.3 Choix de modélisation	
4.3 Modélisation logique des données.....	21
• 4.3.1 Passage du MCD au MLD	
• 4.3.2 Structure relationnelle obtenue	
4.4 Contraintes d'intégrité et règles métier.....	22
• 4.4.1 Contraintes d'intégrité	
• 4.4.2 Règles métier implémentées	
4.5 Implémentation de la base de données.....	23
• 4.5.1 Choix du SGBD	
• 4.5.2 Mise en œuvre	
5. Traitement d'image et détection IA.....	25
5.1 Notions générales sur le traitement d'images.....	25
5.2 Structure de la feuille de réponse.....	27
5.3 Prétraitement sur la feuille	28
5.4 Création des modèles IA et constitution du dataset.....	30
5.5 Détection du numéro étudiant	32
5.6 Détection des réponses cochées	33
5.7 Détection du nom complet par OCR.....	35
5.8 Diagramme d'activité	36

6. Réalisation de l'application web	37
6.1 Présentation	37
6.2 Architecture de l'application web	41
6.3 Gestion de l'authentification et des rôles	45
6.4 Interfaces et parcours utilisateur	48
6.5 Communication avec l'API Backend	50
6.6 Cas d'utilisation détaillés	54
• 6.6.1 Connexion	
• 6.6.2 Tableau de bord	
• 6.6.3 Gestion des QCM	
• 6.6.4 Création d'un QCM	
• 6.6.5 Publication et génération des PDF	
• 6.6.6 Sessions et résultats	
• 6.6.7 Envoi des résultats par mail	
6.7 Conception UML	67
• 6.7.1 Diagramme de cas d'utilisation de l'application web	
• 6.7.2 Diagramme de séquence - Création et publication d'un QCM	
• 6.7.3 Diagramme de cycle de vie d'un QCM	
7. Réalisation de l'application mobile iOS	71
7.1 Présentation	71
7.2 Rôle de l'application mobile	71
7.3 Architecture interne de l'application mobile	71
• 7.3.1 Authentification de l'enseignant	
• 7.3.2 Tableau de bord	
• 7.3.3 Consultation des sessions de correction	
• 7.3.4 Scan de la feuille de réponses	
• 7.3.5 Envoi de l'image vers l'API	
• 7.3.6 Affichage des résultats de correction	
• 7.3.7 Historique des corrections	
7.4 Conception UML	76
• 7.4.1 Diagramme de cas d'utilisation de l'application mobile	
• 7.4.2 Diagramme de séquence - Scan d'une feuille et affichage du résultat	
• 7.4.3 Diagramme d'activité - Parcours de correction depuis l'application mobile	
8. Technologies et outils de développement	78
8.1 Technologies et outils utilisés pour la création de la base de données	78
8.2 Technologies et outils utilisés pour le traitement d'image et le service IA	79
8.3 Technologies et outils utilisés dans le développement de l'application web	82
8.4 Technologies et outils utilisés dans le développement de l'application mobile	89
9- Difficultés rencontrées et perspectives	90
10- Conclusion	92

1- TABLE DES FIGURES

Figure 1 : Héritage Utilisateur	20
Figure 2 : Héritage Personne	21
Figure 3 : MCD de la base de données.....	24
Figure 4 : MLD de la base de données.....	24
Figure 5 : Exemple de conversion en niveaux de gris.....	26
Figure 6 : Exemple de seuillage d'Otsu.....	26
Figure 7 : Exemple de flou Gaussien.....	26
Figure 8 : Exemple de correction de perspective	27
Figure 9 : Structure de la feuille de réponse QCM	28
Figure 10 : Transformation de perspective de la feuille de réponse	29
Figure 11 : Extrait du dataset des cases de réponses et du numéro étudiant	31
Figure 12 : Grille numéro étudiant vide	33
Figure 13 : Grille numéro étudiant remplie et scannée	33
Figure 14 : Extraction des cases	34
Figure 15 : Zone d'identification du nom complet vide	36
Figure 16 : Zone d'identification du nom complet remplie et scannée	36
Figure 17 : Diagramme d'activité	36
Figure 18 : Actions de l'administrateur sur les classes, professeurs et étudiants	38
Figure 19 : Page de création d'un QCM par un administrateur pour un professeur	39
Figure 20 : Communication entre l'utilisateur, l'application web, l'API Backend et la base PostgreSQL	41
Figure 21 : Organisation fonctionnelle des principales rubriques de l'application web.....	42
Figure 22 : Schéma des principales communications entre le front-end, le backend et les services associés	44
Figure 23 : Schéma du fonctionnement général de l'authentification	45
Figure 24 : Répartition des fonctionnalités entre l'administrateur et le professeur	47
Figure 25 : Parcours général depuis la connexion jusqu'à l'envoi des résultats	49
Figure 26 : Flux de communication entre l'utilisateur, l'application web, l'API Backend et la base de données	50
Figure 27 : Synthèse des principales communications entre l'application web et le backend.....	53
Figure 28 : Page de choix du profil utilisateur.....	54
Figure 29 : Formulaire de connexion administrateur et professeur.....	55
Figure 30 : Tableau de bord.....	55
Figure 31 : Interface de gestion des QCM.....	56
Figure 32 : Création d'un QCM - Informations générales.....	57
Figure 33 : Création d'un QCM - Saisie des questions et des réponses.....	58
Figure 34 : Récapitulatif du QCM avant publication.....	59
Figure 35 : Page de confirmation.....	60
Figure 36 : Publication du QCM.....	60
Figure 37 : Génération du sujet PDF.....	61
Figure 38 : Génération de la grille de correction PDF.....	62
Figure 39 : Sessions et résultats.....	63
Figure 40 : Détail d'une session et affichage du score.....	64
Figure 41 : Envoi des résultats par mail.....	65
Figure 42 : Mail reçu par l'étudiant	66
Figure 43 : Diagramme de cas d'utilisation de l'application web	67
Figure 44 : Diagramme de séquence - Création et publication d'un QCM	69
Figure 45 : Diagramme de cycle de vie d'un QCM	70

Figure 46 : Page d'authentification	72
Figure 47 : Tableau de bord mobile	73
Figure 48 : Page de scan	74
Figure 49 : Page d'affichage des résultats	75
Figure 50 : Page de l'historique	75
Figure 51 : Diagramme de cas d'utilisation de l'application mobile	76
Figure 52 : Diagramme de séquence - Scan d'une feuille et affichage du résultat	76
Figure 53 : Diagramme d'activité - Parcours de correction	77

2-Présentation générale :

2.1 Contexte du projet :

Dans le domaine de l'enseignement, les questionnaires à choix multiples, appelés QCM, sont très utilisés pour évaluer rapidement les connaissances des étudiants. Ils permettent aux enseignants de poser un grand nombre de questions en peu de temps et d'obtenir une évaluation structurée des réponses.

Cependant, malgré leur simplicité d'utilisation, les QCM posent un problème important lorsqu'il s'agit de corriger un grand nombre de copies. La correction manuelle demande beaucoup de temps, surtout lorsque le nombre d'étudiants est élevé. Elle peut également provoquer des erreurs humaines lors du comptage des réponses correctes, du calcul des notes ou de la saisie des résultats.

Avec le développement des technologies numériques, il devient possible d'automatiser une partie de ce processus. Le traitement d'image et l'intelligence artificielle permettent aujourd'hui d'analyser automatiquement une feuille de réponses photographiée ou scannée, de détecter les cases cochées et de produire un résultat exploitable.

Notre projet s'inscrit dans ce contexte. Il consiste à concevoir une solution informatique permettant aux enseignants de créer des QCM, de corriger automatiquement les feuilles de réponses et d'archiver les résultats. L'objectif est de rendre le processus d'évaluation plus rapide, plus fiable et plus facile à gérer.

Le projet QCM Corrector repose sur plusieurs composants complémentaires : une application web pour la gestion des QCM et des résultats, une application mobile pour le scan des feuilles, un service de traitement d'image pour la détection automatique des réponses, une base de données pour l'archivage, et un service de notification par mail pour communiquer les résultats aux étudiants.

Ainsi, le projet ne se limite pas à une simple application de correction. Il propose une solution complète permettant de gérer le cycle de vie d'un QCM, depuis sa création jusqu'à l'envoi du résultat à l'étudiant.

2.2 Problématique :

La correction des QCM est une tâche répétitive et chronophage pour les enseignants. Lorsqu'un enseignant doit corriger plusieurs dizaines ou centaines de copies, le temps consacré à cette activité devient très important. Ce temps pourrait être utilisé pour d'autres tâches pédagogiques plus utiles, comme l'analyse des difficultés des étudiants, la préparation des cours ou l'accompagnement individuel.

En plus du temps nécessaire, la correction manuelle peut entraîner plusieurs problèmes :

- erreur dans la lecture d'une réponse
- oubli d'une question
- erreur dans le calcul de la note
- difficulté à retrouver les anciennes copies
- absence d'historique numérique

- communication lente des résultats aux étudiants
- difficulté à produire des statistiques sur les performances

Un autre problème concerne l'organisation des QCM. Lorsqu'un QCM est utilisé pour une session de correction, il ne doit plus être modifié, car cela pourrait fausser les résultats déjà enregistrés. Il faut donc mettre en place un système capable de protéger l'intégrité des QCM utilisés et des résultats archivés.

La problématique principale du projet peut donc être formulée ainsi :

Comment concevoir une solution capable de créer, corriger, archiver et communiquer automatiquement les résultats des QCM, tout en réduisant le temps de correction, les erreurs humaines et les difficultés de gestion pour les enseignants ?

Pour répondre à cette problématique, notre projet doit assurer plusieurs fonctions importantes : la création des QCM, le scan des feuilles de réponses, la détection automatique des cases cochées, l'identification de l'étudiant, le calcul du score, l'archivage des résultats et l'envoi automatique d'un mail à l'étudiant.

2.3 Objectifs du projet

L'objectif principal du projet QCM Corrector est de simplifier et automatiser la correction des QCM dans un environnement scolaire ou universitaire.

Pour atteindre cet objectif général, plusieurs objectifs spécifiques ont été définis.

Premièrement, le système doit permettre à l'enseignant de créer des QCM à partir d'une application web. L'enseignant doit pouvoir renseigner les questions, les choix de réponses et les bonnes réponses. Il doit également pouvoir gérer les QCM créés et les utiliser dans des sessions de correction.

Deuxièmement, le projet doit permettre la création de sessions de correction. Une session correspond à une évaluation donnée, associée à un QCM et à une classe ou un groupe d'étudiants. Cette organisation permet de regrouper les copies corrigées et de consulter les résultats par session.

Troisièmement, l'application mobile doit permettre à l'enseignant de scanner ou photographier une feuille de réponses. L'image obtenue doit ensuite être envoyée au système pour être analysée automatiquement.

Quatrièmement, le service de traitement d'image doit être capable de redresser la feuille, d'extraire les zones importantes, de lire le numéro étudiant, de reconnaître le nom de l'étudiant et de détecter les réponses cochées.

Cinquièmement, le backend doit exploiter les réponses détectées afin de calculer automatiquement le score de l'étudiant en comparant ses réponses avec la grille de correction du QCM.

Sixièmement, les résultats doivent être enregistrés dans une base de données afin d'assurer l'archivage, la traçabilité et la consultation ultérieure.

Enfin, le système doit envoyer automatiquement un mail à l'étudiant après la correction. Ce mail contient les informations essentielles comme le nom de l'évaluation, la date et le score obtenu.

Les objectifs du projet peuvent donc être résumés comme suit :

- automatiser la correction des QCM
- réduire le temps de correction
- limiter les erreurs humaines
- faciliter la création et la gestion des QCM
- archiver les résultats de manière fiable
- permettre la consultation de l'historique
- notifier automatiquement les étudiants
- offrir une solution évolutive pouvant être améliorée par la suite

2.4 Solution proposée :

Pour répondre à la problématique, nous proposons une solution appelée QCM Corrector. Il s'agit d'un système composé de plusieurs parties qui collaborent entre elles afin d'automatiser le processus de correction.

La solution comprend d'abord une application web. Cette application est utilisée principalement par les enseignants et les administrateurs. Elle permet de créer les QCM, gérer les questions, gérer les classes, créer les sessions de correction, consulter les résultats, accéder à l'historique et afficher des statistiques. Elle représente donc l'espace principal de gestion du projet.

La deuxième partie est une application mobile iOS. Cette application est utilisée par l'enseignant au moment de la correction. Elle permet de choisir une session, scanner la feuille de réponses avec la caméra du téléphone, envoyer l'image au backend et afficher le résultat de la correction.

La troisième partie est une API backend. Elle joue le rôle d'intermédiaire entre l'application web, l'application mobile, le service de traitement d'image, la base de données et le service mail. Elle gère l'authentification, les QCM, les sessions, les résultats et l'envoi des notifications.

La quatrième partie est le service de traitement d'image et détection IA. Ce service reçoit l'image de la feuille de réponses, applique les traitements nécessaires et retourne les informations détectées : nom de l'étudiant, numéro étudiant et réponses cochées. Ce module utilise OpenCV, EasyOCR et des modèles d'intelligence artificielle pour analyser la feuille.

La cinquième partie est la base de données. Elle permet de stocker les utilisateurs, les étudiants, les classes, les QCM, les questions, les sessions, les résultats et les notifications. Grâce à cette base de données, le système peut conserver un historique fiable de toutes les corrections.

Enfin, la solution comprend un service de notification par mail. Après chaque correction, le système envoie automatiquement un mail à l'étudiant contenant son résultat. Cela permet d'éviter la création d'un compte étudiant dans l'application et de simplifier la communication.

Le fonctionnement général de la solution est le suivant :

1. L'enseignant crée un QCM depuis l'application web
2. Il publie le QCM et crée une session de correction
3. Il scanne la feuille de réponses avec l'application mobile
4. L'image est envoyée au backend
5. Le backend transmet l'image au service de traitement d'image
6. Le service détecte le nom, le numéro étudiant et les réponses cochées
7. Le backend calcule le score
8. Le résultat est enregistré dans la base de données
9. Un mail est envoyé automatiquement à l'étudiant
10. L'enseignant peut consulter le résultat dans l'historique

Cette solution permet donc de couvrir tout le processus, depuis la création du QCM jusqu'à la communication du résultat. Elle offre un gain de temps important, améliore la fiabilité de la correction et facilite le suivi des évaluations.

3-Architecture générale :

L'architecture générale de notre solution repose sur plusieurs composants qui collaborent entre eux afin d'assurer le fonctionnement complet du système. Le projet ne se limite pas à une simple interface de correction, mais se présente comme une plateforme complète permettant de gérer la création des QCM, le scan des feuilles, la correction automatique, l'archivage des résultats et l'envoi des notifications aux étudiants.

La solution est organisée autour de sept éléments principaux :

- une application web
- une application mobile iOS
- une API Backend
- un service IA et traitement d'image
- une base de données
- un service de notification par mail
- une intégration avec les listes d'étudiants et de classes

Cette séparation permet d'avoir une architecture claire, modulaire et évolutive. Chaque partie possède un rôle précis et peut être développée, testée ou améliorée indépendamment des autres.

3.1 Vue d'ensemble :

Le fonctionnement global de la solution commence par la création du QCM depuis l'application web. L'enseignant crée un QCM, définit les questions, les choix de réponses et les bonnes réponses. Une fois le QCM prêt, il peut être publié et utilisé dans une session de correction.

Lors de la correction, l'enseignant utilise l'application mobile iOS pour scanner la feuille de réponses d'un étudiant. L'image capturée est envoyée à l'API Backend. Cette API transmet ensuite l'image au service IA chargé du traitement d'image.

Le service IA analyse la feuille. Il détecte les repères de coin, redresse l'image, extrait les zones utiles, lit le nom de l'étudiant, détecte le numéro étudiant et identifie les réponses cochées. Les informations extraites sont ensuite retournées au backend.

Le backend compare les réponses détectées avec la grille de correction du QCM. Il calcule le score, enregistre le résultat dans la base de données et déclenche l'envoi automatique d'un mail à l'étudiant. L'enseignant peut ensuite consulter le résultat dans l'application web ou mobile.

Le flux général peut être résumé ainsi :

1. Création du QCM depuis l'application web
2. Publication du QCM
3. Création d'une session de correction
4. Scan de la feuille depuis l'application mobile
5. Envoi de l'image vers l'API Backend
6. Analyse de l'image par le service IA

7. Retour des réponses détectées
8. Calcul du score par le backend
9. Enregistrement du résultat en base de données
10. Envoi automatique du résultat par mail
11. Consultation des résultats par l'enseignant

Cette organisation permet de séparer clairement les responsabilités. L'application web est dédiée à la gestion, l'application mobile est dédiée au scan, le backend assure la coordination, le service IA réalise l'analyse des feuilles, et la base de données conserve toutes les informations.

3.2 Application Web :

L'application web représente l'interface principale de gestion du système QCM Corrector. Elle est utilisée par deux types d'utilisateurs : les enseignants et l'administrateur. Son objectif est de centraliser la gestion des QCM, des sessions de correction, des classes, des étudiants, des résultats et des comptes utilisateurs.

Elle a été développée avec Next.js, React et TypeScript, avec une interface organisée autour d'un tableau de bord. L'application communique avec le backend à travers des requêtes HTTP, et les données sont échangées principalement au format JSON. Elle ne communique donc pas directement avec la base de données : toutes les opérations passent par l'API backend.

Pour l'enseignant, l'application web permet principalement de créer et gérer des QCM. Il peut saisir les informations générales de l'évaluation, comme le titre, la matière, la classe, la date d'examen et le nombre de questions. Il peut ensuite ajouter les questions, définir les choix de réponses, choisir entre une réponse unique ou des réponses multiples, indiquer les bonnes réponses et attribuer une pondération à chaque question. L'interface a été conçue pour rester simple et claire afin qu'un enseignant non technicien puisse créer un QCM facilement.

L'application web permet également de gérer le cycle de vie des QCM. Un QCM peut être enregistré comme brouillon, publié, puis verrouillé après utilisation. Tant qu'il est en brouillon, il peut être modifié. Une fois publié, il devient disponible pour une session de correction. Lorsqu'il est déjà utilisé pour recevoir des résultats, il doit être verrouillé afin d'éviter toute modification de la grille de correction, ce qui pourrait fausser les scores déjà enregistrés.

Après la publication d'un QCM, l'application permet de générer deux documents importants : le sujet PDF destiné aux étudiants et la grille PDF contenant les bonnes réponses. Cette grille sert de référence pour la correction automatique et permet aussi à l'enseignant de vérifier le contenu publié.

L'application web permet aussi de créer et suivre les sessions de correction. Une session correspond à une évaluation donnée, associée à un QCM, à une classe, à une matière et à une date. Grâce à cette organisation, les résultats sont regroupés clairement et peuvent être consultés par session. L'enseignant peut donc suivre l'avancement des corrections et retrouver l'historique des évaluations précédentes.

Une partie importante de l'application concerne l'affichage des résultats. Après réception des réponses détectées par l'application mobile ou le service IA, l'enseignant peut consulter le score de chaque étudiant, son identifiant, les réponses détectées, le nombre de questions correctes, le statut de correction et le statut d'envoi du mail. Cela permet de vérifier rapidement les copies valides et celles qui nécessitent une vérification manuelle.

Pour l'administrateur, l'application web offre une vue plus globale. Il peut consulter l'ensemble des QCM, des sessions, des classes, des étudiants et des professeurs. Il peut aussi créer un QCM à la place d'un professeur en sélectionnant le professeur, la matière et la classe concernés. Cette fonctionnalité est importante car elle permet à l'administration de préparer ou corriger des évaluations pour différents enseignants sans se limiter à une seule matière.

L'administrateur peut également gérer les comptes professeurs, les classes et les listes d'étudiants. Le système peut être enrichi plus tard par un import depuis une source externe comme Edusign afin de récupérer automatiquement les étudiants et leurs informations. Cela permettrait d'éviter la saisie manuelle et de maintenir les données à jour.

L'application web adapte aussi son affichage selon le profil connecté. L'administrateur dispose d'une vision complète de la plateforme, tandis qu'un professeur ne voit que les QCM, classes, matières, sessions et résultats liés à ses propres affectations. Cette séparation des accès permet de mieux organiser les responsabilités et de protéger les données pédagogiques.

En résumé, l'application web constitue la partie administrative et pédagogique du projet. Elle centralise la création des QCM, la gestion des utilisateurs, le suivi des classes, la consultation des résultats, la génération des PDF et l'envoi des notes par mail. Elle sert donc de point de contrôle principal pour toute la plateforme QCM Corrector.

3.3 Application Mobile iOS :

L'application mobile iOS est destinée à l'enseignant au moment de la correction des copies. Son rôle principal est de permettre le scan ou la prise de photo de la feuille de réponses.

Le choix d'une application mobile est justifié par le besoin de travailler directement en situation réelle, par exemple dans une salle de classe ou un bureau d'enseignant. Au lieu d'utiliser un scanner classique, l'enseignant peut simplement utiliser la caméra de son téléphone pour capturer la feuille.

Le fonctionnement de l'application mobile est simple. L'enseignant se connecte, choisit une session de correction, puis photographie la feuille de réponses. L'application doit guider l'utilisateur pour que la feuille soit bien cadrée et que les repères de coin soient visibles. Une bonne prise de photo améliore la précision de la détection.

Après la capture, l'image est envoyée au backend à travers une requête HTTP sécurisée. Le backend transmet ensuite l'image au service IA. Lorsque la correction est terminée, l'application mobile peut afficher le résultat à l'enseignant : nom de l'étudiant, numéro étudiant, score et statut de correction.

L'application mobile doit également gérer certains cas d'erreur. Par exemple, si la feuille est mal cadrée, si les repères ne sont pas détectés ou si la connexion réseau est instable, l'application doit informer l'enseignant et lui permettre de reprendre la photo.

Ainsi, l'application mobile constitue la partie terrain du projet. Elle permet de transformer rapidement une feuille papier en image numérique exploitable par le système.

3.4 API Backend :

L'API Backend constitue le cœur de la solution QCM Corrector. Elle joue le rôle d'intermédiaire entre l'application web, l'application mobile iOS, le service IA de traitement d'image, la base de données PostgreSQL et le service d'envoi de mails. Elle centralise toute la logique métier du système et garantit que les différentes parties de l'application communiquent de manière cohérente.

Le backend a été développé avec FastAPI, un framework Python adapté à la création d'API REST rapides et structurées. Les échanges avec les applications clientes se font principalement à travers des requêtes HTTP et des réponses au format JSON. Le backend est exécuté avec Uvicorn, qui permet de lancer le serveur API en local ou sur un environnement de déploiement.

Son premier rôle est de gérer l'authentification. Les enseignants et l'administrateur doivent se connecter avant d'accéder aux fonctionnalités du système. L'authentification permet d'identifier l'utilisateur connecté, de connaître son rôle et de limiter ses actions selon ses droits. L'administrateur possède une vue globale sur les données, tandis qu'un professeur ne peut consulter que les classes, matières, QCM, sessions et résultats liés à ses propres affectations.

Le backend gère également les QCM. Depuis l'application web, il reçoit les informations générales du QCM, les questions, les propositions, les bonnes réponses, la pondération, la matière, la classe et le professeur responsable. Il permet d'enregistrer un QCM comme brouillon, de le publier, puis de l'associer à une session de correction. Il applique aussi les règles métier importantes, notamment le fait d'éviter la modification d'un QCM déjà utilisé dans une session, afin de ne pas fausser les résultats déjà calculés.

Lorsqu'un QCM est publié, le backend crée ou met à jour les données nécessaires dans PostgreSQL : QCM, questions, propositions, bonnes réponses, matière, classe et session de correction. Il fournit ensuite à l'application web les informations permettant de générer le sujet PDF et la grille PDF des bonnes réponses. Ces documents sont utilisés pour l'évaluation et pour vérifier la correction attendue.

Le backend gère aussi les sessions de correction. Une session correspond à une évaluation donnée, associée à un QCM, une classe, une matière et un enseignant. Cette organisation permet de regrouper clairement les résultats des étudiants. L'application web peut ensuite récupérer la liste des sessions, leur statut, le nombre de copies reçues, la moyenne, le nombre de corrections nécessitant une vérification manuelle et le statut d'envoi des mails.

Une autre responsabilité importante du backend concerne la réception des résultats détectés. Lorsque les réponses d'une copie sont analysées par l'application mobile ou le service IA, les informations détectées sont envoyées à l'API. Ces données peuvent contenir le nom de l'étudiant, son numéro étudiant et les réponses cochées. Le backend associe alors la copie à l'étudiant correspondant grâce à son identifiant, puis compare les réponses détectées avec les bonnes réponses du QCM enregistré. Après cette comparaison, le backend calcule le score obtenu par l'étudiant. Il prend en compte le nombre de questions, les réponses correctes, les réponses multiples et la pondération définie pour chaque question. Le résultat est ensuite enregistré dans la base de données afin d'être consulté depuis l'application web. Si une incohérence est détectée, par exemple un étudiant introuvable ou une réponse ambiguë, le résultat peut être marqué comme nécessitant une vérification manuelle.

Le backend fournit également les routes nécessaires à l'administration du système. Il permet de récupérer les classes, les étudiants, les professeurs, les matières, les QCM et les sessions. Il applique les filtres selon le rôle connecté : l'administrateur voit l'ensemble des données, tandis que chaque professeur ne voit que les éléments qui lui sont associés.

Enfin, l'API backend est responsable du déclenchement de l'envoi des résultats par mail. Une fois les scores calculés et enregistrés, l'application web peut demander au backend d'envoyer les notes aux étudiants concernés. Le backend récupère alors les informations nécessaires, génère un mail personnalisé pour chaque étudiant et utilise le service SMTP configuré pour effectuer l'envoi.

En résumé, l'API Backend assure la coordination complète du système. Elle relie les interfaces utilisateur, la base de données, le module IA et le service mail. Elle garantit la cohérence des données, applique les règles métier et permet à la solution QCM Corrector de fonctionner comme une plateforme unifiée.

3.5 Service IA et traitement d'image :

Le service IA est responsable de l'analyse automatique des feuilles de réponses. Il reçoit une image envoyée par le backend et retourne les informations détectées sur la feuille.

Ce service applique d'abord une étape de prétraitement. Il détecte les quatre repères de coin de la feuille, puis applique une correction de perspective afin de redresser l'image. Cette étape est indispensable, car une photo prise avec un smartphone peut être inclinée ou déformée.

Après redressement, l'image est ramenée à une taille fixe. Cela permet d'utiliser des coordonnées précises pour découper les zones importantes de la feuille : zone du nom, zone du numéro étudiant et zone des réponses.

Le service IA réalise ensuite la détection du numéro étudiant. La zone du numéro est extraite, puis les cases sont détectées et classées. Le système utilise un modèle d'intelligence artificielle entraîné pour déterminer si chaque case est cochée ou non. Les cases détectées sont regroupées par colonnes afin de reconstruire le numéro étudiant.

Ensuite, le service détecte les réponses cochées. Les coordonnées des cases sont chargées depuis un fichier de configuration. Les cases sont regroupées par question, puis chaque case est analysée par un modèle de classification. Le système détermine alors si la réponse A, B, C ou D a été cochée.

Le service IA utilise également un module OCR pour lire le nom complet de l'étudiant. La zone du nom est extraite, améliorée par plusieurs traitements, puis analysée avec EasyOCR. Le texte obtenu est ensuite nettoyé afin de supprimer les caractères inutiles.

Le résultat final du service IA contient les informations détectées sous une forme structurée :

- nom complet
- numéro étudiant
- réponses de q1 à q60

Ce service est isolé du reste de l'application. Cette séparation est importante, car elle permet de modifier ou améliorer les modèles de détection sans changer l'application web, l'application mobile ou le backend.

3.6 Base de données :

La base de données permet de stocker toutes les informations nécessaires au fonctionnement du système. Elle assure la persistance des données, c'est-à-dire que les informations restent disponibles même après fermeture de l'application.

Elle contient d'abord les utilisateurs du système, notamment les enseignants et les administrateurs. Chaque utilisateur possède des informations d'identification et un rôle qui définit ses droits d'accès.

Elle contient également les classes et les étudiants. Les étudiants peuvent être associés à une classe, ce qui permet d'organiser les sessions de correction par groupe.

La base de données stocke aussi les QCM créés par les enseignants. Chaque QCM contient des informations générales comme le titre, la matière, le niveau, le statut et l'enseignant créateur. Les questions sont associées au QCM avec leurs choix de réponses et leurs bonnes réponses.

Les sessions de correction sont également enregistrées. Une session est liée à un QCM, à une classe et à un enseignant. Elle sert à regrouper les résultats d'une évaluation.

Enfin, la base de données conserve les résultats des étudiants. Pour chaque correction, elle peut stocker le numéro étudiant, les réponses détectées, le score, le statut de détection et le statut d'envoi du mail.

Cette base de données permet donc d'assurer l'historique, la traçabilité et la cohérence du système. Elle est indispensable pour consulter les anciennes corrections, vérifier les résultats et produire des statistiques.

3.7 Service de notification par mail :

Le service de notification par mail permet de communiquer automatiquement les résultats aux étudiants après la correction des QCM. Dans notre projet, l'étudiant n'a pas besoin de se connecter à l'application web ou mobile pour consulter sa note. Une fois la correction effectuée, il peut recevoir directement son résultat par mail.

Le fonctionnement du service repose sur le backend. Après la réception des réponses détectées et le calcul du score, le backend enregistre d'abord le résultat dans la base de données PostgreSQL. Cette étape est essentielle, car elle garantit que le résultat reste disponible dans l'application web, même si l'envoi du mail échoue. L'enseignant ou l'administrateur peut ensuite déclencher l'envoi des mails depuis l'interface web, à partir de la page des résultats d'une session.

Le mail envoyé à l'étudiant contient les informations principales liées à son évaluation : son nom, le nom du QCM, la matière concernée, la classe et le score obtenu. Le message est présenté sous forme structurée avec l'identité de l'école afin de rendre la communication plus claire et plus professionnelle.

Dans le cadre du projet, l'envoi des mails a été configuré avec un service SMTP réel. Un compte Gmail est utilisé avec un mot de passe d'application. Les informations sensibles comme l'hôte SMTP, le port, l'adresse d'envoi et le mot de passe sont stockées dans le fichier .env du backend. Cela permet de séparer la configuration du code source et de modifier les paramètres d'envoi sans changer le code de l'application.

Le système gère également un statut d'envoi. Si les mails sont envoyés correctement, le statut de la session devient envoyé. Si seulement une partie des mails est envoyée ou si certaines erreurs apparaissent, le statut peut devenir partiel. Si les mails n'ont pas encore été envoyés, le statut reste en attente. Ces statuts permettent à l'enseignant ou à l'administrateur de savoir rapidement si les étudiants ont bien reçu leurs résultats.

Idéalement, l'envoi du mail doit être asynchrone afin de ne pas bloquer l'utilisation de l'application. Cela signifie que le résultat doit rester enregistré et visible immédiatement, même si l'envoi prend du temps ou rencontre une erreur. Dans le fonctionnement actuel, le système respecte déjà le principe important de sauvegarde préalable : les résultats sont enregistrés avant l'envoi des mails.

Cette fonctionnalité améliore la communication avec les étudiants. Elle évite à l'enseignant d'envoyer manuellement les résultats un par un et garantit que chaque étudiant peut recevoir rapidement son score après correction. Elle complète donc le processus automatique de correction en passant d'un simple résultat enregistré à une communication directe avec l'étudiant.

En résumé, le service mail transforme la plateforme en une solution plus complète. Il ne se limite pas à afficher les notes dans l'application web, mais permet aussi de transmettre automatiquement les résultats aux étudiants concernés, de manière claire, personnalisée et traçable.

4-Conception de la base de données

4.1 Analyse des besoins métier :

La conception de la base de données repose sur les besoins fonctionnels définis dans le cahier des charges de la plateforme QCM Corrector. L'objectif est de fournir une structure de données capable de supporter l'ensemble des opérations liées à la création, la gestion, la publication et l'archivage des QCM.

L'étude des besoins a permis d'identifier plusieurs acteurs intervenant dans le système :

- L'administrateur, chargé de la gestion des utilisateurs et des référentiels
- L'enseignant, responsable de la création et de l'administration des QCM
- L'étudiant, participant aux évaluations pédagogiques
- Le candidat, concerné par les évaluations de type concours

Les principales fonctionnalités à prendre en charge sont :

- la gestion des comptes utilisateurs
- la création et la gestion des QCM
- la gestion des matières
- l'organisation des concours
- la gestion des questions et des réponses
- la sauvegarde automatique des brouillons
- le suivi des sessions de passage des QCM

Ces besoins constituent la base de la modélisation des données.

4.2 Modélisation conceptuelle des données (MCD) :

4.2.1 Présentation générale :

Le Modèle Conceptuel de Données (MCD) représente les informations manipulées par le système ainsi que les relations existant entre elles. Il constitue une représentation indépendante de toute technologie de stockage.

Le modèle élaboré s'articule autour de plusieurs entités principales :

- Utilisateur ;
- Admin ;
- Enseignant ;
- Personne ;
- Étudiant ;
- Candidat ;
- Matière ;

- QCM ;
- Concours ;
- Question ;
- Réponse ;
- Brouillon/Autosave ;
- Instance_QCM.

4.2.2 Principales associations :

Les associations retenues permettent de représenter les règles métier essentielles de la plateforme :

- Création des QCM : Un utilisateur peut créer plusieurs QCM tandis qu'un QCM possède un auteur principal unique.
- Composition d'un QCM : Un QCM est composé de plusieurs questions. Chaque question appartient à un seul QCM.
- Gestion des réponses : Chaque question possède plusieurs réponses. Une réponse appartient exclusivement à une question.
- Organisation pédagogique : Chaque QCM est associé à une matière. Dans le cas d'un QCM de type concours, il peut également être rattaché à un concours.
- Passage des évaluations : Une personne peut participer à plusieurs sessions d'évaluation représentées par l'entité Instance_QCM.
- Gestion des brouillons : Le système conserve les sauvegardes intermédiaires grâce à l'entité Brouillon/Autosave, associée à un utilisateur et à un QCM.

4.2.3 Choix de modélisation :

Afin de limiter les redondances et de favoriser la maintenabilité du modèle, deux mécanismes d'héritage ont été mis en place.

Héritage Utilisateur

L'entité Utilisateur regroupe les informations communes aux personnes disposant d'un accès à la plateforme :

- identifiant ;
- adresse électronique ;
- mot de passe chiffré ;
- préférences d'interface.

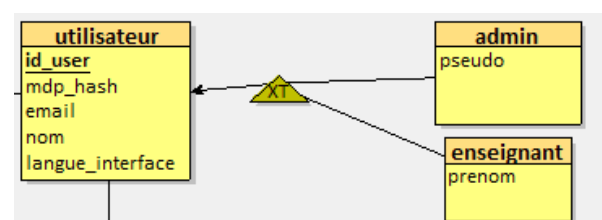


Figure 1 : Héritage Utilisateur

Cette entité est spécialisée en :

- Admin ;
- Enseignant.

Cette approche évite la duplication des attributs communs.

Héritage Personne

L'entité Personne regroupe les informations partagées par les étudiants et les candidats :

- nom ;
- prénom ;
- adresse électronique ;
- groupe.

Elle est spécialisée en :

- Étudiant ;
- Candidat.

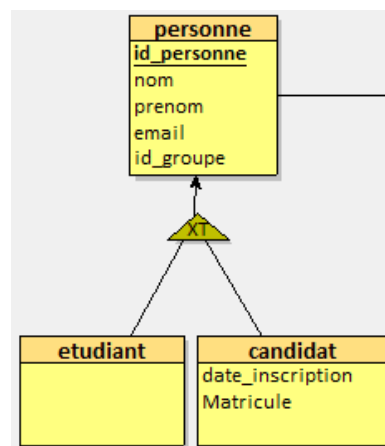


Figure 2 : Héritage Personne

Cette modélisation améliore la cohérence du système et simplifie les évolutions futures.

4.3 Modélisation logique des données (MLD) :

4.3.1 Passage du MCD au MLD :

Après validation du modèle conceptuel, celui-ci a été transformé en modèle logique relationnel.

Chaque entité du MCD a été convertie en table relationnelle. Les associations ont été traduites à l'aide de clés étrangères ou de tables spécifiques lorsque les cardinalités l'exigeaient.

4.3.2 Structure relationnelle obtenue :

Le modèle logique comporte notamment les tables suivantes :

- utilisateur ;
- admin ;
- enseignant ;
- personne ;
- etudiant ;
- candidat ;
- matiere ;

- qcm ;
- concours ;
- question ;
- reponse ;
- brouillon ;
- instance_qcm.

L'ensemble de ces tables permet de représenter fidèlement les relations métier définies lors de l'analyse des besoins.

4.4 Contraintes d'intégrité et règles métier :

4.4.1 Contraintes d'intégrité :

Afin de garantir la cohérence des données, plusieurs mécanismes ont été mis en œuvre :

- clés primaires pour identifier de manière unique chaque enregistrement
- clés étrangères pour assurer la cohérence des relations
- contraintes d'unicité sur certaines données sensibles comme les adresses électroniques
- contraintes de validation sur certains attributs métiers

4.4.2 Règles métier implémentées :

La base de données applique les principales règles métier du projet :

- un enseignant peut créer plusieurs QCM
- un QCM possède un auteur principal unique
- un QCM appartient à une seule matière
- un QCM peut être associé à un concours
- une question appartient à un seul QCM
- une réponse appartient à une seule question
- un brouillon est associé à un utilisateur et à un QCM
- un QCM publié devient verrouillé lorsque la date de l'examen est atteinte
- les administrateurs disposent d'une visibilité globale sur les données de la plateforme

4.5 Implémentation de la base de données :

4.5.1 Choix du SGBD (Système de Gestion de Base de Données) :

L'implémentation a été réalisée sous PostgreSQL.

Ce système de gestion de bases de données a été retenu pour :

- sa conformité aux standards SQL
- sa robustesse
- ses performances
- sa compatibilité avec l'environnement Node.js
- sa capacité d'évolution

4.5.2 Mise en oeuvre :

L'ensemble des tables, contraintes et relations identifiées dans le modèle logique a été implémenté dans PostgreSQL.

Les scripts SQL réalisés permettent :

- la création des tables
- la définition des clés primaires
- la mise en place des clés étrangères
- l'application des contraintes d'intégrité

La base obtenue constitue le socle de persistance de la plateforme.

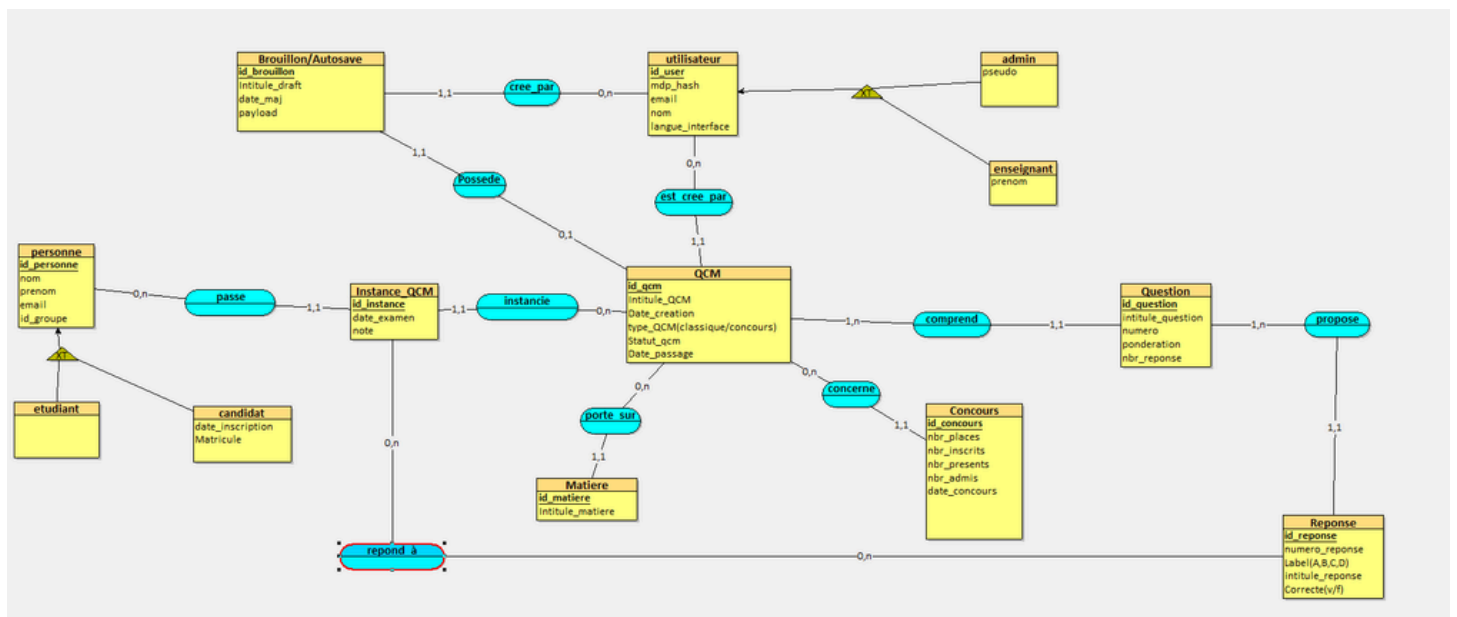


Figure 3 : MCD de la base de données

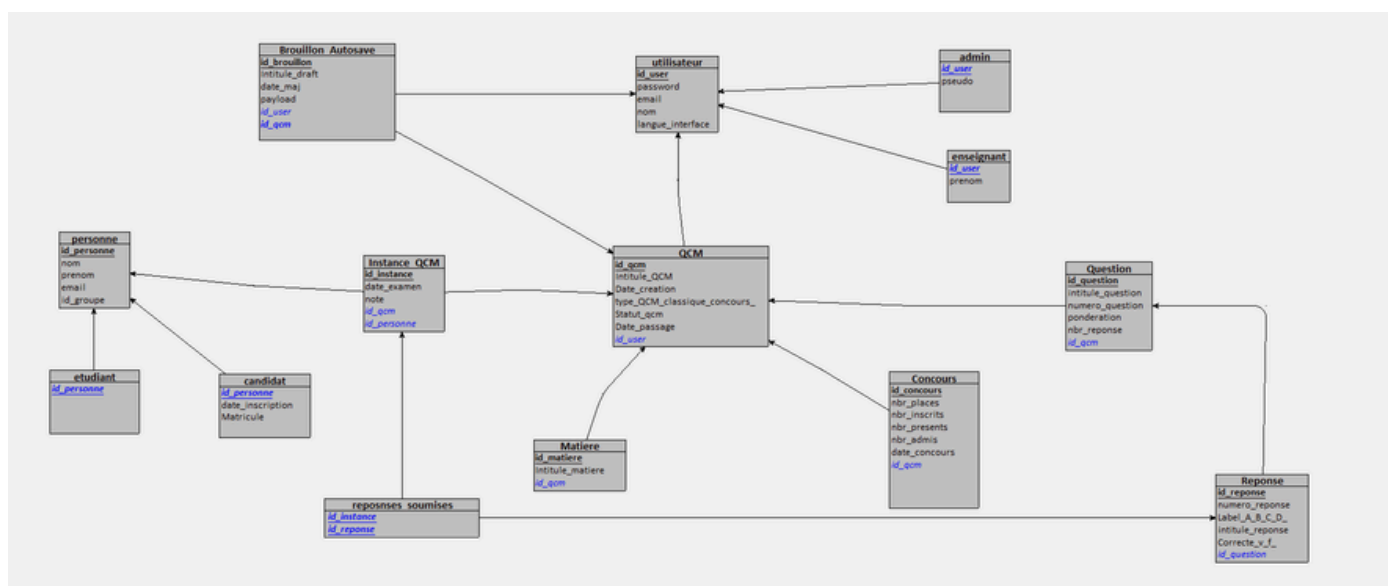


Figure 4 : MLD de la base de données

5-Traitement d'image et détection IA

Le traitement d'image représente une partie centrale de notre projet, car il permet de transformer une simple photo d'une feuille de réponses en informations exploitables par le système. L'objectif est d'identifier automatiquement l'étudiant, de détecter les réponses cochées et de transmettre ces informations au backend afin de calculer le score final.

Dans notre application, l'enseignant photographie la feuille de réponses à l'aide de l'application mobile. Cette image est ensuite envoyée au service de traitement d'image. Le programme analyse la feuille en plusieurs étapes : détection des repères, correction de perspective, extraction des zones utiles, reconnaissance du nom, détection du numéro étudiant et détection des réponses cochées.

Le traitement repose principalement sur OpenCV pour les opérations de vision par ordinateur, EasyOCR pour la reconnaissance du nom et des modèles d'intelligence artificielle entraînés pour classer les cases comme cochées ou non cochées.

5.1 Notions générales sur le traitement d'images :

Le traitement d'image est un domaine de l'informatique qui consiste à analyser, modifier ou améliorer une image numérique afin d'en extraire des informations utiles. Dans notre projet, il permet de passer d'une photo brute d'une feuille QCM à une structure de données contenant le nom de l'étudiant, son numéro et ses réponses.

Une image numérique est composée de pixels, c'est-à-dire de petites unités contenant une information de couleur ou de luminosité. Chaque pixel participe à la construction visuelle de l'image. Dans une image en couleur, un pixel est généralement représenté par trois composantes : rouge, vert et bleu. Dans une image en niveaux de gris, il est représenté par une seule valeur d'intensité lumineuse.

Avant de détecter les réponses, l'image doit être préparée. Une photo prise avec un smartphone peut présenter plusieurs problèmes : inclinaison, mauvaise luminosité, ombres, bruit, flou ou déformation de perspective. Ces problèmes peuvent empêcher une bonne détection des cases. C'est pour cela que le service applique plusieurs opérations de prétraitement.

Les principales notions utilisées dans notre projet sont :

- la conversion en niveaux de gris
- le seuillage ou binarisation
- le filtrage par flou gaussien
- la détection de contours
- la correction de perspective
- l'extraction des zones d'intérêt
- la classification des cases par un modèle d'intelligence artificielle

La conversion en niveaux de gris permet de simplifier l'image en supprimant l'information de couleur. Le seuillage permet de transformer l'image en noir et blanc afin de mieux séparer le fond de la feuille et les éléments remplis par l'étudiant. Le flou gaussien permet de réduire le bruit et d'améliorer la qualité de l'image avant l'analyse. La détection de contours permet de trouver les repères, les cadres et les cases. La correction de perspective permet de redresser la feuille afin d'obtenir une image proche d'un scan. L'extraction des zones d'intérêt permet de découper uniquement les parties utiles, comme le nom, le numéro étudiant et les réponses. Enfin, la classification par modèle d'intelligence artificielle permet de déterminer automatiquement si une case est cochée ou non cochée.



Figure 5 : Exemple de conversion en niveaux de gris

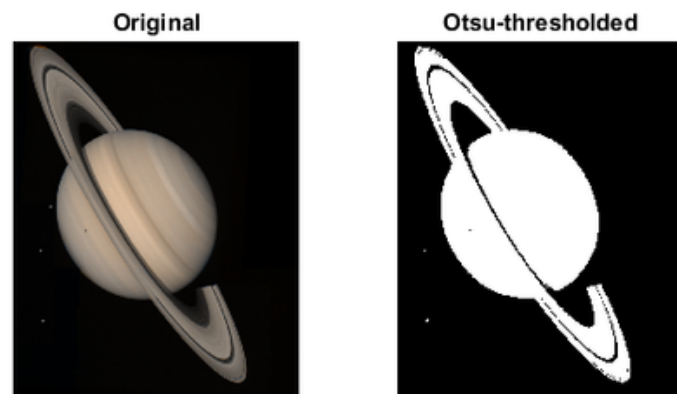


Figure 6 : Exemple de seuillage d'Otsu

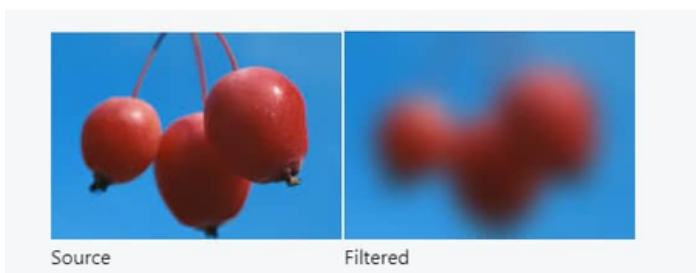


Figure 7 : Exemple de flou Gaussien

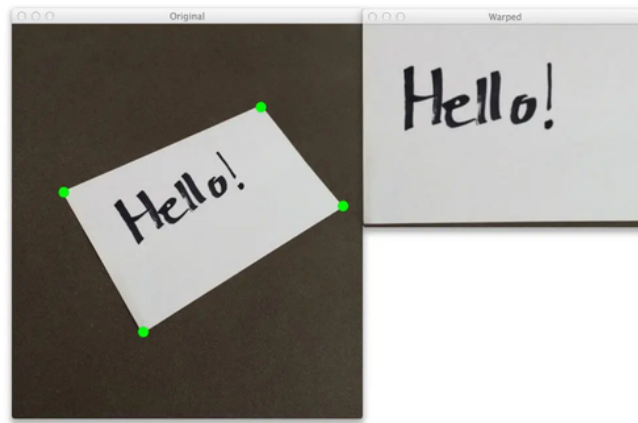


Figure 8 : Exemple de correction de perspective

5.2 Structure de la feuille de réponse :

La feuille de réponse utilisée dans notre projet est standardisée. Cette standardisation est importante, car le programme doit connaître à l'avance l'emplacement des différentes zones à analyser.

La feuille contient principalement trois parties :

- une zone d'identification
- une zone du numéro étudiant
- une zone des réponses QCM

La zone d'identification contient le nom complet de l'étudiant. Cette zone est utilisée par le module OCR afin d'extraire le nom et le prénom écrits sur la feuille.

La zone du numéro étudiant est située dans la partie supérieure droite de la feuille. Elle contient une grille de cases permettant de coder le numéro de l'étudiant. Le principe utilisé est celui du codage One-Hot : pour chaque colonne, l'étudiant coche une seule case correspondant à un chiffre. En lisant les cases cochées colonne par colonne, le système reconstruit le numéro étudiant.

La zone des réponses QCM contient les questions et les choix de réponses. Dans notre projet, le système est prévu pour traiter jusqu'à 60 questions. Chaque question possède 4 choix possibles : A, B, C et D. Les réponses sont organisées sous forme de cases, ce qui facilite leur détection automatique.

La feuille contient aussi quatre repères placés aux coins. Ces repères sont très importants, car ils permettent au système de détecter l'orientation de la feuille et d'appliquer une correction de perspective. Sans ces repères, une photo prise en biais pourrait rendre la détection des cases imprécise.

La structure fixe de la feuille permet donc au programme de travailler avec des zones prédéfinies. Après redressement de l'image, le système peut découper directement les régions correspondant au nom, au numéro étudiant et aux réponses.



EXAMEN QCM

NUMÉRO ÉTUDIANT

0	0	0	0	0
1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4
5	5	5	5	5
6	6	6	6	6
7	7	7	7	7
8	8	8	8	8
9	9	9	9	9

MATIÈRE :	
NOM :	
PRÉNOM :	
DATE : / /	NIVEAU :

	A	B	C	D		A	B	C	D		A	B	C	D
1.	☐	☐	☐	☐	21.	☐	☐	☐	☐	41.	☐	☐	☐	☐
2.	☐	☐	☐	☐	22.	☐	☐	☐	☐	42.	☐	☐	☐	☐
3.	☐	☐	☐	☐	23.	☐	☐	☐	☐	43.	☐	☐	☐	☐
4.	☐	☐	☐	☐	24.	☐	☐	☐	☐	44.	☐	☐	☐	☐
5.	☐	☐	☐	☐	25.	☐	☐	☐	☐	45.	☐	☐	☐	☐
6.	☐	☐	☐	☐	26.	☐	☐	☐	☐	46.	☐	☐	☐	☐
7.	☐	☐	☐	☐	27.	☐	☐	☐	☐	47.	☐	☐	☐	☐
8.	☐	☐	☐	☐	28.	☐	☐	☐	☐	48.	☐	☐	☐	☐
9.	☐	☐	☐	☐	29.	☐	☐	☐	☐	49.	☐	☐	☐	☐
10.	☐	☐	☐	☐	30.	☐	☐	☐	☐	50.	☐	☐	☐	☐
11.	☐	☐	☐	☐	31.	☐	☐	☐	☐	51.	☐	☐	☐	☐
12.	☐	☐	☐	☐	32.	☐	☐	☐	☐	52.	☐	☐	☐	☐
13.	☐	☐	☐	☐	33.	☐	☐	☐	☐	53.	☐	☐	☐	☐
14.	☐	☐	☐	☐	34.	☐	☐	☐	☐	54.	☐	☐	☐	☐
15.	☐	☐	☐	☐	35.	☐	☐	☐	☐	55.	☐	☐	☐	☐
16.	☐	☐	☐	☐	36.	☐	☐	☐	☐	56.	☐	☐	☐	☐
17.	☐	☐	☐	☐	37.	☐	☐	☐	☐	57.	☐	☐	☐	☐
18.	☐	☐	☐	☐	38.	☐	☐	☐	☐	58.	☐	☐	☐	☐
19.	☐	☐	☐	☐	39.	☐	☐	☐	☐	59.	☐	☐	☐	☐
20.	☐	☐	☐	☐	40.	☐	☐	☐	☐	60.	☐	☐	☐	☐

Instructions : Noircissez la case ou cochez-la avec soin. Ne pas plier la feuille.

Figure 9 : Structure de la feuille de réponse QCM

5.3 Prétraitement sur la feuille :

Le prétraitement est la première étape appliquée à l'image après son acquisition. Son objectif est de rendre la feuille exploitable par les modules de détection. Lorsqu'une image est chargée, le programme commence par vérifier qu'elle est valide. Si l'image ne peut pas être lue, une erreur est retournée. Ensuite, le système recherche les quatre repères de coin de la feuille.

La détection des repères se fait en plusieurs étapes. Le programme convertit d'abord l'image en niveaux de gris. Ensuite, il définit quatre zones de recherche correspondant aux coins de la feuille : haut gauche, haut droit, bas droit et bas gauche. Dans chaque zone, il applique un seuillage afin de faire ressortir les éléments sombres. Les contours sont ensuite détectés avec OpenCV. Le programme sélectionne le contour le plus pertinent dans chaque coin afin d'obtenir les quatre points nécessaires au redressement de la feuille.

Une fois les quatre points détectés, ils sont ordonnés dans l'ordre suivant :

- coin supérieur gauche
- coin supérieur droit
- coin inférieur droit
- coin inférieur gauche

Cette étape est indispensable, car la transformation de perspective a besoin de connaître correctement la position de chaque point.

Après cela, le programme applique une homographie. L'homographie est une transformation géométrique qui permet de redresser une image prise avec un angle. Elle transforme la feuille photographiée en une image rectangulaire, comme si elle avait été scannée à plat.

Le résultat est ensuite redimensionné à une taille fixe. Dans notre code, l'image redressée est ramenée à une dimension standard. Cette étape permet d'utiliser des coordonnées fixes pour extraire les zones de la feuille. Grâce à cela, les emplacements du nom, du numéro étudiant et des réponses restent toujours les mêmes après traitement.

Le prétraitement comprend aussi des opérations utilisées dans certaines zones spécifiques, comme le flou gaussien et le seuillage d'Otsu. Le flou gaussien permet de réduire le bruit dans l'image. Le seuillage d'Otsu permet de convertir automatiquement une zone en image binaire en choisissant un seuil adapté.

Ainsi, le prétraitement prépare la feuille à l'analyse. Il corrige les problèmes de perspective, réduit le bruit et permet d'obtenir une image normalisée, plus facile à analyser par les modèles de détection.

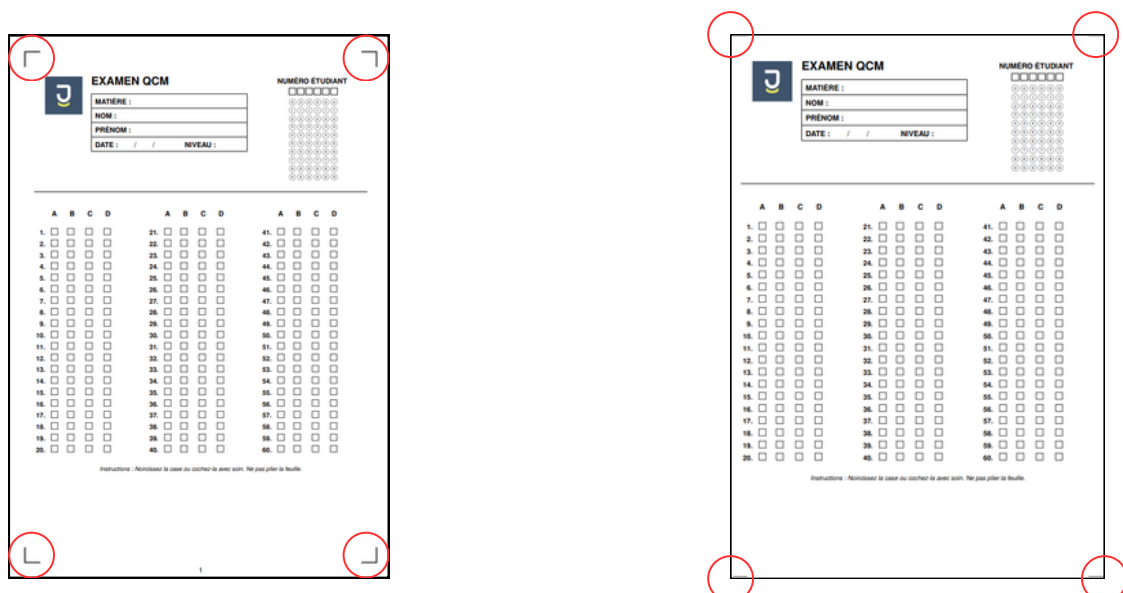


Figure 10 : Transformation de perspective de la feuille de réponse

5.4 Création des modèles IA et constitution du dataset :

Dans notre projet, la détection des cases cochées ne repose pas uniquement sur des méthodes classiques de traitement d'image. Nous avons également utilisé des modèles d'intelligence artificielle entraînés à partir d'un dataset que nous avons constitué nous-mêmes.

Cette étape est importante, car les réponses des étudiants peuvent être cochées de différentes manières. Certains étudiants noircissent complètement la case, d'autres mettent une croix, une coche ou remplissent seulement une partie de la case. Une simple règle basée uniquement sur le nombre de pixels noirs peut donc parfois produire des erreurs. Pour améliorer la fiabilité de la détection, nous avons entraîné des modèles capables de reconnaître automatiquement si une case est cochée ou non cochée.

Nous avons constitué notre propre dataset à partir de plus de 40 000 exemples de cases extraites des feuilles de réponses. Ces exemples ont été séparés en deux catégories principales :

- cases cochées
- cases non cochées

Le dataset a ensuite été utilisé pour entraîner des modèles de classification. Le rôle de ces modèles est de recevoir une image de petite taille représentant une case, puis de prédire si cette case est cochée ou vide.

Dans notre système, nous avons utilisé deux modèles différents :

- un modèle pour la détection des réponses QCM
- un modèle pour la détection des cases du numéro étudiant

Cette séparation permet d'adapter chaque modèle à son type de case. En effet, les cases des réponses QCM et les cases du numéro étudiant n'ont pas exactement la même structure ni le même contexte visuel sur la feuille. Le modèle du numéro étudiant est spécialisé dans la lecture de la grille d'identification, tandis que le modèle QCM est spécialisé dans la détection des choix A, B, C et D.

Avant d'être envoyée au modèle, chaque case extraite est redimensionnée à une taille fixe. Dans notre implémentation, les images sont redimensionnées en 64×64 pixels, puis normalisées afin que les valeurs des pixels soient comprises entre 0 et 1. Cette normalisation permet au modèle de traiter les images de manière plus stable.

Le modèle retourne ensuite une valeur de prédiction. Cette valeur est comparée à un seuil afin de décider si la case est cochée ou non. Dans notre logique, une prédiction inférieure ou égale au seuil indique que la case est considérée comme cochée.

Pour renforcer la robustesse de la détection, nous avons combiné la prédiction du modèle avec une mesure classique appelée ratio de pixels noirs. Ce ratio représente la proportion de pixels sombres dans la case. Ainsi, lorsqu'une prédiction est proche du seuil, le ratio de pixels noirs peut confirmer ou non la présence d'une marque dans la case.

Cette combinaison entre intelligence artificielle et traitement d'image classique permet d'obtenir une détection plus fiable. Le modèle apprend à reconnaître les formes de cochage à partir du dataset, tandis que le ratio de pixels noirs apporte une vérification supplémentaire basée sur l'intensité visuelle de la case.

Le processus général utilisé pour créer et exploiter les modèles est le suivant :

1. Collecte des exemples de cases à partir des feuilles de réponses
2. Séparation des images en cases cochées et non cochées
3. Préparation du dataset
4. Redimensionnement des images
5. Normalisation des pixels
6. Entraînement du modèle de classification
7. Sauvegarde du modèle entraîné
8. Chargement du modèle dans le service de traitement d'image
9. Prédiction sur chaque case extraite
10. Décision finale : case cochée ou non cochée

Grâce à cette approche, le système devient plus flexible qu'une méthode basée uniquement sur des seuils fixes. Les modèles IA permettent de mieux gérer les variations d'écriture et de remplissage des cases par les étudiants.

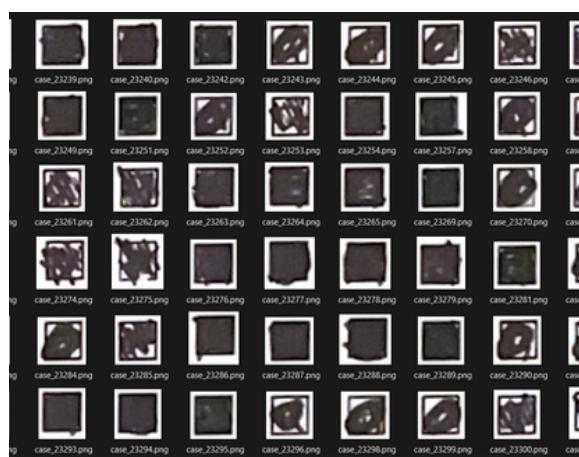
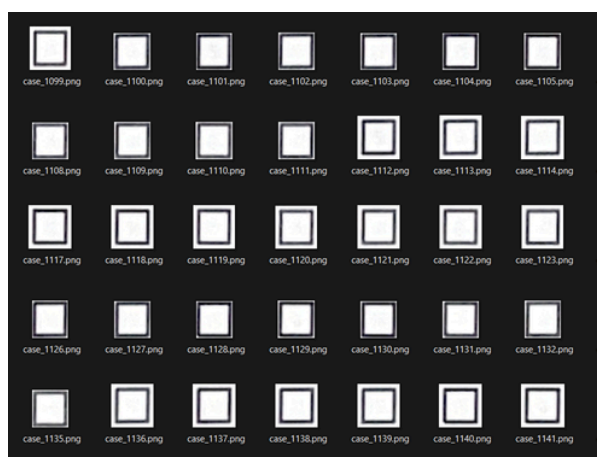


Figure 11 : Extrait du dataset des cases de réponses et du numéro étudiant

5.5 Détection du numéro étudiant :

La détection du numéro étudiant est réalisée à partir d'une zone précise de la feuille. Après redressement de l'image, le programme découpe la partie supérieure droite où se trouve la grille du numéro étudiant.

Cette grille fonctionne selon le principe One-Hot. Chaque colonne représente un chiffre du numéro étudiant. Dans chaque colonne, l'étudiant doit cocher une seule case parmi les chiffres proposés. Le système lit ensuite les colonnes de gauche à droite pour reconstruire le numéro complet.

Le traitement commence par l'extraction de la zone du numéro étudiant. Cette zone est ensuite convertie en niveaux de gris. Un flou gaussien est appliqué afin de réduire le bruit, puis un seuillage binaire inversé est utilisé pour faire ressortir les cases et les marques noires.

Après cette étape, le programme utilise la détection de contours pour trouver les cases présentes dans la grille. Les contours trop petits ou non pertinents sont ignorés. Les zones détectées sont ensuite découpées une par une afin d'être analysées.

Pour savoir si une case est cochée ou non, le système utilise le modèle d'intelligence artificielle entraîné spécialement pour cette tâche. Chaque case est redimensionnée puis normalisée avant d'être envoyée au modèle. Le modèle retourne une valeur de prédiction. Si cette valeur est inférieure ou égale à un seuil défini, la case est considérée comme cochée. Dans cette partie, le modèle utilisé pour classer les cases du numéro étudiant a été entraîné à partir de notre propre dataset. Nous avons extrait plusieurs exemples de cases de la grille d'identification, puis nous les avons classés manuellement en deux catégories : cochée et non cochée. Ce dataset a permis d'apprendre au modèle les différentes formes possibles de remplissage d'une case. Grâce à cela, la lecture du numéro étudiant ne dépend pas uniquement d'un seuil de noirceur, mais aussi d'une prédiction issue d'un modèle IA spécialisé.

Après classification des cases, le programme regroupe les cases par colonnes selon leur position horizontale. Chaque colonne est ensuite triée verticalement pour retrouver l'ordre des chiffres. Le chiffre retenu est celui correspondant à la case cochée dans la colonne.

Si aucune case n'est détectée dans une colonne, le programme peut retourner un caractère d'incertitude. Cela permet de signaler que le numéro étudiant n'a pas été lu correctement.

Finalement, les chiffres détectés sont concaténés pour former le numéro étudiant complet. Ce numéro est ensuite retourné avec le reste des informations extraites de la feuille.

Cette méthode est importante, car le numéro étudiant permet d'identifier l'étudiant de manière plus fiable que le nom manuscrit. Même si l'OCR donne un résultat imparfait, le numéro étudiant peut être utilisé pour retrouver l'étudiant dans la base de données.

Figure 12 : Grille numéro étudiant vide

Figure 13 : Grille numéro étudiant remplie et scannée

5.6 Détection des réponses cochées :

La détection des réponses cochées est l'étape qui permet de lire les réponses de l'étudiant pour chaque question du QCM.

Dans notre projet, le système traite jusqu'à 60 questions. Chaque question possède quatre choix possibles : A, B, C et D. Le but du programme est de déterminer, pour chaque question, quelle case a été cochée.

La détection des réponses repose sur un fichier de coordonnées. Ce fichier contient les positions des cases de réponse sur la feuille standardisée. Après redressement de l'image, ces coordonnées deviennent fiables, car la feuille est toujours ramenée à la même taille et à la même orientation.

Le programme commence par charger les coordonnées des cases. Ensuite, il organise ces cases en lignes. Les cases d'une même ligne sont regroupées selon leur position verticale. Chaque ligne est ensuite divisée en groupes de quatre cases correspondant aux choix A, B, C et D d'une question.

Pour chaque case, le système extrait une petite zone autour de la position prévue. Un léger padding est ajouté afin de ne pas couper la case. Ensuite, le programme applique plusieurs traitements :

- conversion de la zone en niveaux de gris
- flou gaussien
- seuillage binaire inversé avec Otsu
- détection de contours
- extraction du meilleur contour correspondant à la case

	A	B	C	D		A	B	C	D		A	B	C	D
1.	☐	☐	☐	☐	21.	☐	☐	☐	☐	41.	☐	☐	☐	☐
2.	☐	☐	☐	☐	22.	☐	☐	☐	☐	42.	☐	☐	☐	☐
3.	☐	☐	☐	☐	23.	☐	☐	☐	☐	43.	☐	☐	☐	☐
4.	☐	☐	☐	☐	24.	☐	☐	☐	☐	44.	☐	☐	☐	☐
5.	☐	☐	☐	☐	25.	☐	☐	☐	☐	45.	☐	☐	☐	☐
6.	☐	☐	☐	☐	26.	☐	☐	☐	☐	46.	☐	☐	☐	☐
7.	☐	☐	☐	☐	27.	☐	☐	☐	☐	47.	☐	☐	☐	☐
8.	☐	☐	☐	☐	28.	☐	☐	☐	☐	48.	☐	☐	☐	☐
9.	☐	☐	☐	☐	29.	☐	☐	☐	☐	49.	☐	☐	☐	☐
10.	☐	☐	☐	☐	30.	☐	☐	☐	☐	50.	☐	☐	☐	☐
11.	☐	☐	☐	☐	31.	☐	☐	☐	☐	51.	☐	☐	☐	☐
12.	☐	☐	☐	☐	32.	☐	☐	☐	☐	52.	☐	☐	☐	☐
13.	☐	☐	☐	☐	33.	☐	☐	☐	☐	53.	☐	☐	☐	☐
14.	☐	☐	☐	☐	34.	☐	☐	☐	☐	54.	☐	☐	☐	☐
15.	☐	☐	☐	☐	35.	☐	☐	☐	☐	55.	☐	☐	☐	☐
16.	☐	☐	☐	☐	36.	☐	☐	☐	☐	56.	☐	☐	☐	☐
17.	☐	☐	☐	☐	37.	☐	☐	☐	☐	57.	☐	☐	☐	☐
18.	☐	☐	☐	☐	38.	☐	☐	☐	☐	58.	☐	☐	☐	☐
19.	☐	☐	☐	☐	39.	☐	☐	☐	☐	59.	☐	☐	☐	☐
20.	☐	☐	☐	☐	40.	☐	☐	☐	☐	60.	☐	☐	☐	☐

Figure 14 : Extraction des cases

La zone de la case est ensuite envoyée au modèle de classification QCM. Ce modèle retourne une prédiction indiquant si la case est cochée ou non. Pour la détection des réponses A, B, C et D, nous avons également utilisé un modèle IA entraîné avec notre propre dataset. Ce dataset contient des exemples de cases de réponses extraites des feuilles QCM. L'objectif du modèle est de distinguer automatiquement une case cochée d'une case vide, même lorsque le remplissage n'est pas parfait. Cette approche est plus robuste qu'une simple comparaison de pixels, car elle prend en compte différentes manières de cocher une réponse : case noircie, croix, coche ou remplissage partiel.

Après analyse des quatre cases d'une question, le programme construit la réponse détectée. Si la case A est cochée, la réponse sera "A". Si plusieurs cases sont détectées, le programme peut retourner plusieurs lettres, par exemple "AC". Si aucune case n'est cochée, la réponse reste vide.

À la fin du traitement, le programme retourne un dictionnaire contenant toutes les questions, de q1 jusqu'à q60. Chaque question est associée à la réponse détectée. Les questions non détectées sont complétées automatiquement par une valeur vide afin de garantir que le résultat final possède toujours la même structure.

Cette étape permet donc de transformer les cases cochées par l'étudiant en données numériques directement exploitables par le backend pour le calcul de la note.

5.7 Détection du nom complet par OCR :

L'OCR, ou reconnaissance optique de caractères, est utilisé dans notre projet pour lire le nom complet de l'étudiant écrit sur la feuille de réponse.

Après le redressement de la feuille, le programme commence par localiser la zone d'identification. Cette zone se trouve dans la partie supérieure de la feuille. Le système recherche d'abord le cadre contenant les informations de l'étudiant. Pour cela, il découpe une région approximative dans la partie haute de la feuille, la convertit en niveaux de gris, applique un flou gaussien, puis utilise un seuillage binaire inversé.

Ensuite, une opération morphologique de fermeture est appliquée afin de regrouper les éléments proches et de mieux détecter le cadre d'identification. Les contours sont ensuite analysés. Le programme sélectionne le meilleur contour selon sa taille et son rapport largeur/hauteur. Si aucun cadre n'est détecté, une erreur est retournée, car cela signifie que la feuille n'a pas pu être lue correctement.

Une fois la zone d'identification trouvée, le programme découpe plus précisément le champ du nom complet. Cette zone est ensuite préparée pour améliorer la lecture OCR.

Le prétraitement du texte comprend plusieurs étapes :

- conversion en niveaux de gris
- agrandissement de l'image
- application d'un flou gaussien
- seuillage avec la méthode d'Otsu
- suppression des lignes horizontales
- ajout d'une bordure blanche autour du texte

L'agrandissement permet à l'OCR de mieux distinguer les caractères. La suppression des lignes horizontales est utile lorsque le nom est écrit sur une ligne imprimée, car cette ligne peut perturber la reconnaissance. L'ajout d'une bordure blanche évite que le texte soit collé au bord de l'image, ce qui améliore la lecture.

La reconnaissance est ensuite réalisée avec EasyOCR. Dans notre code, EasyOCR est configuré avec les langues française et anglaise. Le programme limite les caractères acceptés aux lettres majuscules, minuscules et espaces, car le nom complet ne doit pas contenir de chiffres ou de symboles.

Après la lecture OCR, le texte obtenu est nettoyé. Les caractères non alphabétiques sont supprimés et les espaces inutiles sont corrigés. Le résultat final est une chaîne de caractères contenant le nom complet détecté.

Le programme essaie plusieurs variantes de l'image du nom : une version prétraitée, une version brute avec bordure et une version nettoyée. Cette stratégie augmente les chances de lecture correcte, car certaines images sont mieux reconnues sans traitement excessif.

L'OCR n'est pas toujours parfait, surtout avec l'écriture manuscrite. Pour cette raison, le nom reconnu ne doit pas être utilisé seul. Il peut être combiné avec le numéro étudiant afin de fiabiliser l'identification de l'étudiant. Le numéro étudiant reste généralement plus fiable, tandis que le nom peut servir de vérification supplémentaire.

MATIÈRE :	
NOM :	
PRÉNOM :	
DATE : / /	NIVEAU :

Figure 15 : Zone d'identification du nom complet vide

MATIÈRE :	
NOM : JORIO	
PRÉNOM : LINA	
DATE : / /	NIVEAU :

Figure 16 : Zone d'identification du nom complet remplie et scannée

5.8 Diagramme d'activité :

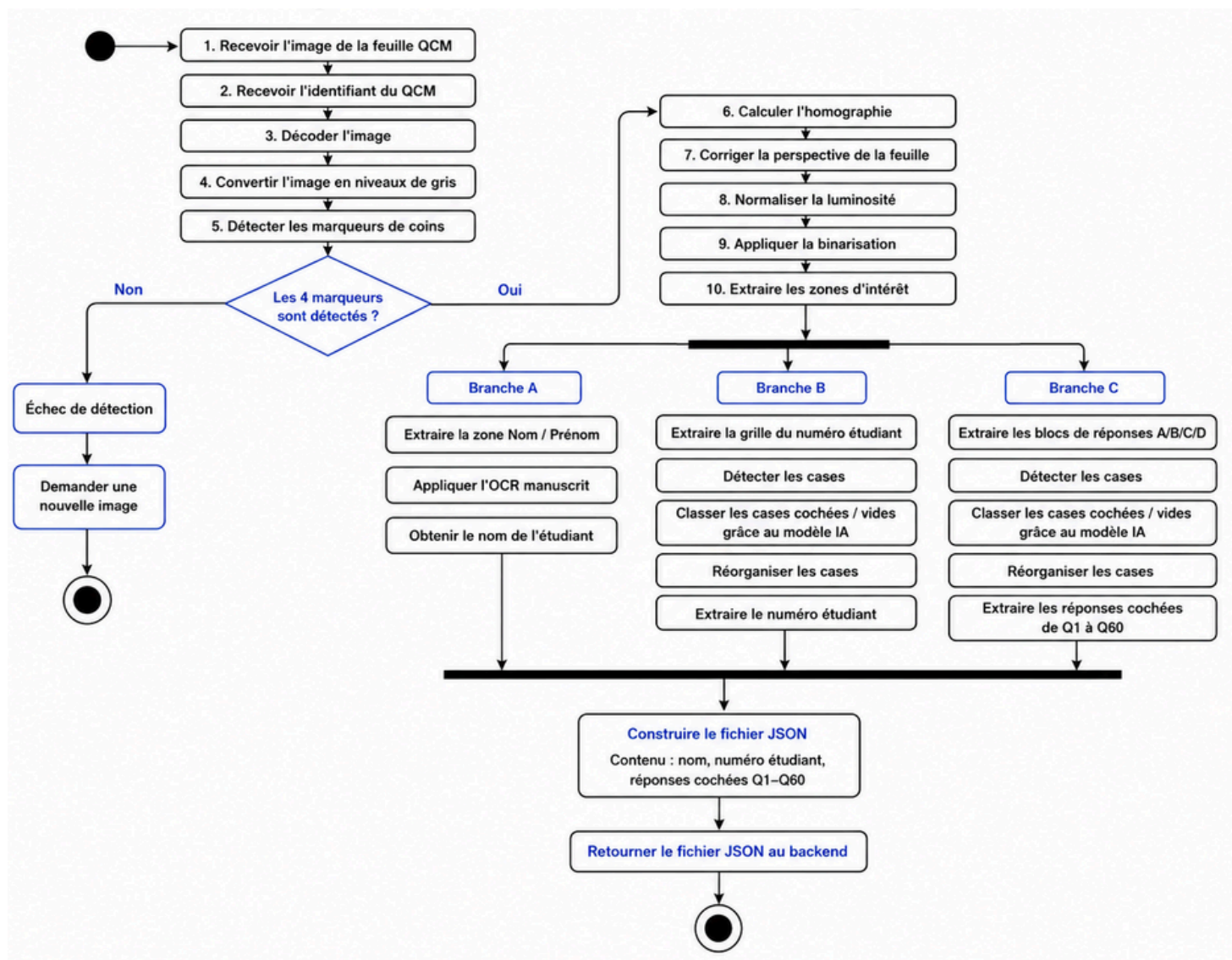


Figure 17 : Diagramme d'activité

6- Réalisation de l'application web

6.1 Présentation :

L'application web constitue l'interface principale de gestion du projet QCM Corrector. Elle a été développée pour permettre aux enseignants et à l'administrateur de piloter l'ensemble du processus lié aux QCM : création des évaluations, publication, génération des documents PDF, suivi des sessions de correction, consultation des résultats, gestion des classes et envoi des notes aux étudiants par mail.

Cette application représente la partie administrative et pédagogique de la solution. Elle se distingue de l'application mobile, qui intervient surtout dans la capture ou l'envoi des réponses détectées. L'application web, quant à elle, permet de préparer les QCM, d'organiser les classes, de suivre les corrections et de centraliser les résultats dans une interface unique.

L'accès à la plateforme se fait à partir d'une page de connexion. Deux profils principaux existent : administrateur et professeur. Après authentification, l'affichage de l'application s'adapte automatiquement au rôle de l'utilisateur connecté. L'administrateur possède une vision globale de la plateforme, tandis qu'un professeur dispose d'un accès limité aux classes, matières, QCM, sessions et résultats qui lui sont associés.

L'administrateur peut consulter l'ensemble des données de l'application. Il peut gérer les classes et les étudiants, ajouter une nouvelle classe, modifier le nom d'une classe, importer une liste d'étudiants depuis un fichier CSV ou Excel, ajouter ou supprimer un étudiant, et consulter toutes les sessions de correction. Il peut également créer un QCM à la place d'un professeur en sélectionnant le professeur, la matière et la classe concernés. Cette fonctionnalité permet à l'administration de préparer des évaluations pour différents enseignants tout en conservant une organisation cohérente avec la base de données.

QCM CORRECTOR - WEB

Administrateur - Admin Plateforme

ADMIN

Classes & etudiants

Admin Plateforme - groupes, matieres rattachees, identifiants et emails.

Rechercher

Classe, nom, email ou identifiant

CLASSES

3

ETUDIANTS

29

SOURCE

DB

CREER CLASSE

Nom classe

Ex: JM4

Niveau

Ex: JM4

Annee scolaire

2025-2026

Creer classe

IMPORTER CLASSE

Nom classe importee

Ex: JM4

Niveau

Ex: JM4

Annee scolaire

2025-2026

Fichier CSV / Excel

Choose File

No file chosen

Colonnes acceptees: numero_etudiant, prenom, nom, email.

Importer classe

2025-2026

JM1

10 actifs

Ajouter etudiant

Changer nom classe

Supprimer classe

QCM CORRECTOR - WEB

Administrateur - Admin Plateforme

Accueil Principal Profs

ADMINISTRATION

Professeurs

Gestion reservee a Admin Plateforme.

Ajouter profil prof

Gerer les profs

PROFESSEUR	MATIERE	CLASSE	QCM	SESSIONS	ACTIONS
Youssef El Amrani youssef.elamrani@ecole.ma	Algorithmique	JM1	4	3	Gerer Supprimer profil prof
Salma Berrada salma.berrada@ecole.ma	Reseaux informatique	JM2	2	2	Gerer Supprimer profil prof
Karim Mansouri karim.mansouri@ecole.ma	Securite informatique	JM3	3	3	Gerer Supprimer profil prof

Figure 18 : Actions de l'administrateur sur les classes, professeurs et étudiants

Le professeur, de son côté, utilise principalement l'application pour créer et suivre ses propres QCM. Il peut renseigner les informations générales de l'évaluation, ajouter les questions, définir les propositions de réponses, choisir les bonnes réponses et publier le QCM. Il peut ensuite consulter les sessions associées, visualiser les résultats des étudiants et envoyer les notes par mail. Son périmètre est volontairement limité à ses propres affectations afin d'éviter l'accès à des données qui ne le concernent pas.

CREATION QCM

Reseaux CC3

Autosave actif: Admin Plateforme.

Enregistrer brouillon

Publier

Annuler

INFORMATIONS GENERALES DU QCM

Professeur / matiere / classe

Youssef El Amrani - Algorithmique - JM1

Professeur responsable

youssef.elamrani@ecole.ma

Titre du QCM

Reseaux CC3

Matiere

Algorithmique

Date d'examen

05/31/2026

Niveau / promotion

JM1

Classe / groupe

JM1

Code QCM

QCM-2026-9279

NOMBRE DE QUESTIONS

-

3

+

Entre 1 et 60 questions.

NOMBRE DE CHOIX PAR QUESTION

4 choix: A, B, C, D

Fixe

Impose par le format de la feuille de reponses standardisee.

Annuler

Enregistrer brouillon

Suivant: Questions

Figure 19 : Page de création d'un QCM par un administrateur pour un professeur

La création d'un QCM se fait à travers un assistant organisé en plusieurs étapes. La première étape concerne les informations générales du QCM : titre, matière, classe, professeur responsable, date d'examen, code du QCM et nombre de questions. La deuxième étape permet de saisir les questions, les choix de réponses, le type de question, les bonnes réponses et la pondération. La troisième étape présente un récapitulatif complet avant publication. Cette organisation permet de guider l'utilisateur et de réduire les erreurs de saisie.

Après publication, l'application permet de générer le sujet PDF et la grille PDF des bonnes réponses. Le sujet PDF est destiné à être utilisé comme support d'évaluation, tandis que la grille PDF permet de vérifier les réponses attendues. Les bonnes réponses y sont clairement indiquées, notamment par des croix dans la grille de correction.

L'application web permet aussi de suivre les sessions de correction. Une session correspond à une évaluation donnée, associée à un QCM, une classe, une matière et un professeur. Depuis l'interface, l'utilisateur peut consulter le nombre de copies reçues, la moyenne, les corrections nécessitant une vérification manuelle, ainsi que le statut d'envoi des mails.

La consultation des résultats est une partie essentielle de l'application. Les réponses détectées par l'application mobile ou par le service IA sont envoyées au backend, enregistrées dans la base de données, puis affichées dans l'application web. Pour chaque étudiant, l'interface affiche l'identifiant, le nom, l'adresse mail, le score obtenu, le nombre de questions correctes, les réponses détectées, le statut de correction et la date de réception.

Enfin, l'application web intègre une fonctionnalité d'envoi des résultats par mail. Une fois les résultats disponibles, l'administrateur ou le professeur peut déclencher l'envoi automatique des notes aux étudiants. Le système prépare alors un mail personnalisé contenant les informations de l'étudiant, la matière concernée, le QCM et la note obtenue. Cette fonctionnalité permet d'automatiser la communication des résultats et d'éviter l'envoi manuel des notes.

En résumé, l'application web est le point de contrôle principal de la solution QCM Corrector. Elle relie les besoins pédagogiques des enseignants, les besoins administratifs de l'établissement et les traitements techniques réalisés par le backend. Elle permet de passer d'une simple correction automatisée à une plateforme complète de gestion, de suivi et de communication des résultats.

6.2 Architecture de l'application web :

L'application web de QCM Corrector a été conçue selon une architecture front-end organisée, modulaire et connectée à une API backend. Elle repose sur une séparation claire entre les pages, les composants d'interface, la gestion des données et la communication avec le backend. Cette organisation permet de rendre l'application plus maintenable et plus facile à faire évoluer.

L'architecture générale de l'application web peut être représentée comme une couche d'interface située entre l'utilisateur et le backend. L'utilisateur interagit avec les pages web, puis l'application envoie des requêtes HTTP à l'API backend pour récupérer ou enregistrer les données. Le backend se charge ensuite de communiquer avec la base de données PostgreSQL et de renvoyer les informations nécessaires au front-end.

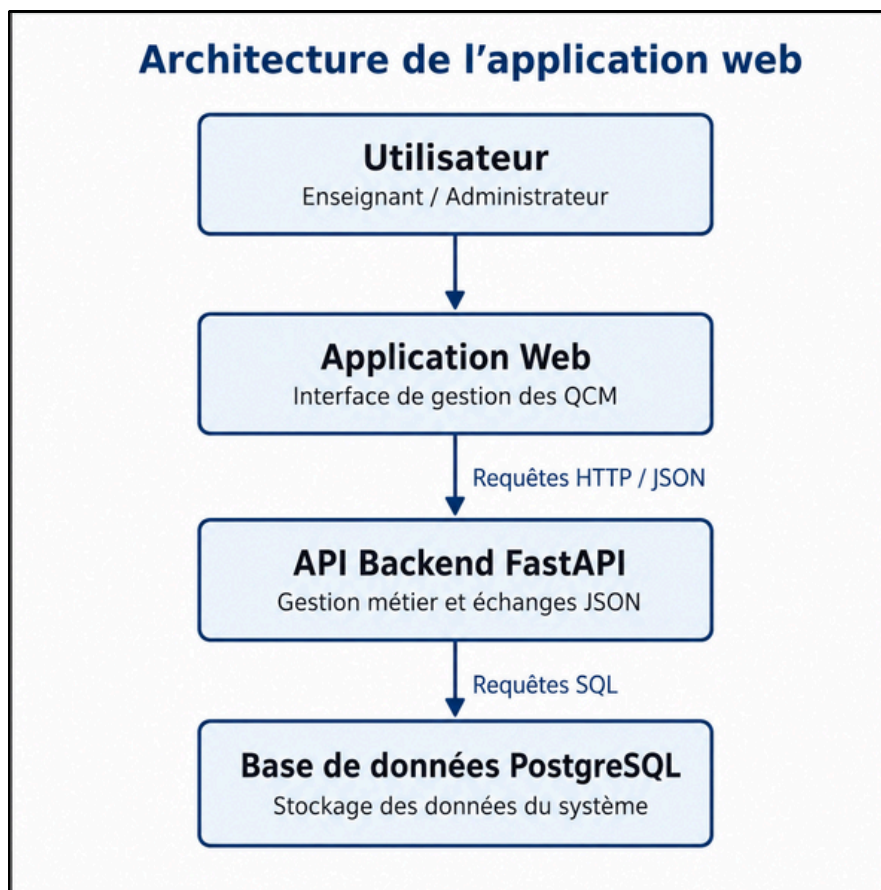


Figure 20 : Communication entre l'utilisateur, l'application web, l'API Backend et la base PostgreSQL

Dans cette architecture, l'application web ne se connecte jamais directement à la base de données. Elle passe toujours par l'API backend. Ce choix permet de centraliser les règles métier dans le backend et de garder le front-end concentré sur l'affichage, la navigation et l'interaction avec l'utilisateur.

L'application web est organisée autour de plusieurs pages principales. Chaque page correspond à une fonctionnalité précise du système. La page de connexion permet à l'utilisateur de choisir son profil et de s'authentifier. Le tableau de bord donne une vue d'ensemble des données importantes.

La page des QCM permet de consulter les évaluations disponibles. La page de création permet de créer un nouveau QCM. La page des sessions permet de consulter les sessions de correction. La page de détail d'une session permet d'afficher les résultats des étudiants. Enfin, les pages d'administration permettent de gérer les classes, les étudiants et les professeurs.

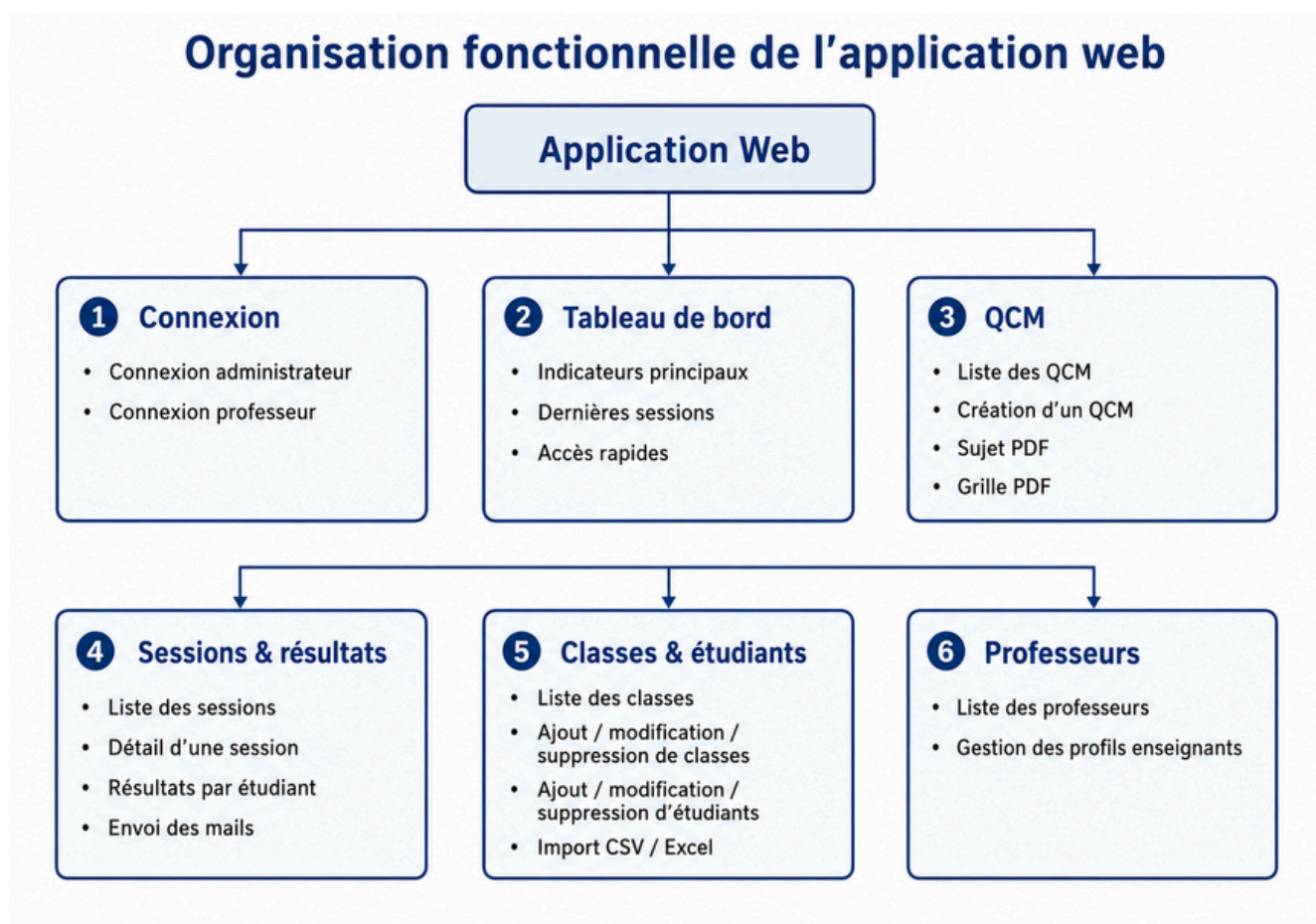


Figure 21 : Organisation fonctionnelle des principales rubriques de l'application web

La navigation est assurée par une structure commune après connexion. Une barre latérale permet d'accéder aux principales sections : tableau de bord, QCM, création d'un QCM, sessions et résultats, classes, professeurs et profil. Cette barre latérale n'est affichée qu'après connexion afin de bien séparer l'espace public de l'espace authentifié.

L'architecture prend également en compte les rôles utilisateurs. Lorsqu'un administrateur est connecté, l'application affiche les sections d'administration et lui donne accès à l'ensemble des données. Lorsqu'un professeur est connecté, l'interface affiche uniquement les éléments liés à ses affectations. Cette logique de filtrage est appliquée à la fois côté front-end pour adapter l'affichage, et côté backend pour sécuriser réellement l'accès aux données.

La création d'un QCM suit une architecture en plusieurs étapes. Le formulaire est découpé en trois parties : informations générales, questions et réponses, puis récapitulatif. Cette organisation permet de mieux contrôler les données saisies et d'éviter les erreurs avant la publication. Les informations du QCM sont conservées dans l'état de l'application pendant la création, puis envoyées au backend lors de la sauvegarde ou de la publication.

La gestion des données côté front-end repose sur des appels à l'API. Par exemple, lorsqu'un utilisateur ouvre la page des classes, l'application appelle une route backend pour récupérer les classes, les étudiants et les matières associées. Lorsqu'un administrateur ajoute un étudiant, le front-end envoie les informations saisies au backend, qui les enregistre dans PostgreSQL. Après la réponse de l'API, la page est mise à jour pour afficher les nouvelles données.

La figure suivante présente les principales communications entre le front-end et le backend :

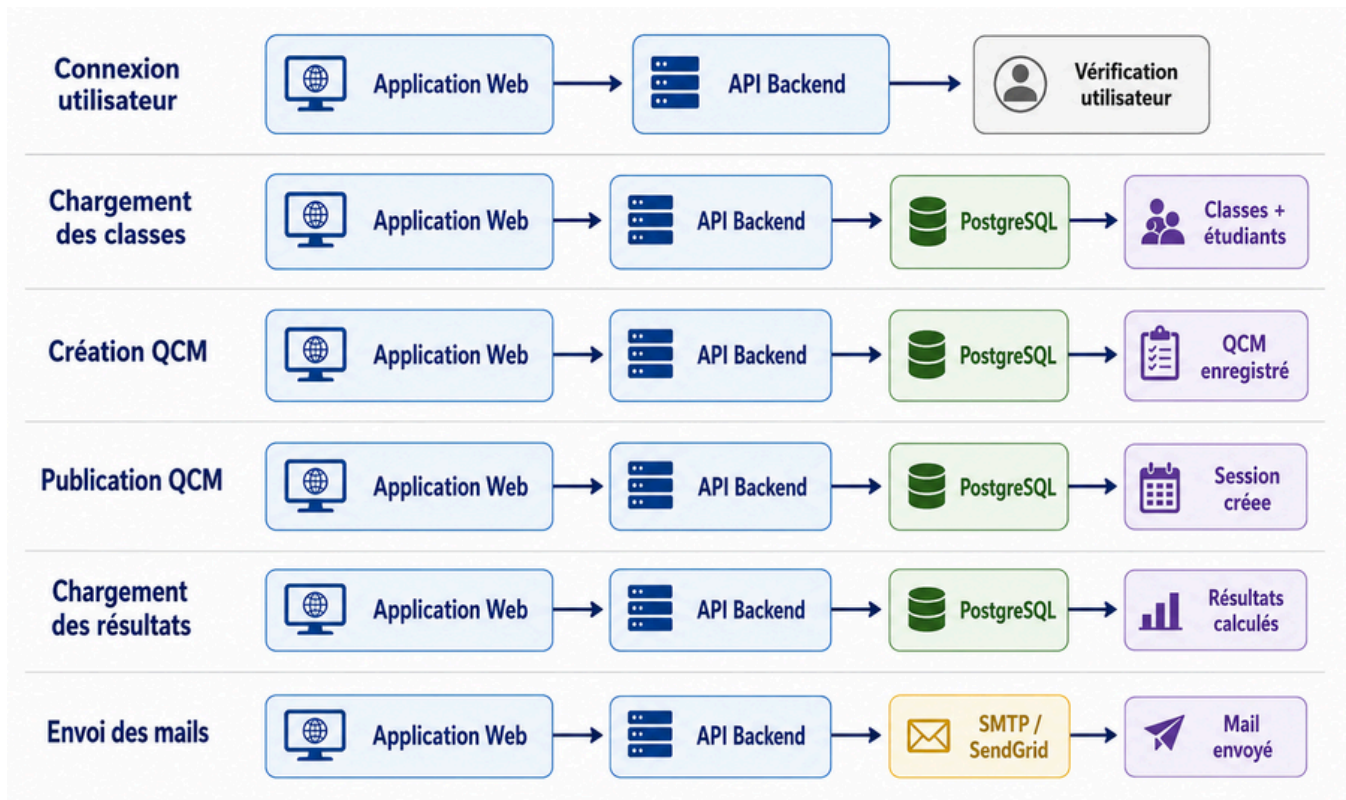


Figure 22 : Schéma des principales communications entre le front-end, le backend et les services associés

Cette architecture permet une bonne séparation des responsabilités. L'application web affiche les données, guide l'utilisateur et envoie les actions au backend. Le backend applique les règles métier, vérifie les droits, calcule les scores et interagit avec la base de données. Cette séparation rend la solution plus robuste, car une modification de la logique métier peut être réalisée côté backend sans réécrire toute l'interface.

En résumé, l'architecture de l'application web est pensée pour être claire, modulaire et évolutive. Elle organise les fonctionnalités en pages cohérentes, utilise une navigation adaptée aux rôles et communique avec le backend à travers des API. Cette structure permet à l'application web de jouer pleinement son rôle de plateforme de gestion pédagogique et administrative pour QCM Corrector.

6.3 Gestion de l'authentification et des rôles :

La gestion de l'authentification et des rôles est un élément essentiel de l'application web QCM Corrector. Elle permet de sécuriser l'accès à la plateforme et d'adapter les fonctionnalités selon le profil de l'utilisateur connecté. Dans le cadre du projet, deux profils principaux ont été définis: administrateur et professeur.

L'accès à l'application se fait à partir d'une page de connexion. Avant authentification, l'utilisateur ne voit pas l'espace de travail complet. La barre latérale, le tableau de bord, les sections QCM, sessions, classes et professeurs sont masqués. Cela permet de séparer clairement l'espace public de l'espace connecté. Une fois l'utilisateur authentifié, l'application affiche l'interface principale avec la navigation adaptée à son rôle.

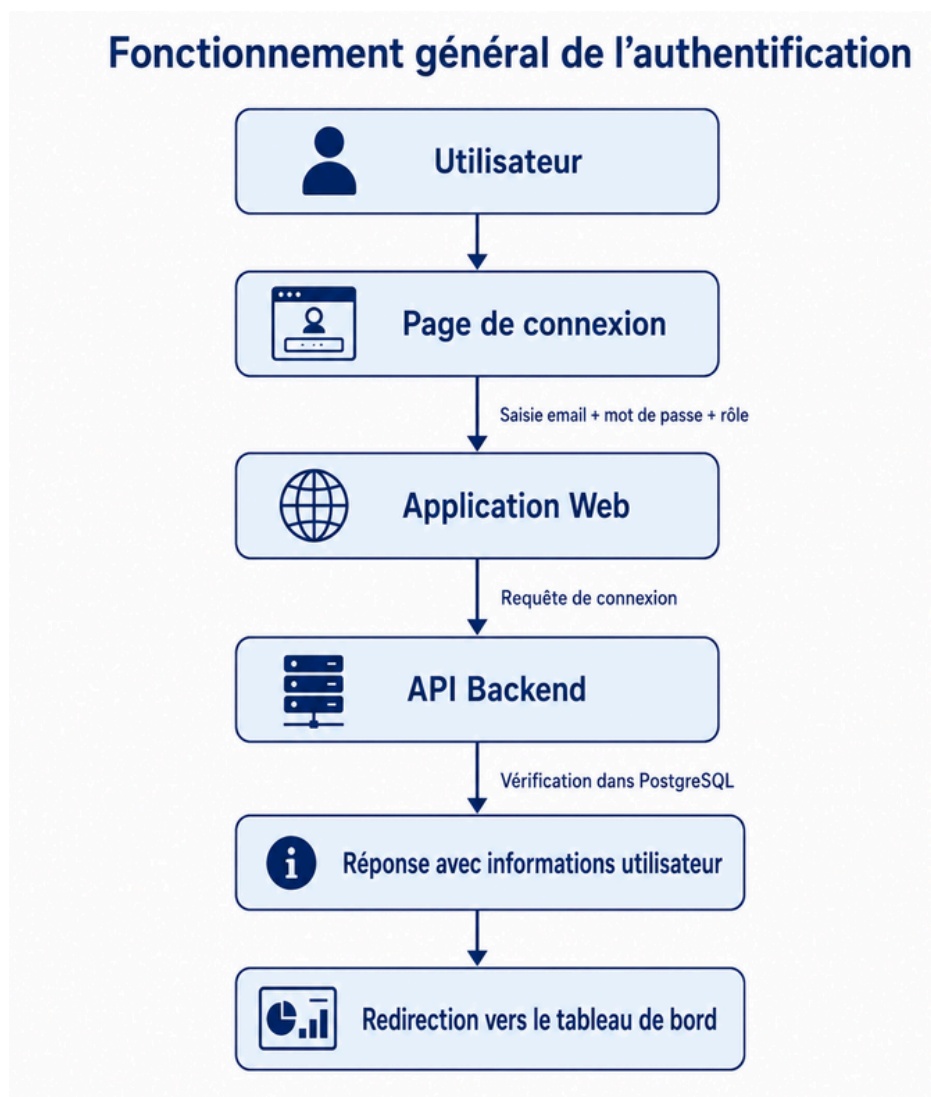


Figure 23 : Schéma du fonctionnement général de l'authentification

Lorsqu'un utilisateur tente de se connecter, l'application web envoie au backend les informations saisies : adresse email, mot de passe et rôle choisi. Le backend vérifie ensuite ces informations dans la base de données. Si les identifiants sont corrects et que le rôle correspond au profil sélectionné, l'utilisateur est authentifié et redirigé vers le tableau de bord.

Le rôle de l'utilisateur est important, car il détermine les données et les actions accessibles dans l'application. L'administrateur possède une vision globale de la plateforme, tandis que le professeur dispose d'un accès limité à son propre périmètre pédagogique.

L'administrateur peut accéder à l'ensemble des fonctionnalités de l'application. Il peut consulter tous les QCM, toutes les sessions, toutes les classes, tous les étudiants et tous les professeurs. Il peut aussi créer un QCM à la place d'un professeur, en sélectionnant le professeur responsable, la matière et la classe concernée. Il dispose également d'actions supplémentaires dans la section Classes & étudiants, comme la création d'une classe, l'import d'étudiants depuis un fichier CSV ou Excel, l'ajout d'un étudiant, la modification du nom d'une classe ou la suppression d'un étudiant.

Le professeur, quant à lui, accède uniquement aux données qui le concernent. Il peut consulter ses propres QCM, ses sessions de correction, ses classes et les résultats associés à ses matières. Par exemple, si un professeur est affecté à la matière Algorithmique pour la classe JM1, il ne doit pas consulter les QCM ou les résultats d'un autre professeur responsable d'une autre classe ou d'une autre matière. Cette restriction permet de respecter les responsabilités de chaque enseignant et de protéger les données pédagogiques.

La gestion des rôles intervient donc à deux niveaux.

Le premier niveau est l'interface web. Le front-end adapte l'affichage selon le rôle connecté. Certaines sections ou certains boutons ne sont visibles que pour l'administrateur. Par exemple, la section Profs est réservée à l'administrateur, de même que les boutons de création, modification, suppression et import dans la section Classes & étudiants. Un professeur peut visualiser les informations liées à ses classes, mais ne dispose pas des mêmes actions administratives.

Le second niveau est le backend. Cette vérification est indispensable, car masquer un bouton dans l'interface ne suffit pas à sécuriser une application. Le backend vérifie également le rôle de l'utilisateur avant d'exécuter certaines actions. Par exemple, les routes de gestion des professeurs, de création de classes ou d'import d'étudiants sont réservées à l'administrateur. Si un utilisateur professeur tente d'accéder à une action administrative, le backend refuse la demande.

Cette double vérification améliore la sécurité de l'application. Le front-end rend l'expérience utilisateur plus claire en affichant uniquement les actions utiles, tandis que le backend garantit que les règles d'accès sont réellement appliquées.

L'application conserve les informations de l'utilisateur connecté afin d'adapter la navigation et les appels API. Ces informations comprennent notamment l'identifiant de l'utilisateur, son adresse email, son prénom, son nom et son rôle. Elles sont utilisées pour afficher le nom du profil connecté dans l'en-tête, adapter la barre latérale et envoyer au backend les paramètres nécessaires au filtrage des données.

Le filtrage des données est particulièrement important pour les professeurs. Lorsqu'un professeur demande la liste des classes, des QCM ou des sessions, le backend ne renvoie que les éléments associés à ses matières. À l'inverse, lorsque l'administrateur effectue la même demande, le backend renvoie l'ensemble des données. Cette logique permet d'utiliser les mêmes pages dans l'application tout en adaptant leur contenu selon le rôle.

◆ Répartition des fonctionnalités selon le rôle utilisateur ◆

Dans notre application, les droits d'accès dépendent du rôle de l'utilisateur connecté. Deux profils principaux sont distingués : l'administrateur et le professeur.

Fonctionnalité	Administrateur	Professeur
Connexion à l'application	Oui	Oui
Accès au tableau de bord	Oui	Oui
Voir tous les QCM	Oui	Non
Voir ses propres QCM	Oui	Oui
Créer un QCM	Oui	Oui
Créer un QCM pour un professeur	Oui	Non
Voir toutes les sessions	Oui	Non
Voir ses sessions	Oui	Oui
Voir toutes les classes	Oui	Non
Voir ses classes	Oui	Oui
Gérer les classes	Oui	Non
Ajouter / importer des étudiants	Oui	Non
Supprimer / modifier un étudiant	Oui	Non
Gérer les professeurs	Oui	Non
Consulter les résultats	Oui	Oui, selon ses classes
Envoyer les résultats par mail	Oui	Oui, selon ses sessions

Figure 24 : Répartition des fonctionnalités entre l'administrateur et le professeur

Dans le fonctionnement actuel de l'application, cette gestion des rôles rend l'interface plus cohérente. Un professeur n'est pas surchargé par des options administratives inutiles, tandis que l'administrateur dispose d'un espace complet pour gérer la plateforme. Cette séparation permet aussi de mieux organiser les responsabilités dans un contexte réel d'établissement.

En résumé, l'authentification permet de sécuriser l'accès à l'application, tandis que la gestion des rôles permet d'adapter l'expérience utilisateur et de protéger les données. L'administrateur possède une vue globale et des droits de gestion étendus, alors que le professeur dispose d'un accès limité à ses propres classes, matières, QCM, sessions et résultats. Cette logique constitue une base importante pour assurer la fiabilité et la cohérence de l'application web.

6.4 Interfaces et parcours utilisateur :

L'application web QCM Corrector est organisée autour d'un parcours utilisateur simple et progressif. L'objectif principal est de permettre à l'utilisateur de passer facilement d'une étape à l'autre : connexion, accès au tableau de bord, gestion des QCM, publication, consultation des sessions, affichage des résultats et envoi des notes par mail.

Après authentification, l'utilisateur accède à un espace de travail composé d'une barre latérale, d'un en-tête et d'une zone principale de contenu. La barre latérale permet de naviguer entre les sections essentielles de l'application : tableau de bord, QCM, création d'un nouveau QCM, sessions et résultats, classes, statistiques et profil. Pour l'administrateur, une section supplémentaire permet d'accéder à la gestion des professeurs.

Le parcours dépend du rôle de l'utilisateur connecté. L'administrateur dispose d'un accès global aux données de la plateforme. Il peut consulter l'ensemble des QCM, des sessions, des classes, des étudiants et des professeurs. Il peut aussi créer un QCM pour un professeur donné et gérer les classes ainsi que les listes d'étudiants. Le professeur, quant à lui, accède uniquement aux éléments liés à ses propres matières et classes.

Le parcours principal lié aux QCM suit une logique pédagogique. L'utilisateur crée d'abord un QCM, renseigne les informations générales, ajoute les questions et les réponses, puis vérifie le récapitulatif avant publication. Une fois le QCM publié, il peut générer le sujet PDF et la grille des bonnes réponses. Les corrections reçues sont ensuite regroupées dans une session, où l'utilisateur peut consulter les résultats et déclencher l'envoi des notes par mail.

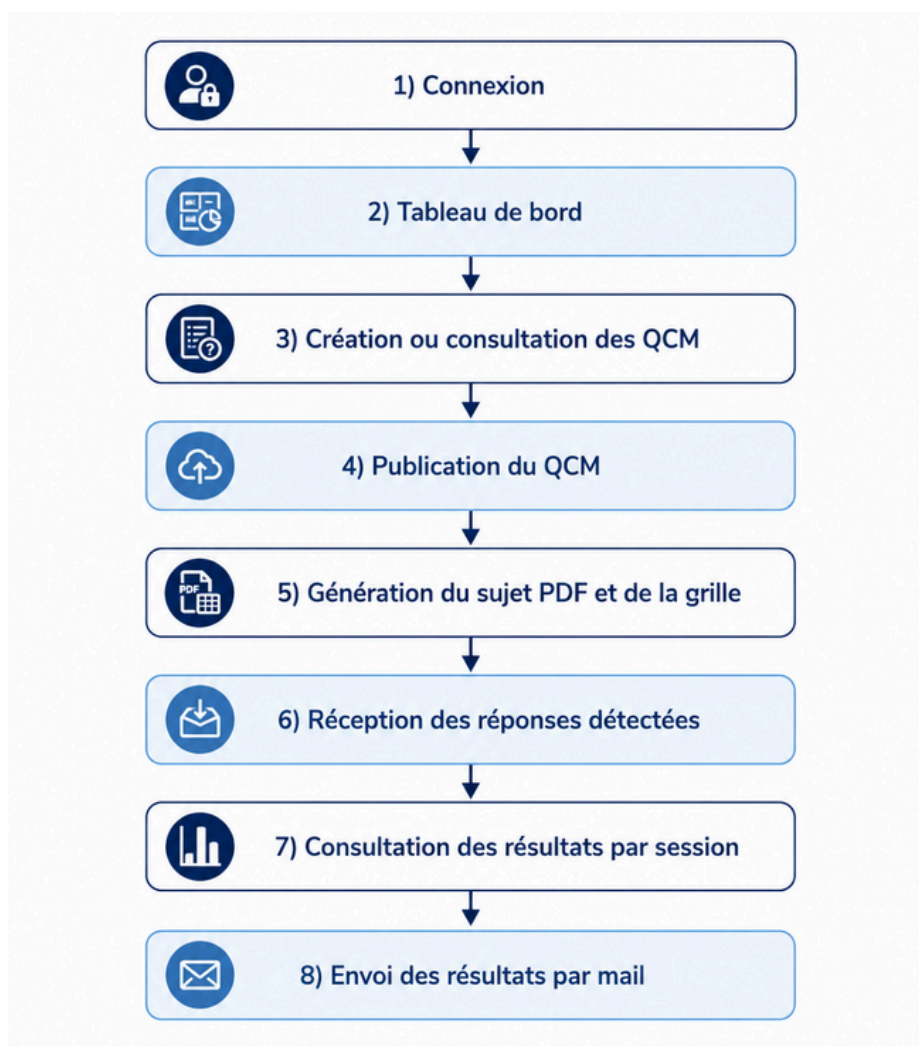


Figure 25 : Parcours général depuis la connexion jusqu'à l'envoi des résultats

Les détails de chaque page, les actions disponibles et les captures d'écran associées seront présentés dans la partie 6.6 consacrée aux cas d'utilisation détaillés.

6.5 Communication avec l'API backend :

L'application web QCM Corrector ne fonctionne pas de manière isolée. Elle communique en permanence avec l'API Backend afin de récupérer, enregistrer ou modifier les données de la plateforme. Cette communication est indispensable, car l'application web ne se connecte jamais directement à la base de données PostgreSQL. Toutes les opérations passent par le backend, qui joue le rôle d'intermédiaire entre l'interface utilisateur et la base de données.

Cette architecture permet de séparer les responsabilités. L'application web se concentre sur l'affichage, la navigation et l'interaction avec l'utilisateur, tandis que le backend se charge de la logique métier, de la vérification des droits, du calcul des scores, de la gestion des QCM, de l'enregistrement des résultats et de l'envoi des mails.

La communication entre le front-end et le backend repose sur des requêtes HTTP. Les données sont échangées principalement au format JSON. Lorsqu'un utilisateur effectue une action dans l'interface, le front-end envoie une requête à une route API. Le backend traite ensuite la demande, interagit avec PostgreSQL si nécessaire, puis renvoie une réponse à l'application web.

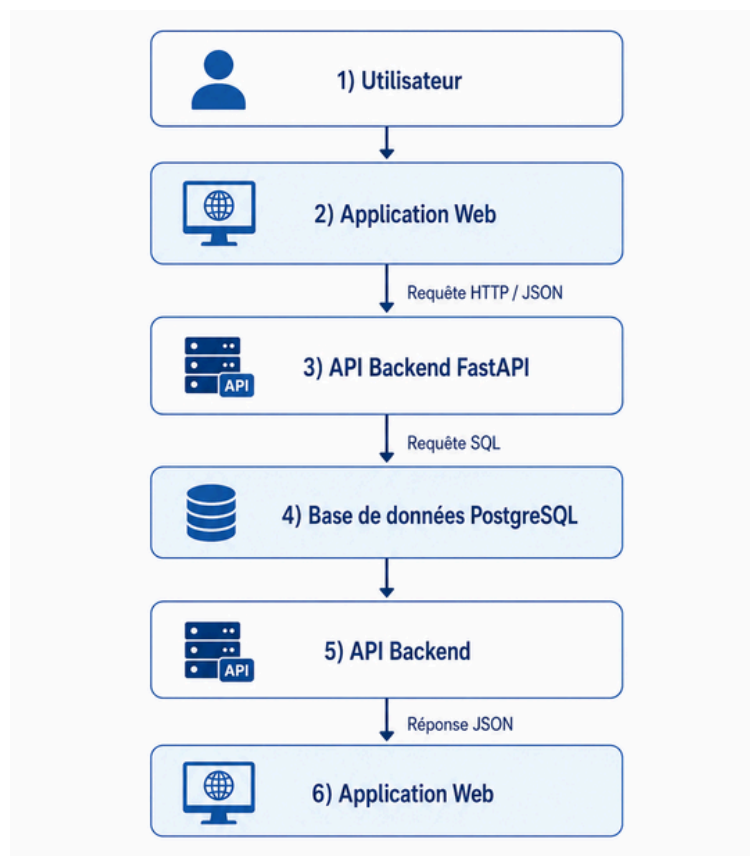


Figure 26 : Flux de communication entre l'utilisateur, l'application web , l'API Backend et la base de données

La première communication importante concerne l'authentification. Lorsqu'un utilisateur se connecte, l'application web envoie son adresse email, son mot de passe et son rôle au backend. Le backend vérifie ces informations dans PostgreSQL. Si elles sont valides, il renvoie les informations de l'utilisateur connecté : identifiant, nom, prénom, email et rôle. Ces données sont ensuite utilisées par l'application web pour adapter l'affichage et les droits d'accès.

L'application web communique aussi avec l'API pour récupérer les classes et les étudiants. Lorsqu'un administrateur ouvre la page Classes & étudiants, le front-end demande au backend la liste des classes, les étudiants associés, les matières liées et les professeurs responsables. Si l'utilisateur connecté est un professeur, le backend filtre les données afin de ne renvoyer que les classes liées à ses propres matières.

Pour la gestion des classes, l'administrateur peut effectuer plusieurs actions depuis l'interface web: créer une classe, modifier le nom d'une classe, supprimer une classe, ajouter un étudiant, modifier un étudiant, supprimer un étudiant ou importer une classe depuis un fichier CSV ou Excel. À chaque action, l'application web envoie une requête au backend. Le backend vérifie que l'utilisateur est bien administrateur, puis effectue l'opération dans PostgreSQL.

La création d'un QCM repose également sur une communication importante avec le backend. Pendant la création, l'utilisateur renseigne les informations générales, les questions, les propositions, les bonnes réponses et la pondération. Lorsqu'il sauvegarde un brouillon ou publie le QCM, l'application web envoie toutes ces données au backend. Le backend enregistre alors le QCM, les questions, les propositions et les réponses correctes dans la base de données.

Lorsqu'un QCM est publié, le backend crée aussi la session de correction associée. Cette session permet ensuite de regrouper les résultats des étudiants. Le front-end reçoit en retour les identifiants du QCM et de la session, ce qui permet d'accéder aux pages de génération PDF et de suivi des résultats.

La génération des documents PDF dépend également de l'API backend. L'application web affiche des boutons permettant de générer le sujet PDF et la grille PDF. Lorsqu'un utilisateur clique sur ces boutons, le front-end demande au backend de produire le document correspondant à partir des données du QCM. Le backend récupère les questions et les réponses dans PostgreSQL, génère le fichier PDF, puis le renvoie à l'application web.

La consultation des sessions et des résultats est une autre partie importante de la communication avec l'API. L'application web demande au backend la liste des sessions disponibles. Le backend renvoie les sessions selon le rôle connecté : toutes les sessions pour l'administrateur, et uniquement les sessions liées aux matières du professeur pour un enseignant. Pour chaque session, l'interface affiche le nombre de copies reçues, la moyenne, les corrections à vérifier et le statut d'envoi des mails.

Lorsqu'un utilisateur ouvre une session précise, l'application web récupère les résultats détaillés. Le backend renvoie les informations des étudiants, les réponses détectées, le score obtenu, le statut de correction et la date de réception. Ces données sont ensuite affichées dans un tableau afin que l'enseignant ou l'administrateur puisse analyser les résultats.

La réception des réponses détectées est principalement assurée par le backend, mais elle se reflète dans l'application web. Les réponses envoyées par l'application mobile ou le service IA sont reçues par l'API, associées à une session, comparées à la grille de correction, puis enregistrées dans PostgreSQL. L'application web récupère ensuite ces résultats via l'API afin de les afficher à l'utilisateur.

Enfin, l'envoi des mails est déclenché depuis l'application web mais exécuté par le backend. Lorsqu'un utilisateur clique sur le bouton d'envoi des mails, l'application web envoie une requête à l'API. Le backend récupère les résultats de la session, prépare un mail personnalisé pour chaque étudiant, utilise le service SMTP pour envoyer les messages, puis renvoie un statut d'envoi. L'application web affiche ensuite si les mails sont envoyés, partiellement envoyés ou encore en attente.

Les principales communications entre l'application web et le backend peuvent être résumées ainsi:

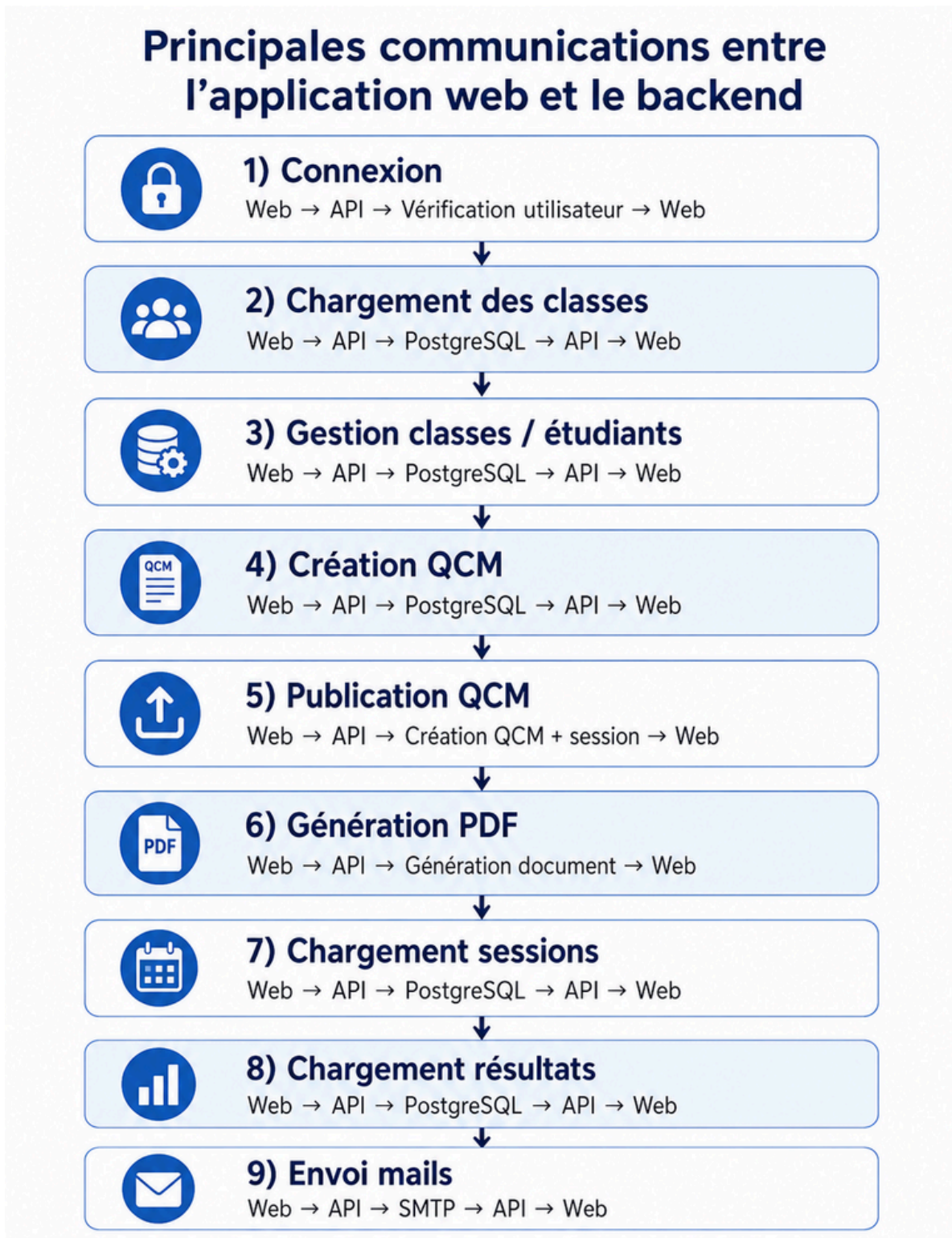


Figure 27 : Synthèse des principales communications entre l'application web et le backend

Cette communication permanente permet à l'application web d'être dynamique et connectée aux données réelles. Les modifications effectuées dans l'interface, comme l'ajout d'un étudiant ou la création d'un QCM, sont directement enregistrées dans PostgreSQL. De la même manière, les résultats reçus par l'API sont immédiatement disponibles dans l'application web.

En résumé, la communication avec l'API Backend constitue la base du fonctionnement de l'application web. Elle permet au front-end d'afficher les données, d'envoyer les actions de l'utilisateur, de récupérer les résultats, de générer les documents PDF et de déclencher l'envoi des mails. Grâce à cette séparation entre interface web et logique backend, la solution reste plus structurée, plus sécurisée et plus facile à maintenir.

6.6 Cas d'utilisation détaillés :

6.6.1 Connexion :

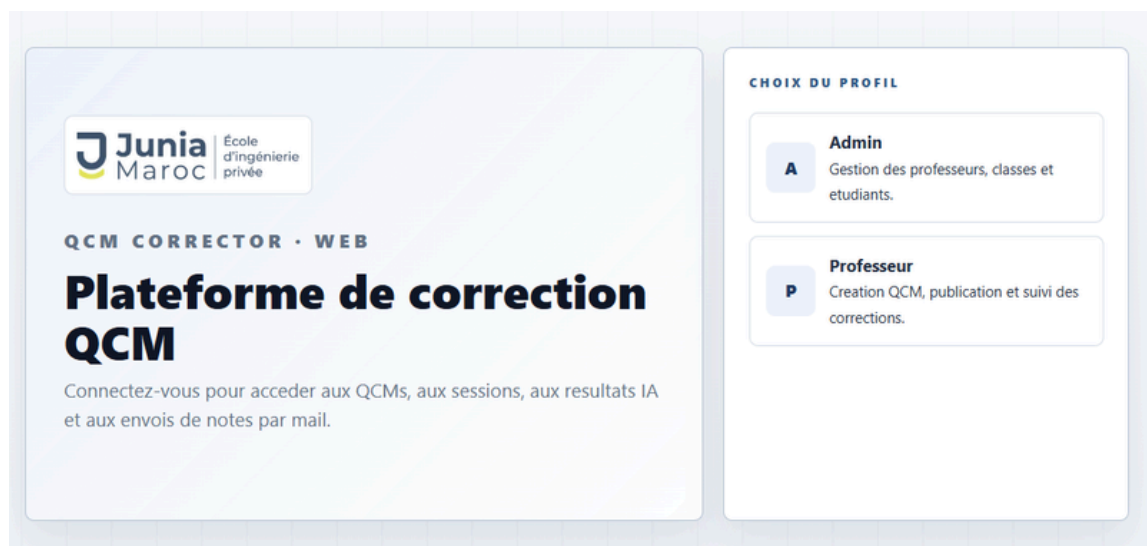


Figure 28 : Page de choix du profil utilisateur

Cette capture présente la première page affichée lors de l'accès à l'application web QCM Corrector. Elle sert de point d'entrée vers la plateforme et permet à l'utilisateur de choisir son profil avant de se connecter.

L'interface est divisée en deux parties. À gauche, un bloc de présentation affiche le logo de JUNIA Maroc, le nom de la plateforme et une courte description de son objectif. Ce texte indique que l'application permet d'accéder aux QCM, aux sessions, aux résultats issus de l'IA et à l'envoi des notes par mail.

À droite, la section Choix du profil propose deux accès distincts: Admin et Professeur. Le profil administrateur est destiné à la gestion globale de la plateforme, notamment les professeurs, les classes et les étudiants. Le profil professeur est destiné à la création des QCM, à leur publication et au suivi des corrections.

Ces deux interfaces permettent l'authentification selon le rôle de l'utilisateur. Le formulaire administrateur donne accès à la gestion globale de la plateforme: professeurs, classes, étudiants, QCM et sessions. Le formulaire professeur donne accès à l'espace enseignant, limité aux QCM, classes et résultats liés à ses propres affectations.

Chaque utilisateur saisit son email et son mot de passe, puis clique sur Entrer. Les identifiants sont envoyés au backend, qui vérifie le compte, le mot de passe et le rôle sélectionné avant d'autoriser l'accès au tableau de bord.

CONNEXION PROFESSEUR

Espace enseignant

Email

Mot de passe

Entrer

CONNEXION ADMIN

Administration

Email

Mot de passe

Entrer

Figure 29 : Formulaire de connexion administrateur et professeur

6.6.2 Tableau de bord :

Junia Maroc
École d'ingénierie privée

QCM Corrector
JUNIA MAROC

PRINCIPAL

- Tableau de bord
- Tous les QCMs
- Sessions & resultats
- Classes
- MR Profs

STATISTIQUES

- Tableau de bord stats

COMPTE

- AP Profil & mot de passe

Admin Plateforme
Administrateur

QCM CORRECTOR - WEB

Administrateur - Admin Plateforme

Accueil > Tableau de bord

ADMINISTRATION - TABLEAU DE BORD

Admin Plateforme

Sessions, classes et resultats charges selon le profil connecte.

SESSIONS
8
dans le perimetre

COPIES
10
resultats recus

A VERIFIER
0
contrôle manuel

MOYENNE
4
score moyen

PRINCIPAL

- Mes QCMs
Filtres par statut, brouillons, liens PDF et sessions.
- Sessions & resultats
Scores par etudiant apres reception des JSON IA.
- Classes
Etudiants, identifiants a 6 chiffres et groupes.

DERNIERES SESSIONS

SESSION	CLASSE	COPIES	ACTION
Test mail Khalil - Algorithmique JM1 session-9	JM1	2	Voir
CC1 algo session-8	JM1	0	Voir
Securite informatique - Controle S3 session-7	JM3	0	Voir
Programmation Python - Controle S2	JM3	1	Voir

Figure 30 : Tableau de bord

Cette image présente le tableau de bord affiché après la connexion de l'administrateur. Il donne une vue générale sur l'état de la plateforme: nombre de sessions, copies reçues, corrections à vérifier et moyenne globale.

La barre latérale permet d'accéder rapidement aux principales sections: QCM, sessions et résultats, classes, professeurs, statistiques et profil. Au centre, des raccourcis facilitent l'accès aux fonctions principales, comme la gestion des QCM, des sessions et des classes. La partie droite affiche les dernières sessions de correction avec leur classe, le nombre de copies reçues et un bouton permettant d'ouvrir les résultats.

Ce tableau de bord sert donc de point d'entrée principal après authentification. Il permet à l'administrateur de suivre rapidement l'activité de la plateforme et d'accéder aux différentes fonctionnalités de gestion.

6.6.3 Gestion des QCM :

QCM CORRECTOR - WEB
Enseignant - Youssef El Amrani

ENSEIGNANT - MES QCMs
Mes QCMs
Youssef El Amrani - creation, brouillons, publication et liens PDF.

Sessions & résultats **Nouveau QCM**

TOTAL
4
QCMs visibles

PUBLIES
3
disponibles correction

BROUILLONS
1
à reprendre

ARCHIVES
0
historique

FILTRE
Liste des QCMs

Statut à afficher
Tous les statuts

QCM	MATIERE	CLASSE	QUESTIONS	TOTAL	STATUT	DERNIERE MODIFICATION	ACTIONS
CC1 algo QCM-2026-1497	Reseaux	Ingenierie 3A - Reseaux	3	5 pts	Brouillon	02 juin 2026	Continuer
Test mail Khalil - Algorithmique JM1 QCM-TEST-KHALIL-JM1	Algorithmique	JM1	4	5 pts	Publie	01 juin 2026	Resultats Sujet PDF Grille PDF
CC1 algo QCM-2026-1497	Algorithmique	JM1	3	5 pts	Publie	01 juin 2026	Resultats Sujet PDF

Figure 31 : Interface de gestion des QCM

Cette interface permet de consulter et gérer les QCM enregistrés dans la plateforme. L'administrateur peut visualiser l'ensemble des QCM, tandis qu'un professeur ne voit que les QCM liés à ses propres matières et classes.

Chaque QCM est affiché avec ses informations principales: titre, matière, classe, nombre de questions, total des points, statut et dernière modification. Les statuts permettent de distinguer les QCM en brouillon, publiés ou archivés.

Depuis cette page, l'utilisateur peut accéder aux actions disponibles selon le statut du QCM. Un QCM en brouillon peut être repris ou modifié, tandis qu'un QCM publié peut être consulté et utilisé pour les sessions de correction. Cette page sert donc de point central pour retrouver les évaluations créées et suivre leur état dans le système.

6.6.4 Création d'un QCM :

The screenshot displays the 'QCM CORRECTOR - WEB' interface for a user named 'Youssef El Amrani'. The interface is divided into a left sidebar and a main content area. The sidebar contains navigation links under 'PRINCIPAL' (Tableau de bord, Mes QCMs, Nouveau QCM, Sessions & résultats, Classes), 'STATISTIQUES' (Tableau de bord stats), and 'COMPTE' (Profil & mot de passe). The main content area shows a progress bar with three steps: 'Etape 1/3 Informations générales' (active), 'Etape 2/3 Questions & réponses', and 'Etape 3/3 Recapitulatif'. Below the progress bar, the 'CREATION QCM' section shows the title 'CC1 algo' and the user 'Autosave actif: Youssef El Amrani'. The 'INFORMATIONS GENERALES DU QCM' section contains several input fields: 'Affectation' (Youssef El Amrani), 'Titre du QCM' (CC1 algo), 'Date d'examen' (07/07/2026), 'Classe / groupe' (JM1), 'Professeur responsable' (youssef.elamrani@ecole.ma), 'Matiere' (Algorithmique), 'Niveau / promotion' (JM1), 'Code QCM' (QCM-2026-1497), and 'NOMBRE DE QUESTIONS' (3). There are also buttons for 'Enregistrer brouillon', 'Publier', 'Annuler', and 'Suivant: Questions'.

Figure 32 : Création d'un QCM - Informations générales

Cette interface correspond à la première étape de création d'un QCM. Le professeur renseigne les informations principales de l'évaluation: titre du QCM, date d'examen, classe, matière, niveau, code QCM et nombre de questions.

L'affectation du professeur est automatiquement reprise depuis son compte connecté. Dans cet exemple, le QCM est associé au professeur Youssef El Amrani, à la matière Algorithmique et à la classe JM1.

Cette étape permet de définir le cadre général de l'évaluation avant de passer à la saisie des questions. L'utilisateur peut enregistrer le QCM comme brouillon, annuler la création ou continuer vers l'étape suivante. Le nombre de questions peut aussi être ajusté avant la saisie des réponses.

QUESTIONS

1 completes / 3

Vert: complet. Orange: champ manquant.

1

2

3

Cliquez sur une case orange pour compléter la question concernée.

QUESTION 1

Complete

Question complete

Enonce

Quelle structure implemente le principe LIFO ?

Reponse unique

Reponses multiples

Ponderation

1

A

Pile (Stack)

☒ Correcte

B

File (Queue)

☐ Correcte

C

Liste chaine

☐ Correcte

D

Arbre binaire

☐ Correcte

Annuler

Retour

Enregistrer brouillon

Ajouter question

Supprimer

Prochaine question

Continuer: Recapitulatif

Figure 33 : Création d'un QCM - Saisie des questions et des réponses

Cette interface correspond à la deuxième étape de création d'un QCM. Elle permet au professeur de saisir l'énoncé de chaque question, les quatre propositions de réponses A, B, C et D, ainsi que la bonne réponse attendue.

À gauche, un panneau indique l'état d'avancement des questions. Les questions complètes sont affichées en vert, tandis que les questions incomplètes restent signalées en orange. Cela permet à l'utilisateur de repérer rapidement les questions qui doivent encore être corrigées avant la publication.

Au centre, le professeur peut choisir le type de question: réponse unique ou réponses multiples. Il peut également définir la pondération de la question. Les boutons situés en bas permettent de passer à la question suivante, d'ajouter une question, de supprimer une question, d'enregistrer le brouillon ou de continuer vers le récapitulatif.

1 Etape 1/3
Informations generales2 Etape 2/3
Questions & reponses3 Etape 3/3
Recapitulatif

CREATION QCM

CC1 algo

Autosave actif: Youssef El Amrani.

Enregistrer brouillon

Publier

Annuler

RECAPITULATIF

QCM complet - pret a publier

Verifiez le contenu ci-dessous avant de publier.

Modifier

TITRE

CC1 algo

MATIERE

Algorithmique

NIVEAU

JM1

PROFESSEUR

youssef.elamrani@ecole.ma

DATE D'EXAMEN

2026-07-07

NOMBRE DE QUESTIONS

3

TOTAL DES POINTS

5 pts

SYNTHESE

QUESTIONS

3

REPONSE UNIQUE

3

REPONSES MULTIPLES

0

TOTAL DES POINTS

5 pts

STATUT

Brouillon

Une fois publie, ce QCM sera verrouille en lecture seule pour les corrections.

GRILLE DE CORRECTION

Reponses correctes

#	A	B	C	D	PTS
1	X				1
2		X			1
3			X		3

QUESTIONS & PROPOSITIONS

Contenu saisi

Modifier

1. Quelle structure implemente le principe LIFO ?

1 pt(s)

A Pile (Stack)

Correcte

B File (Queue)

C Liste chaine

D Arbre binaire

2. Quels tris ont une complexite moyenne en $O(n \log n)$?

1 pt(s)

A Tri a bulles

B Tri fusion Correcte

C Tri rapide

D Tri par insertion

3. Complexite en espace d'un tri fusion ?

3 pt(s)

A $O(1)$ B $O(\log n)$ C $O(n)$

Correcte

D $O(n \log n)$

Annuler

Retour

Enregistrer brouillon

Publier

Figure 34 : Récapitulatif du QCM avant publication

Cette interface correspond à la dernière étape de création du QCM. Elle permet au professeur de vérifier toutes les informations saisies avant la publication: titre, matière, professeur responsable, date d'examen, nombre de questions et total des points.

La page affiche également une grille de correction générée automatiquement à partir des réponses cochées. Les bonnes réponses sont marquées par une croix, ce qui permet de vérifier rapidement la cohérence du QCM avant son enregistrement définitif.

La partie inférieure présente le contenu détaillé des questions et propositions. Le professeur peut revenir en arrière pour modifier le QCM, enregistrer le brouillon ou cliquer sur Publier pour rendre l'évaluation disponible pour la correction. Cette étape sert donc de contrôle final avant la création de la session associée au QCM.

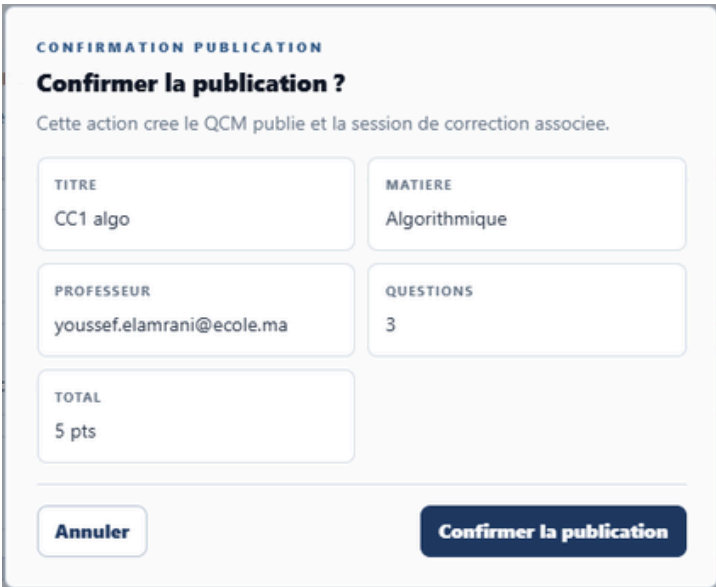


Figure 35 : Page de confirmation

Cette capture montre l'étape finale de publication d'un QCM. Avant de publier, une fenêtre de confirmation résume les informations principales : titre, matière, professeur responsable, nombre de questions et total des points. Cela permet d'éviter une publication accidentelle.

6.6.5 Publication et génération des PDF :



Figure 36 : Publication du QCM

Après confirmation, le QCM passe du statut brouillon au statut publié. L'application affiche alors un message de succès et génère la session de correction associée. Des boutons permettent ensuite de produire directement le sujet PDF, la grille des bonnes réponses ou de revenir à la liste des QCM.

SUJET DU QCM

Code: QCM-2026-1497
JUNIA MAROC

CC1 algo

MATIERE
Algorithmique

PROFESSEUR
Youssef El Amrani

DATE D'EXAMEN
07 juillet 2026

CLASSE
JM1

QUESTIONS
3

TOTAL POINTS
5 points

1. Quelle structure implemente le principe LIFO ?
1 pt(s)

A. Pile (Stack)

B. File (Queue)

C. Liste chainee

D. Arbre binaire

2. Quels tris ont une complexite moyenne en $O(n \log n)$?
1 pt(s)

A. Tri a bulles

B. Tri fusion

C. Tri rapide

D. Tri par insertion

3. Complexite en espace d'un tri fusion ?
3 pt(s)

A. $O(1)$

B. $O(\log n)$

C. $O(n)$

D. $O(n \log n)$

Figure 37 : Génération du sujet PDF

61

Cette capture montre le sujet PDF généré automatiquement après la publication du QCM. Le document reprend les informations principales de l'évaluation : titre, code QCM, matière, professeur, date d'examen, classe, nombre de questions et total des points.

Les questions sont ensuite affichées avec leurs propositions A, B, C et D, sans indiquer les bonnes réponses. Ce fichier est destiné aux étudiants ou à l'impression du sujet, contrairement à la grille de correction qui est réservée à l'enseignant.

6/2/26, 5:20 AM

QCM Corrector

GRILLE DE CORRECTION

Code: QCM-2026-1497

CC1 algo

Youssef El Amrani

Algorithmique - JM1

07 juillet 2026

QUESTION	A	B	C	D	TYPE	POINTS
1	X				Unique	1
2		X			Unique	1
3			X		Unique	3

Figure 38 : Génération de la grille de correction PDF

Cette image montre la grille de correction générée automatiquement après la publication du QCM. Elle reprend le code du QCM, le professeur, la date, la matière et la classe concernée.

Le tableau affiche chaque question avec ses choix possibles A, B, C et D. Les bonnes réponses sont indiquées par une croix, ce qui permet au professeur de vérifier rapidement la correction attendue. Cette grille est utilisée comme référence par le backend pour comparer les réponses détectées et calculer les scores des étudiants.

6.6.6 Sessions et résultats :

PRINCIPAL

Sessions & resultats

Youssef El Amrani - corrections recues depuis l'app mobile ou le moteur IA.

Etat

Tous les etats

Email

Tous les emails

SESSION	PROFESSEUR	CLASSE	COPIE(S)	MOYENNE	CONTROLE MANUEL	EMAIL	ACTIONS
Test mail Khalil - Algorithmique JM1 01 juin 2026 session-9	Youssef El Amrani	JM1	2	2.5/20	0	Envoye	Voir resultats Envoyer mails
CC1 algo 07 juillet 2026 session-8	Youssef El Amrani	JM1	0	-	0	En attente	Voir resultats Envoyer mails
Algorithmique - Controle S3 15 avril 2026 session-1	Youssef El Amrani	JM1	6	6.67/20	0	Envoye	Voir resultats Envoyer mails

Figure 39 : Sessions et résultats

Cette interface regroupe les sessions de correction liées au profil connecté. Dans cet exemple, le professeur voit uniquement les sessions associées à sa matière et à sa classe.

Chaque ligne présente les informations essentielles d'une session : titre du QCM, professeur, classe, nombre de copies reçues, moyenne, corrections à vérifier et statut d'envoi des mails. Les filtres en haut permettent de trier les sessions selon leur état ou le statut email.

Depuis cette page, l'utilisateur peut ouvrir le détail des résultats avec le bouton Voir résultats. Lorsque des copies sont déjà corrigées, il peut aussi déclencher l'envoi des notes aux étudiants avec Envoyer mails.

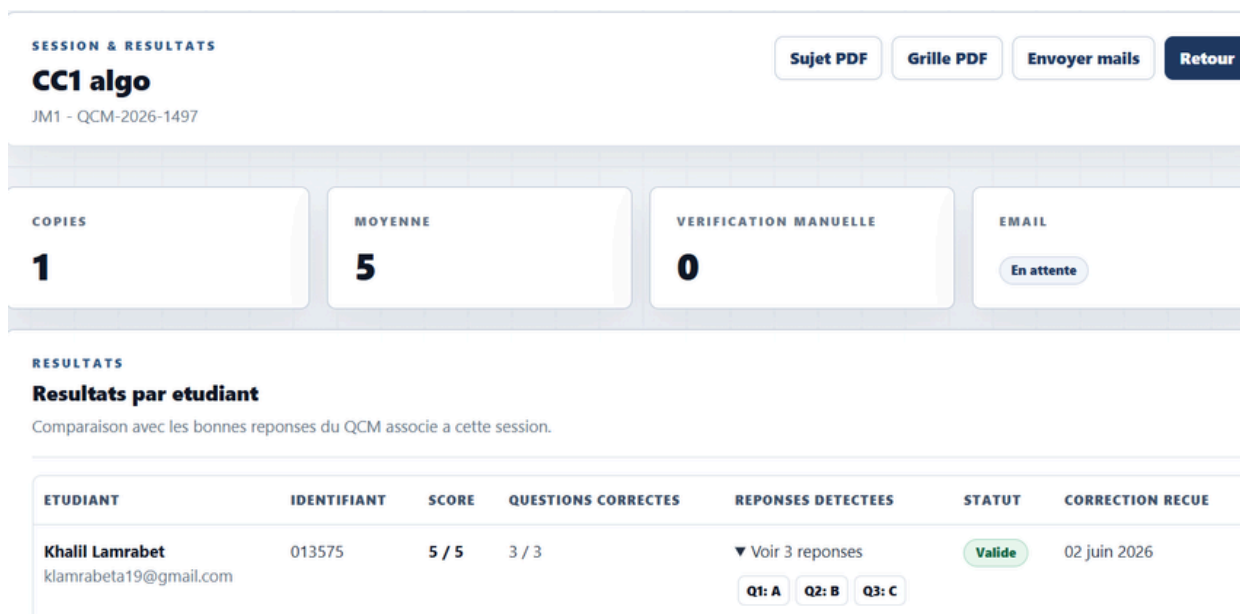


Figure 40 : Détail d'une session et affichage du score

Cette capture présente le détail de la session CC1 algo après réception des réponses d'un étudiant. La page affiche les indicateurs principaux de la session : nombre de copies reçues, moyenne, nombre de corrections à vérifier et statut d'envoi des mails.

Dans le tableau des résultats, l'application affiche l'étudiant concerné, son identifiant, son score, le nombre de questions correctes, les réponses détectées et le statut de correction. Ici, les réponses reçues pour Khalil Lamrabet sont comparées automatiquement avec la grille de correction du QCM.

Le score obtenu est 5 / 5, avec 3 questions correctes sur 3. Le statut Validé indique que l'identifiant étudiant a bien été reconnu et que le résultat a été correctement associé à la session.

6.6.7 Envoi des résultats par mail :

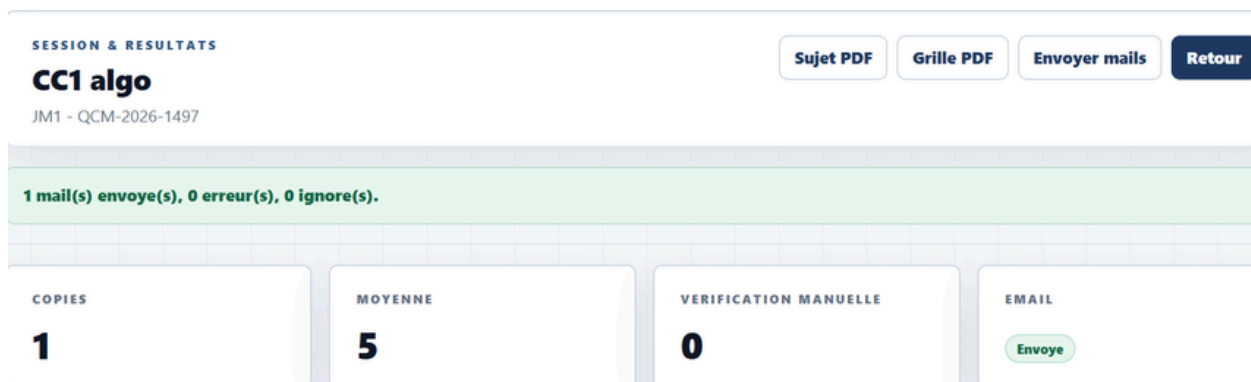


Figure 41 : Envoi des résultats par mail

Cette capture montre l'étape d'envoi des résultats aux étudiants depuis la page d'une session. Après le calcul du score, l'utilisateur peut cliquer sur Envoyer mails afin de transmettre automatiquement la note à l'étudiant concerné.

Le message de confirmation indique que 1 mail a été envoyé, sans erreur ni résultat ignoré. Le statut email de la session passe alors à Envoyé, ce qui permet au professeur ou à l'administrateur de vérifier rapidement que la communication des résultats a bien été effectuée.

Cette fonctionnalité évite l'envoi manuel des notes et complète le processus de correction automatique : réception des réponses, calcul du score, affichage du résultat, puis notification par mail.



Figure 42 : Mail reçu par l'étudiant

Cette capture présente le mail reçu par l'étudiant après l'envoi des résultats depuis l'application web. Le message est généré automatiquement par la plateforme QCM Corrector et reprend l'identité visuelle de l'école avec le logo JUNIA Maroc.

Le contenu du mail indique le nom de l'étudiant, le QCM concerné, la matière, la classe et la note obtenue. Dans cet exemple, Khalil Lamrabet reçoit son résultat pour le contrôle CC1 algo en Algorithmique, avec une note de 20/20, correspondant à 5/5 points.

Cette étape confirme que le processus complet fonctionne correctement : les réponses sont reçues, le score est calculé par le backend, le résultat est affiché dans l'application web, puis envoyé automatiquement à l'étudiant par mail.

6.7 Conception UML :

6.7.1 Diagramme de cas d'utilisation de l'application web :

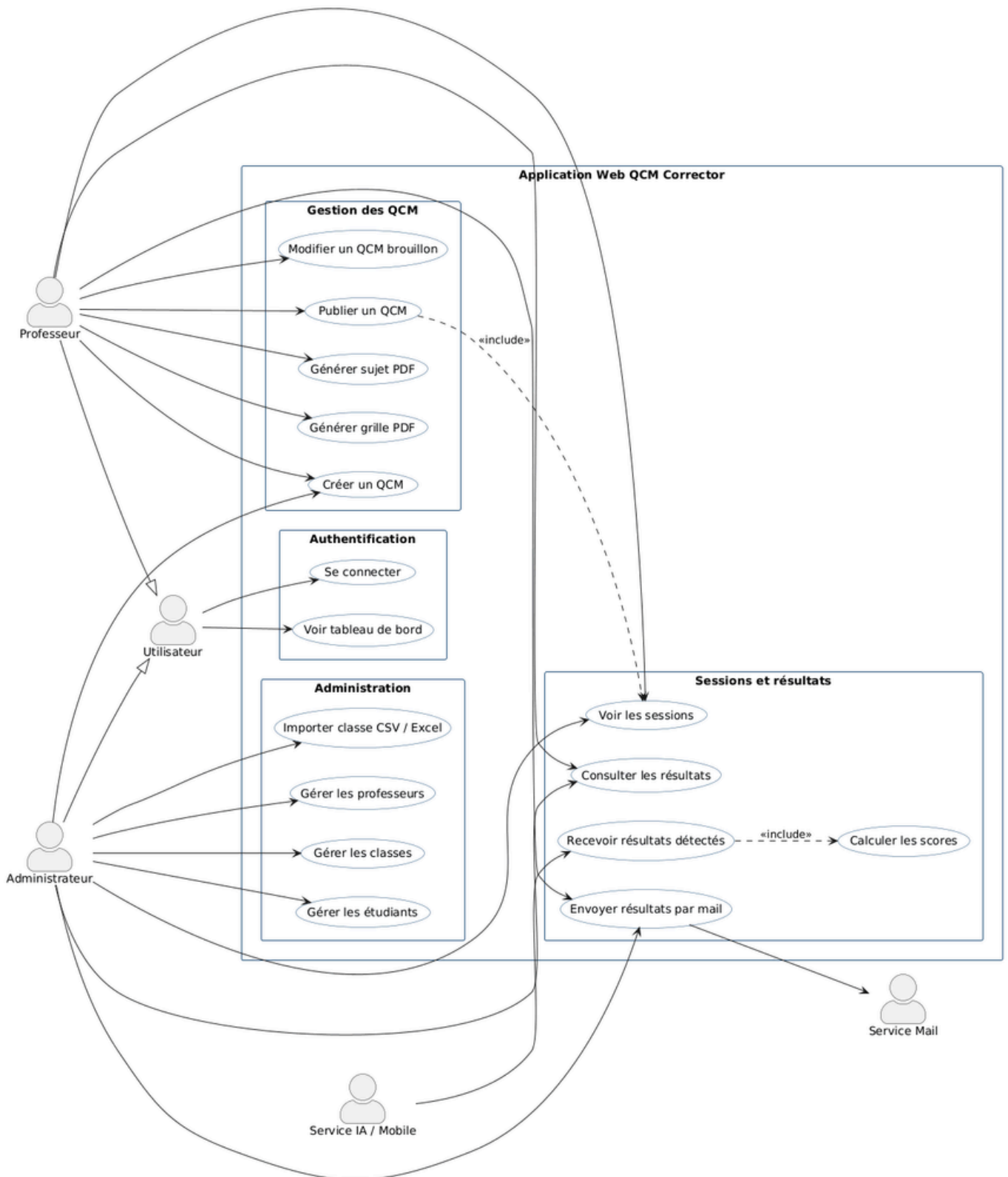


Figure 43 : Diagramme de cas d'utilisation de l'application web

Ce diagramme présente les principales fonctionnalités de l'application web QCM Corrector et les acteurs qui interagissent avec elle. Les deux acteurs principaux sont l'administrateur et le professeur. Tous les deux héritent d'un rôle utilisateur, ce qui signifie qu'ils doivent d'abord s'authentifier avant d'accéder aux fonctionnalités de la plateforme.

Le professeur peut créer un QCM, modifier un brouillon, publier un QCM et générer les documents PDF nécessaires, notamment le sujet et la grille de correction. Il peut aussi consulter ses sessions, visualiser les résultats des étudiants et envoyer les notes par mail.

L'administrateur dispose de droits plus étendus. En plus de l'accès aux QCM, sessions et résultats, il peut gérer les professeurs, les classes, les étudiants et importer des classes depuis des fichiers CSV ou Excel. Le diagramme montre aussi l'intervention du service IA/mobile, qui transmet les réponses détectées, et du service mail, utilisé pour l'envoi automatique des résultats.

6.7.2 Diagramme de séquence - Création et publication d'un QCM :

Ce diagramme décrit le déroulement complet de la création d'un QCM depuis l'interface web. L'utilisateur, qui peut être un professeur ou un administrateur, ouvre le formulaire de création. L'application web demande alors au backend les données nécessaires, comme les classes, les matières et les professeurs disponibles.

Après la saisie des informations générales, des questions et des réponses, le QCM est enregistré dans PostgreSQL avec ses questions et ses propositions. Il peut d'abord rester en brouillon afin d'être modifié plus tard.

Lorsque l'utilisateur publie le QCM, le backend change son statut en publié et crée automatiquement une session de correction associée. Une fois la publication confirmée, l'utilisateur peut générer le sujet PDF et la grille de correction PDF. Ces documents sont produits à partir des données enregistrées du QCM.

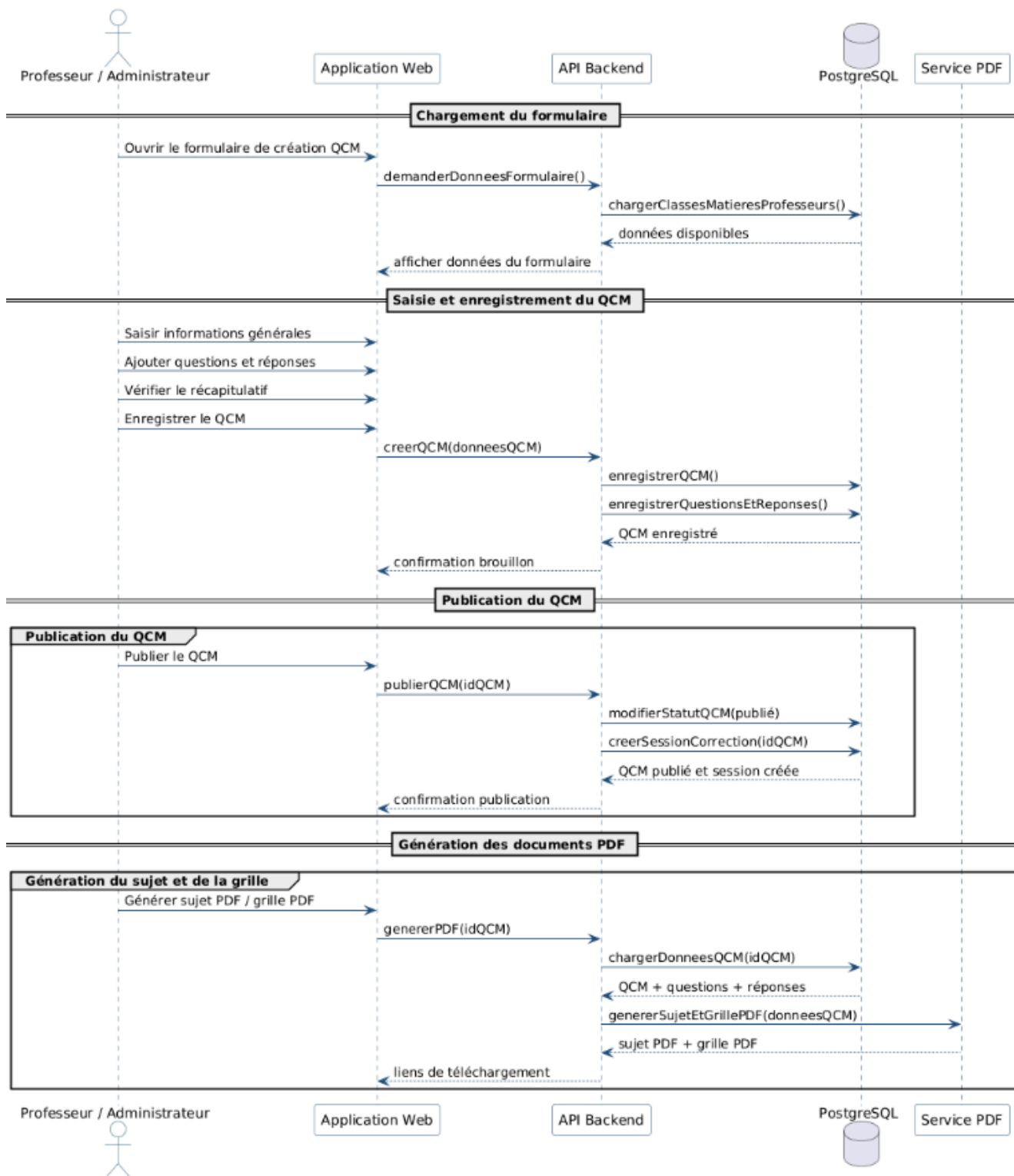


Figure 44 : Diagramme de séquence - Création et publication d'un QCM

6.7.3 Diagramme de cycle de vie d'un QCM :

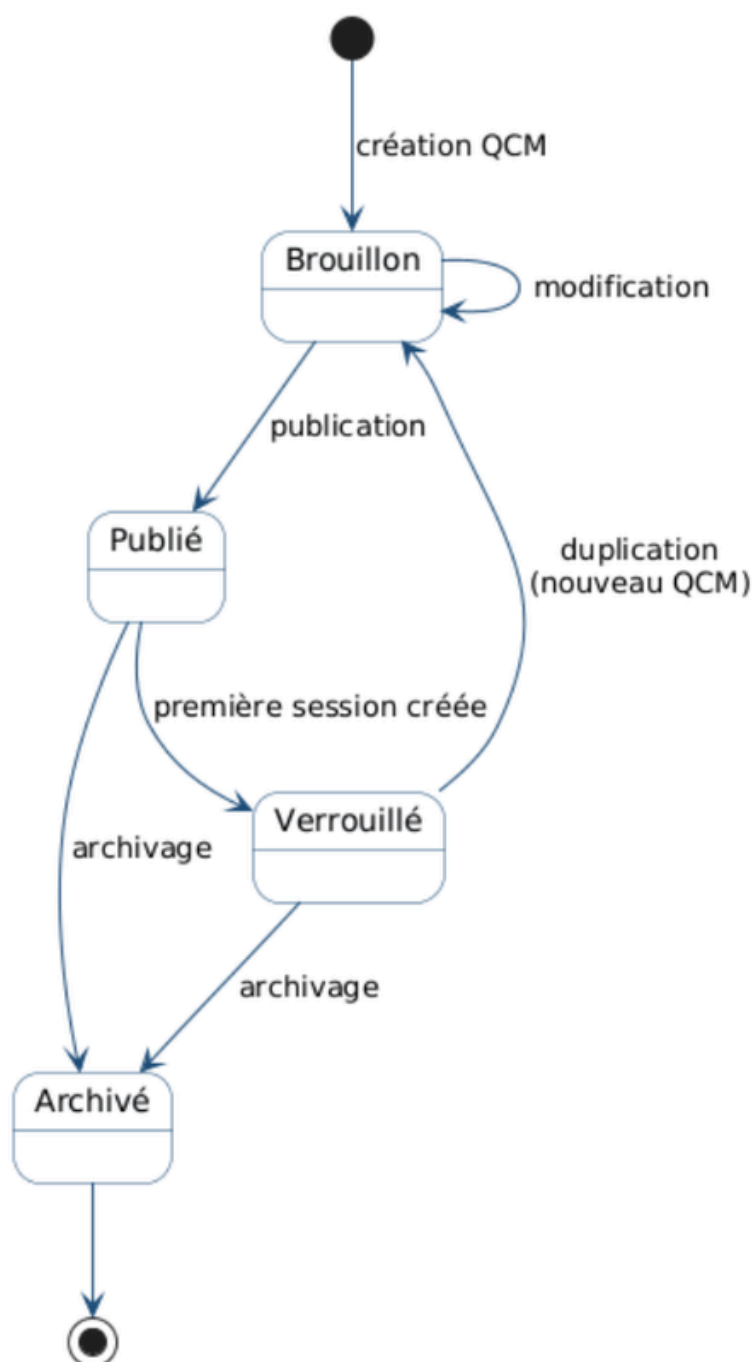


Figure 45 : Diagramme de cycle de vie d'un QCM

7- Réalisation de l'application mobile iOS :

7.1 Présentation :

Cette partie présente l'application mobile iOS développée dans le cadre du projet QCM Corrector. Elle est destinée aux enseignants et représente l'outil principal utilisé pour scanner les feuilles de réponses QCM.

L'application mobile permet à l'enseignant de se connecter, d'accéder à ses sessions de correction, de capturer une feuille de réponses à l'aide de la caméra de l'iPhone, puis d'envoyer l'image pour correction automatique. Après le traitement, l'enseignant peut consulter les résultats détectés et suivre l'historique des corrections effectuées. Contrairement à l'application web, qui est utilisée pour la gestion des QCM, des classes, des étudiants et des professeurs, l'application mobile se concentre principalement sur le processus de scan et de consultation rapide des résultats.

7.2 Rôle de l'application mobile dans le projet :

L'application mobile joue un rôle complémentaire à l'application web. Elle intervient après la création et la publication d'un QCM. Une fois la session de correction disponible, l'enseignant peut utiliser son iPhone pour scanner les feuilles remplies par les étudiants.

Son rôle principal est donc de faciliter la phase de correction en rendant le processus plus rapide et plus pratique. L'enseignant n'a pas besoin d'utiliser un scanner classique ni de transférer manuellement les images vers un ordinateur. La feuille est scannée directement avec l'application mobile, puis envoyée automatiquement au système de correction.

L'application mobile sert également d'interface de consultation. Elle permet à l'enseignant de visualiser les résultats obtenus après correction, de vérifier les informations détectées et de consulter l'historique des sessions déjà traitées.

7.3 Architecture interne de l'application mobile :

L'application mobile est organisée autour de plusieurs blocs fonctionnels. Cette organisation permet de séparer les interfaces graphiques, la navigation, la gestion des données et la communication avec le serveur.

La première partie concerne les interfaces utilisateur. Elle regroupe les écrans principaux de l'application, comme l'écran de connexion, le tableau de bord, la liste des sessions, l'écran de scan, l'écran d'affichage des résultats et l'historique des corrections.

La deuxième partie concerne la logique de navigation. Elle permet de gérer le passage d'un écran à un autre selon les actions de l'enseignant. Par exemple, après une connexion réussie, l'utilisateur est redirigé vers le tableau de bord. Lorsqu'il choisit une session, il peut accéder à l'écran de scan ou à la consultation des résultats.

La troisième partie correspond à la gestion des données affichées. Elle permet de stocker temporairement les informations reçues depuis l'API, comme les sessions disponibles, les résultats d'une correction ou les détails d'une feuille analysée.

La quatrième partie concerne la communication réseau. Elle permet à l'application mobile d'envoyer des requêtes vers l'API, notamment pour l'authentification, la récupération des sessions, l'envoi d'une image scannée et la récupération des résultats.

Enfin, l'application contient un module lié à la capture d'image, qui permet d'utiliser la caméra de scan de l'iPhone pour photographier la feuille de réponses dans les meilleures conditions possibles.

7.3.1 Authentification de l'enseignant :

L'accès à l'application mobile commence par une phase d'authentification. L'enseignant saisit son adresse email et son mot de passe dans l'écran de connexion. Ces informations sont ensuite envoyées à l'API afin de vérifier l'identité de l'utilisateur.

Si les identifiants sont corrects, l'application reçoit une réponse positive et redirige l'enseignant vers l'écran principal. Dans le cas contraire, un message d'erreur est affiché afin d'indiquer que les informations saisies sont incorrectes.

Cette étape permet de sécuriser l'accès à l'application et de garantir que seuls les utilisateurs autorisés peuvent accéder aux sessions de correction et aux résultats.



Figure 46 : Page d'authentification

7.3.2 Tableau de bord mobile :

Après la connexion, l'enseignant accède au tableau de bord de l'application mobile. Cet écran sert de point d'entrée vers les principales fonctionnalités disponibles.

Le tableau de bord permet à l'enseignant de consulter rapidement les sessions de correction disponibles, d'accéder au scan d'une nouvelle feuille, de voir les dernières corrections effectuées et de naviguer vers l'historique.

Cette interface a été conçue pour être simple et directe, car l'application mobile doit être utilisée rapidement pendant ou après une séance d'évaluation.

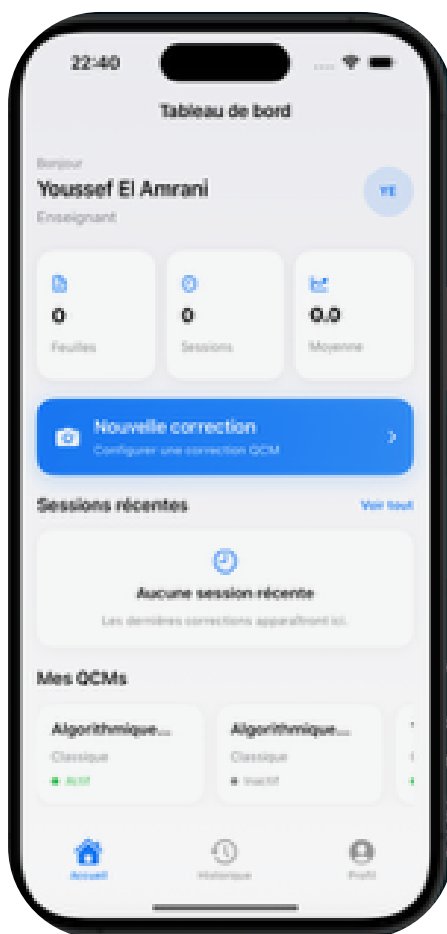


Figure 47 : Tableau de bord mobile

7.3.3 Consultation des sessions de correction :

L'application mobile permet à l'enseignant de consulter les sessions de correction qui lui sont associées. Chaque session correspond à un QCM publié et à une classe concernée.

Depuis cette liste, l'enseignant peut sélectionner une session afin de scanner une nouvelle feuille de réponses ou consulter les résultats déjà enregistrés. Cette organisation permet de lier chaque correction à un QCM précis, à une classe et à un contexte d'évaluation.

La consultation des sessions évite les erreurs de correction, car l'enseignant choisit d'abord la session avant d'envoyer une feuille à analyser.

7.3.4 Scan de la feuille de réponses :

Le scan de la feuille est la fonctionnalité principale de l'application mobile. L'enseignant utilise la caméra de l'iPhone pour photographier la feuille de réponses remplie par l'étudiant directement depuis l'application mobile grâce à la fonctionnalité VisionKit d'Apple.

L'objectif est d'obtenir une image claire, bien cadrée et suffisamment lisible pour permettre une correction fiable. L'application guide l'utilisateur afin qu'il capture correctement la feuille, en veillant à ce que toute la zone utile soit visible.

Une fois la photo prise, l'image est préparée puis envoyée à l'API. Cette image sera ensuite transmise au service IA afin d'extraire les réponses cochées et les informations de l'étudiant.

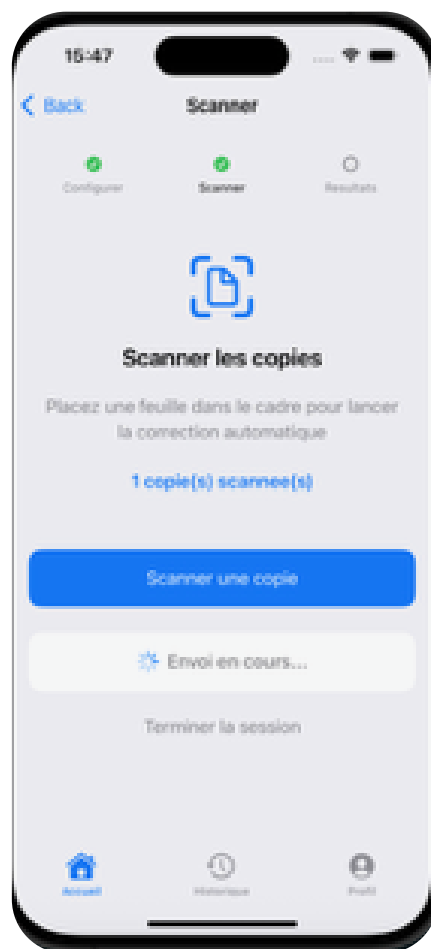


Figure 48 : Page de scan

7.3.5 Envoi de l'image vers l'API :

Après la capture, l'application mobile envoie l'image de la feuille scannée vers l'API. Cette communication se fait à travers une requête HTTP contenant l'image et l'identifiant de la session concernée.

Cette étape permet d'associer la feuille envoyée au bon QCM et à la bonne session de correction. Une fois la requête envoyée, l'application attend la réponse du serveur. Selon le résultat, elle affiche soit les informations détectées, soit un message d'erreur demandant de reprendre la photo.

7.3.6 Affichage des résultats de correction :

Après le traitement de la feuille, l'application mobile reçoit les résultats détectés. Ces résultats contiennent le numéro étudiant, le nom détecté et les réponses reconnues. Le score est alors calculé puis affiché.

L'enseignant peut alors consulter le résultat directement depuis son téléphone. Cette visualisation rapide lui permet de vérifier si la correction a été effectuée correctement ou si une anomalie est présente.

En cas de détection partielle ou d'erreur, l'application peut afficher un message indiquant que la feuille doit être vérifiée ou rescannée. Cela permet d'éviter l'enregistrement de résultats incorrects.

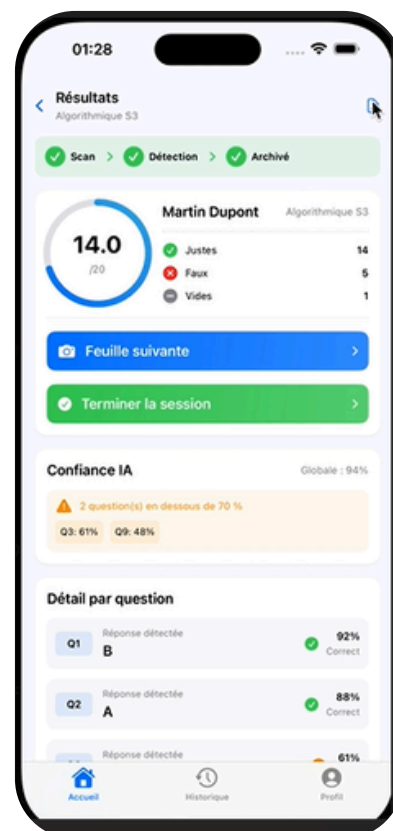


Figure 49 : Page d'affichage des résultats

7.3.7 Historique des corrections :

L'application mobile permet également de consulter l'historique des corrections. Cette fonctionnalité donne à l'enseignant la possibilité de retrouver les corrections déjà effectuées pour une session donnée.

L'historique contient les informations importantes liées aux feuilles corrigées, comme l'étudiant concerné, la date de correction, le score obtenu et le statut du scan.

Cette fonctionnalité améliore la traçabilité du processus de correction et permet à l'enseignant de suivre l'avancement d'une session directement depuis l'application mobile.



Figure 50 : Page de l'historique

7.4 Conception UML :

7.4.1 Diagramme de cas d'utilisation :

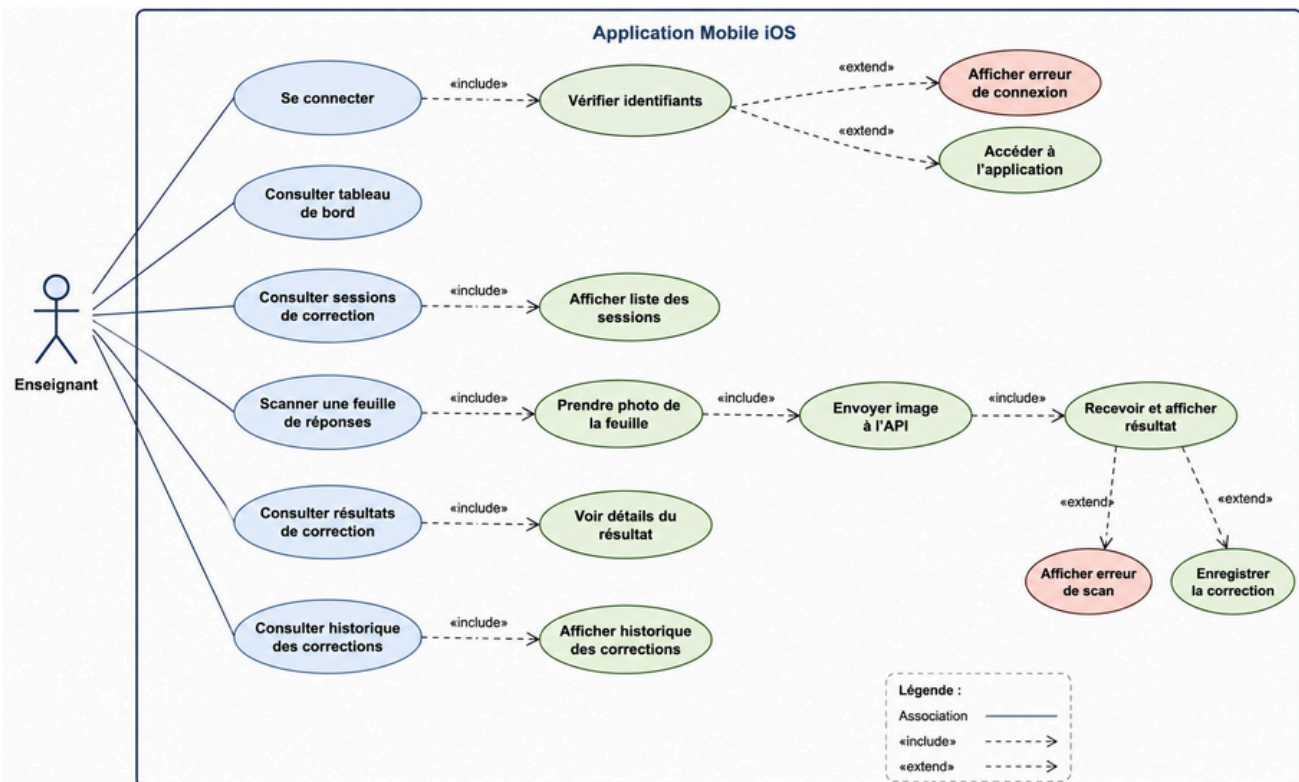


Figure 51 : Diagramme de cas d'utilisation de l'application mobile

7.4.2 Diagramme de séquence - Scan d'une feuille et affichage du résultat :

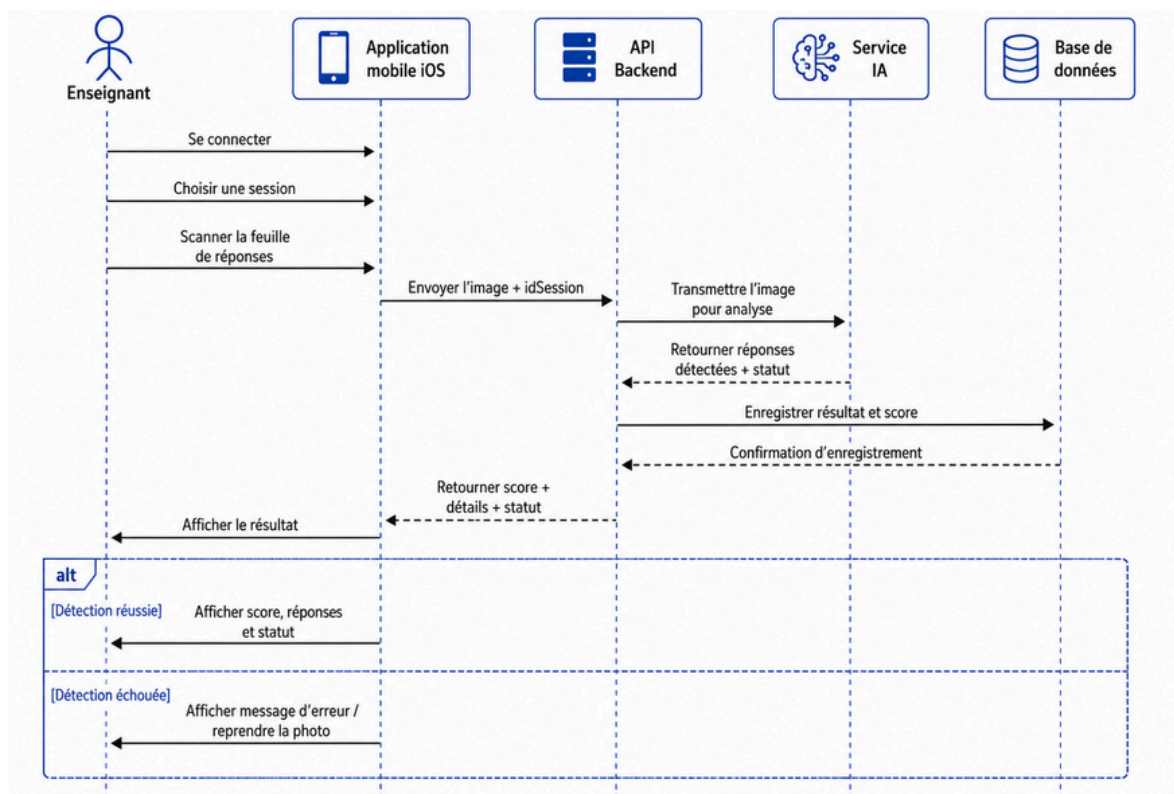


Figure 52 : Diagramme de séquence - Scan d'une feuille et affichage du résultat

7.4.3 Diagramme d'activité - Parcours de correction depuis l'application mobile :

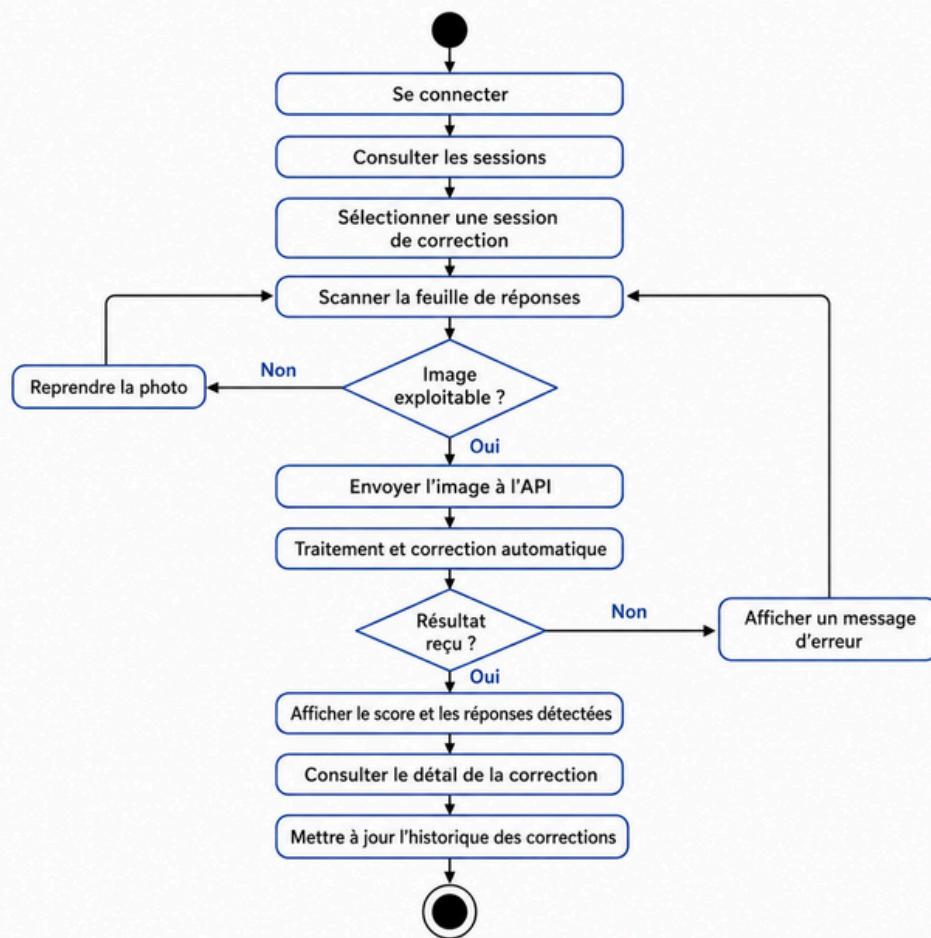


Figure 53 : Diagramme d'activité - Parcours de correction

8- Technologies et outils de développement

8.1 Technologies et outils utilisés pour la création de la base de données :

Pour concevoir et gérer la base de données de notre solution, nous avons utilisé plusieurs outils permettant de modéliser les données, créer les tables, définir les relations et assurer la communication avec les applications web et mobile.

- [Looping](#):

Looping est un outil de modélisation de bases de données permettant de concevoir les modèles de données de manière visuelle. Dans notre projet, il a été utilisé pour la réalisation du Modèle Conceptuel de Données (MCD) ainsi que pour sa transformation en Modèle Logique de Données (MLD).

Grâce à Looping, nous avons pu représenter clairement les différentes entités du système, leurs associations ainsi que les cardinalités entre elles. Cet outil a facilité la phase de conception en donnant une vision globale de la structure de la base de données avant son implémentation réelle.



- [PostgreSQL](#):

PostgreSQL est un système de gestion de base de données relationnelle open source, robuste et performant. Il permet de stocker, organiser et gérer les données de manière structurée à travers des tables reliées entre elles.

Dans notre projet, PostgreSQL a été retenu comme base de données principale pour assurer le stockage persistant des informations de la plateforme, notamment les utilisateurs, les QCM, les classes, les étudiants, les sessions de correction et les résultats. Il offre également des mécanismes avancés pour gérer les contraintes, les clés primaires, les clés étrangères, les relations entre les tables et la sécurité des données.



- [pgAdmin](#):

pgAdmin est un outil graphique d'administration de bases de données PostgreSQL. Il permet de créer, modifier, visualiser et gérer les bases de données à travers une interface simple et pratique.

Dans notre projet, pgAdmin a été utilisé pour administrer la base de données, exécuter les scripts SQL, créer les tables et vérifier les relations entre les différentes données. Il nous a également permis de tester les requêtes, d'insérer des données d'essai et de contrôler le bon fonctionnement de la structure de la base de données.



8.2 Technologies et outils utilisés pour le traitement d'image et le service IA :

Cette section présente les différents outils et technologies adoptés pour développer le système de traitement d'image et de détection automatique. Chaque technologie joue un rôle spécifique dans le processus global, depuis l'acquisition et le prétraitement de l'image jusqu'à l'identification des réponses et l'extraction des informations nécessaires.

- [Python:](#)

Python est le langage principal utilisé pour développer le service de traitement d'image. Il a été choisi pour sa simplicité, sa lisibilité et sa richesse en bibliothèques spécialisées dans l'intelligence artificielle et la vision par ordinateur. Dans notre projet, Python permet de regrouper toutes les étapes du traitement : chargement de l'image, analyse, détection des réponses, OCR et retour des résultats.



- [OpenCV:](#)

OpenCV est la bibliothèque principale utilisée pour les opérations de vision par ordinateur. Elle permet de convertir l'image en niveaux de gris, d'appliquer le seuillage, de détecter les contours, de corriger la perspective et d'extraire les zones d'intérêt. Dans notre projet, OpenCV est utilisé notamment pour détecter les quatre repères de la feuille, redresser l'image et découper les zones contenant le nom, le numéro étudiant et les réponses.



- [NumPy:](#)

NumPy est utilisé pour manipuler les images sous forme de matrices numériques. Comme une image est composée de pixels, elle peut être représentée par un tableau de valeurs. NumPy facilite donc les calculs sur les coordonnées, les dimensions, les zones découpées et les données envoyées aux modèles d'intelligence artificielle.



- [PIL/Pillow](#):

PIL, utilisé à travers la bibliothèque Pillow, permet de charger et convertir les images. Dans notre projet, il sert notamment à lire les images reçues sous forme de fichiers ou d'octets, vérifier qu'elles sont valides, puis les convertir dans un format exploitable par OpenCV.



- [EasyOCR](#):

EasyOCR est utilisé pour la reconnaissance optique de caractères. Dans notre projet, il permet de lire automatiquement le nom complet de l'étudiant à partir de la feuille de réponse. Avant la lecture OCR, la zone du nom est extraite et améliorée afin d'augmenter les chances d'obtenir un texte correct.



- [TensorFlow / Keras](#) :

TensorFlow et Keras sont utilisés pour charger et exploiter les modèles d'intelligence artificielle créés pour la détection des cases cochées. Dans notre projet, deux modèles sont utilisés : un modèle pour les cases des réponses QCM et un autre pour les cases du numéro étudiant. Ces modèles ont été entraînés à partir de notre propre dataset composé d'exemples de cases cochées et non cochées.



- [JSON](#):

Le format JSON est utilisé pour stocker les coordonnées des cases de réponses sur la feuille. Après le redressement de l'image, ces coordonnées deviennent fixes et permettent au programme de localiser précisément chaque case. Le fichier JSON facilite donc l'extraction automatique des zones correspondant aux choix A, B, C et D.



- [Visual Studio Code :](#)

Visual Studio Code est un éditeur de code moderne, léger et puissant développé par Microsoft. Il permet aux développeurs d'écrire, modifier, organiser et tester leur code dans un environnement simple et efficace. Dans cette partie, VS Code a été utilisé comme environnement principal de développement pour la partie traitement d'image et détection IA. Grâce à ses extensions Python et sa compatibilité avec Docker, il nous a permis de développer et tester les scripts liés au prétraitement des images, à la détection des cases cochées et à l'intégration de l'environnement TensorFlow.



- [Ubuntu Linux :](#)

Ubuntu est un système d'exploitation open source basé sur Linux. Il est largement utilisé dans le développement informatique, l'intelligence artificielle, le traitement d'image et les environnements serveur. Il est apprécié pour sa stabilité, sa sécurité et sa compatibilité avec de nombreux outils de programmation.

Dans notre projet, nous avons utilisé Ubuntu Linux comme environnement de travail pour la partie traitement d'image et détection IA. Ce système nous a permis d'installer et d'exécuter facilement les bibliothèques Python nécessaires, notamment OpenCV et TensorFlow. Il a également facilité l'utilisation de Docker pour créer un environnement isolé, stable et reproductible dédié au service IA.



- [Docker :](#)

Docker est une plateforme qui permet de créer et d'exécuter des applications dans des environnements isolés appelés conteneurs. Un conteneur contient tous les éléments nécessaires au fonctionnement d'une application, comme le code source, les bibliothèques, les dépendances et les fichiers de configuration.

Dans notre projet, Docker a été utilisé pour créer un environnement personnalisé dédié à la partie intelligence artificielle. Cet environnement contient TensorFlow ainsi que les bibliothèques nécessaires au traitement d'image. L'utilisation de Docker nous a permis d'éviter les problèmes de compatibilité entre les versions des outils et de garantir que le service IA fonctionne de la même manière sur différentes machines.



8.3 Technologies et outils utilisés dans le développement de l'application web :

Le développement de l'application web du projet QCM Corrector repose sur plusieurs technologies complémentaires. Chaque outil intervient à un niveau précis : interface utilisateur, communication avec le backend, typage du code, génération des pages, gestion des données, style visuel ou environnement d'exécution.

- [React](#) :

React est la bibliothèque JavaScript utilisée pour construire l'interface utilisateur de l'application web. Elle permet de créer des composants réutilisables, comme les formulaires, les tableaux, les boutons, les cartes de statistiques, les badges de statut ou encore les sections de navigation.



Dans le projet, React intervient dans toutes les pages de l'application web: page de connexion, tableau de bord, création de QCM, affichage des sessions, consultation des résultats et gestion des classes. Grâce à React, chaque partie de l'interface peut être organisée sous forme de composants indépendants, ce qui facilite la maintenance et l'évolution de l'application.

- [Next.js](#) :

Next.js est le framework utilisé pour structurer l'application web. Il repose sur React, mais ajoute une organisation plus complète avec un système de routes, de pages et de rendu optimisé. Dans le projet QCM Corrector, Next.js permet de créer les différentes pages de l'application: /login, /dashboard, /qcms, /qcms/new, /sessions, /sessions/[id], /admin/classes, /admin/profs, etc.

Next.js facilite aussi la navigation entre les pages grâce à son système de routing. Par exemple, lorsqu'un utilisateur clique sur une session, il est redirigé vers la page détaillée des résultats correspondants. Il permet donc d'organiser clairement toute la structure front-end de la plateforme.



- [TypeScript](#) :

TypeScript est utilisé pour ajouter un typage statique au code JavaScript. Il permet de définir clairement la forme des données manipulées dans l'application: QCM, questions, choix de réponses, sessions, résultats, étudiants, professeurs ou classes.

TypeScript

Dans le projet, TypeScript aide à éviter certaines erreurs de développement. Par exemple, lorsqu'une page récupère les résultats d'une session depuis l'API, les types permettent de savoir précisément quelles propriétés sont disponibles: nom de l'étudiant, score, réponses détectées, statut, date de correction, etc. Cela rend le code plus fiable et plus facile à comprendre.

- [Node.js](#) :

Node.js est l'environnement d'exécution utilisé pour lancer l'application web en développement. Il permet d'exécuter les commandes liées au projet Next.js, comme le lancement du serveur local, l'installation des dépendances ou la compilation du projet.



Dans notre cas, Node.js intervient principalement pour lancer le front-end avec la commande:

```
powershell  
  
npm run dev
```

Il permet ainsi d'exécuter l'application web localement et de la tester dans le navigateur

npm est le gestionnaire de paquets utilisé avec Node.js. Il permet d'installer et de gérer les dépendances nécessaires au fonctionnement de l'application web. Ces dépendances incluent Next.js, React, TypeScript et les autres bibliothèques utilisées dans le projet.

npm permet aussi d'exécuter les scripts du projet, comme:

```
powershell  
  
npm run dev  
npm run build  
npm run lint
```

La commande *"npm run dev"* lance le serveur de développement, *"npm run build"* vérifie que l'application peut être compilée correctement, et *"npm run lint"* permet de détecter certaines erreurs ou mauvaises pratiques dans le code.

- [HTML:](#)

HTML est utilisé pour structurer le contenu des pages web. Même si l'application est développée avec React et JSX, la structure finale repose sur des éléments HTML : formulaires, tableaux, sections, boutons, champs de saisie, listes et liens.

Dans QCM Corrector, HTML intervient dans la construction des pages comme la page de connexion, les formulaires de création de QCM, les tableaux de résultats et les pages de gestion des classes ou professeurs.



- [CSS:](#)

CSS est utilisé pour définir l'apparence visuelle de l'application web. Il permet de gérer les couleurs, les espacements, les bordures, les tailles de police, les grilles, les cartes, la barre latérale et l'adaptation de l'interface aux différentes tailles d'écran.

Dans le projet, le CSS est important pour rendre l'interface plus claire et professionnelle. Il permet par exemple de distinguer les statuts avec des badges, de structurer les formulaires en plusieurs blocs et de rendre les tableaux plus lisibles.



- [Bootstrap :](#)

Bootstrap est une bibliothèque CSS utilisée pour accélérer la mise en forme de certains éléments d'interface. Elle fournit des classes prêtes à l'emploi pour les boutons, les formulaires, les grilles, les espacements et certains composants visuels.

Dans le projet, Bootstrap aide à obtenir rapidement une interface cohérente et responsive. Il complète le CSS personnalisé utilisé pour adapter le style à l'identité visuelle de QCM Corrector et de JUNIA Maroc.



- [JSON :](#)

JSON est le format principal utilisé pour l'échange de données entre l'application web et le backend. Lorsqu'une page web demande les sessions, les classes ou les résultats, le backend renvoie une réponse au format JSON. De même, lorsque l'utilisateur crée un QCM, l'application web envoie les informations au backend sous forme JSON.

Par exemple, un QCM envoyé à l'API contient le titre, la matière, la classe, le professeur, les questions, les propositions, les bonnes réponses et les points. JSON permet donc une communication simple et structurée entre le front-end et le backend.



- [Fetch API :](#)

La Fetch API est utilisée côté front-end pour envoyer des requêtes HTTP vers le backend FastAPI. Elle permet de récupérer les données depuis l'API ou d'envoyer de nouvelles informations. Dans le projet, elle intervient par exemple pour:

- connecter un utilisateur;
- récupérer les QCM;
- récupérer les classes;
- récupérer les sessions;
- récupérer les résultats d'une session;
- publier un QCM;
- envoyer les résultats par mail.
-

Elle constitue donc le lien direct entre l'application web et l'API backend.



- [FastAPI:](#)

Même si FastAPI appartient surtout à la partie backend, il intervient directement dans le fonctionnement de l'application web, car toutes les données affichées dans le front-end proviennent de routes FastAPI.

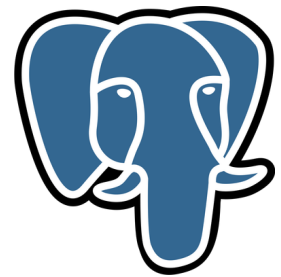
L'application web dépend de FastAPI pour obtenir les classes, les professeurs, les QCM, les sessions et les résultats. FastAPI reçoit aussi les données envoyées depuis le formulaire de création de QCM. Il joue donc le rôle d'interface entre l'application web et la base PostgreSQL.



- [PostgreSQL:](#)

PostgreSQL est le système de gestion de base de données utilisé pour stocker les données de la plateforme. Pour l'application web, PostgreSQL est la source principale des informations affichées: utilisateurs, professeurs, classes, étudiants, matières, QCM, questions, propositions, sessions et résultats.

L'application web ne se connecte pas directement à PostgreSQL. Elle passe par le backend FastAPI. Cependant, toutes les pages dynamiques de l'application web dépendent des données stockées dans PostgreSQL.



- [SQL:](#)

SQL est le langage utilisé pour interroger et modifier les données dans PostgreSQL. Il intervient dans le backend pour récupérer les informations demandées par l'application web ou pour enregistrer de nouvelles données.

Par exemple, lorsqu'un administrateur consulte la page des classes, le backend exécute des requêtes SQL pour récupérer les classes, les étudiants et les matières associées. Lorsqu'un QCM est publié, SQL permet d'insérer les questions et les réponses dans la base.



- [Uvicorn :](#)

Uvicorn est le serveur utilisé pour exécuter l'application FastAPI. Il permet au backend de recevoir les requêtes envoyées par l'application web. Sans Uvicorn, l'API FastAPI ne serait pas accessible depuis le navigateur.

Dans le projet, Uvicorn est lancé avec une commande du type:

```
powershell
uvicorn main:app --reload --host 127.0.0.1 --port 8001
```



Il rend donc le backend disponible localement pour que le front-end puisse communiquer avec lui.

- [PyPDF / Génération PDF :](#)

La génération PDF intervient dans le projet pour produire les documents liés aux QCM. L'application web permet de générer le sujet PDF et la grille PDF des bonnes réponses. Ces documents sont construits côté backend à partir des données du QCM enregistrées en base.

Le sujet PDF contient les questions et les choix proposés aux étudiants. La grille PDF contient les bonnes réponses, marquées par des croix. Cette fonctionnalité permet de transformer un QCM créé dans l'interface web en document imprimable et exploitable pour une évaluation réelle.



- [SMTP Gmail :](#)

SMTP est le protocole utilisé pour envoyer les mails de résultats aux étudiants. Dans le projet, un compte Gmail est configuré avec un mot de passe d'application. Les paramètres SMTP sont stockés dans le fichier .env du backend.

L'application web déclenche l'envoi des mails à partir de la page des résultats. Le backend récupère alors les notes des étudiants, génère un mail personnalisé et l'envoie via SMTP. Ce service permet d'automatiser la communication des résultats.



- [Cloudflare :](#)

Cloudflare peut intervenir comme outil d'exposition ou de tunneling lorsque l'application doit être rendue accessible à distance, par exemple pour permettre à un autre appareil ou à une autre personne de communiquer avec le backend pendant les tests.



Dans le cadre du projet, cela peut être utile pour permettre à l'application mobile ou à un service externe d'envoyer des résultats vers l'API backend même si celle-ci tourne localement sur une machine de développement. Cloudflare sert donc à faciliter les tests réseau et la communication entre modules.

- [Visual Studio Code :](#)

Visual Studio Code est l'environnement de développement utilisé pour écrire, organiser et modifier le code de l'application. Il permet de gérer les fichiers du projet, d'ouvrir les dossiers front-end et backend, d'utiliser le terminal intégré et de suivre les erreurs de syntaxe.



Dans cette partie, VS Code est utilisé pour travailler sur les fichiers Next.js, TypeScript, CSS, Python, SQL et .env.

- [Git :](#)

Git est l'outil de gestion de versions utilisé pour suivre l'évolution du code source. Il permet de conserver l'historique des modifications, de comparer les versions et de collaborer plus facilement entre les membres du projet.



Dans un projet réalisé par plusieurs membres, Git est utile pour séparer les contributions: application web, backend, base de données, application mobile et traitement d'image.

8.4 Technologies et outils utilisés dans le développement de l'application mobile :

- [Swift 5.9+ :](#)

Langage de programmation moderne d'Apple, typé et sûr, conçu pour iOS/macOS en remplacement d'Objective-C. Syntaxe concise, gestion mémoire automatique (ARC), hautes performances. La version 5.9 a introduit les macros et un parallélisme amélioré.



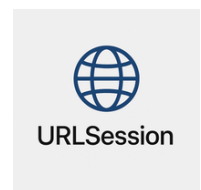
- [SwiftUI :](#)

Framework déclaratif d'Apple pour construire les interfaces : l'interface est décrite à partir de son état, le framework gère le rendu. Aperçu en direct dans Xcode, animations fluides, multiplateforme Apple.



- [URLSession :](#)

API réseau native (Foundation) pour les requêtes HTTP/HTTPS : GET/POST, upload, download, tâches asynchrones. C'est la base de la communication réseau iOS sans bibliothèque tierce.



- [VisionKit :](#)

Framework Apple de numérisation de documents par caméra : détection des bords, recadrage automatique, OCR en direct. Il permet de gérer le scan de documents.



- [PhotosUI :](#)

Framework Apple (PHPicker) pour sélectionner photos et vidéos depuis la galerie de manière sécurisée, sans accès complet à la photothèque.



- [Xcode :](#)

IDE officiel d'Apple : éditeur, compilateur, simulateur, débogueur, signing et déploiement App Store. Indispensable au développement Apple.



9- Difficultés rencontrées et perspectives :

La réalisation du projet QCM Corrector n'a pas été limitée à la création d'une simple application web. Le projet regroupe plusieurs parties qui doivent fonctionner ensemble : l'application web, l'application mobile, la base de données, l'API Backend, le service IA, le traitement d'image et l'envoi automatique des résultats par mail. L'une des premières difficultés était donc de construire une architecture cohérente, où chaque composant communique correctement avec les autres. L'application web devait gérer les QCM, les classes, les professeurs, les étudiants, les sessions de correction et les résultats, tandis que le service IA devait recevoir les images des feuilles de réponses, les traiter, puis retourner au backend les informations détectées sous forme de fichier JSON.

Une autre difficulté importante était liée à la gestion des rôles. L'administrateur et le professeur n'ont pas les mêmes droits dans l'application. L'administrateur doit pouvoir gérer toute la plateforme, notamment les professeurs, les classes et les étudiants, alors que le professeur doit seulement accéder aux données qui le concernent. Il a donc fallu adapter l'affichage des menus, des pages, des boutons et des actions selon le profil connecté. Cette séparation permet de mieux sécuriser les données et d'éviter qu'un utilisateur accède à des informations qui ne lui sont pas destinées.

La création des QCM a aussi demandé plusieurs ajustements. Le formulaire contient beaucoup d'informations, mais il devait rester simple à utiliser pour l'enseignant. Pour cela, nous avons organisé la création du QCM en plusieurs étapes : les informations générales, la saisie des questions, le choix des bonnes réponses, le récapitulatif puis la publication. Il fallait également prendre en compte le cycle de vie du QCM, avec les états brouillon, publié et verrouillé. Cette règle est importante, car un QCM déjà utilisé dans une session de correction ne doit plus être modifié afin de ne pas fausser les résultats déjà enregistrés.

La base de données a également demandé un travail de conception précis. Elle devait permettre de stocker les utilisateurs, les QCM, les questions, les réponses, les classes, les étudiants, les sessions et les résultats de manière organisée. La difficulté était de créer un modèle relationnel clair, capable de garder la cohérence des données tout en restant évolutif pour les futures améliorations du projet. L'utilisation de PostgreSQL a permis de mettre en place une base solide, mais il fallait bien définir les relations entre les tables, les clés étrangères, les contraintes et la manière dont les données seraient récupérées par l'application.

La partie traitement d'image et service IA a été l'une des plus délicates. Les images prises depuis l'application mobile peuvent être inclinées, floues, mal éclairées ou contenir des ombres. C'est pour cette raison que le choix de VisionKit d'Apple était important pour faciliter le scan mobile de la feuille. Malgré cela, il fallait encore prévoir plusieurs étapes de traitement, comme la correction de perspective, la normalisation de la luminosité, la binarisation et l'extraction des zones d'intérêt.

Ensuite, le système devait détecter les cases du numéro étudiant ainsi que les réponses cochées de Q1 à Q60.

La création d'un modèle IA pour classer les cases cochées ou vides a demandé la création d'un dataset conséquent de plus de 40 000 exemples que nous avons remplis et scannés nous-mêmes, ainsi que des tests sur des feuilles réelles afin de vérifier le comportement du système dans différents cas.

L'application mobile a aussi présenté certaines difficultés. Son rôle ne se limite pas seulement à prendre une photo de la feuille QCM. Elle doit aider l'enseignant à obtenir une image exploitable, vérifier que la feuille est bien cadrée, envoyer cette image au backend et afficher un message clair lorsque le traitement ne peut pas être effectué. La qualité de la photo est donc un élément très important, car elle influence directement les résultats du traitement d'image. Une feuille mal cadrée, des marqueurs non visibles, une image floue ou une connexion réseau instable peuvent empêcher le bon déroulement de la correction. Il fallait donc prévoir ces cas d'erreur afin que l'utilisateur puisse comprendre le problème et refaire la photo si nécessaire.

L'intégration entre les différentes parties du projet a également demandé beaucoup d'attention. Le backend occupe une place centrale dans l'architecture : il reçoit les requêtes venant de l'application web ou mobile, communique avec la base de données, transmet l'image au service IA, récupère le fichier JSON généré, archive les résultats et déclenche ensuite l'envoi du mail. Cette communication devait être bien organisée pour éviter les incohérences entre les données détectées par l'IA, les sessions enregistrées et les résultats affichés à l'utilisateur. L'envoi automatique des notes par mail représentait aussi un point sensible, car il devait se faire de manière fiable, sans ralentir ni bloquer le fonctionnement principal de l'application.

Concernant les perspectives, plusieurs améliorations peuvent être envisagées. La première concerne le modèle IA, qui pourrait être amélioré avec un dataset plus riche contenant davantage d'exemples de cases scannées dans différentes conditions : feuilles bien éclairées, images sombres, feuilles inclinées, cases partiellement cochées ou images légèrement floues. Cela permettrait d'augmenter la précision de la détection et de réduire les erreurs lors de l'extraction du numéro étudiant et des réponses cochées.

L'application mobile pourrait également être améliorée en ajoutant une assistance en temps réel pendant la prise de photo. Par exemple, elle pourrait indiquer à l'enseignant si la feuille est bien cadrée, si les quatre marqueurs sont visibles ou si la luminosité est suffisante. Cette amélioration permettrait d'éviter certaines erreurs avant même l'envoi de l'image au backend. L'OCR manuscrit pourrait aussi être renforcé afin de mieux reconnaître le nom et le prénom des étudiants, surtout lorsque l'écriture est différente d'un étudiant à un autre. D'autres évolutions peuvent concerner l'application web. La plateforme pourrait proposer des tableaux de bord plus avancés, des statistiques pédagogiques par classe, par matière ou par QCM, ainsi que des exports PDF ou Excel plus détaillés. Le suivi des performances des étudiants pourrait aussi être amélioré afin d'aider les enseignants à mieux analyser les résultats. Le projet pourrait enfin évoluer vers une synchronisation plus complète avec des plateformes externes comme Edusign, ou vers l'ajout d'un espace étudiant dans une version future. Grâce à l'architecture organisée en composants séparés, il sera possible de faire évoluer le service IA, le backend, l'application mobile ou l'application web sans devoir reconstruire toute la plateforme.

10- Conclusion :

En conclusion, ce projet nous a permis de transformer une problématique courante du milieu éducatif en une solution concrète et exploitable. À travers QCM Corrector, nous avons cherché à simplifier une tâche souvent répétitive pour les enseignants, tout en proposant une approche plus moderne, plus organisée et mieux adaptée aux besoins actuels de l'évaluation.

Au-delà de la réalisation de l'application elle-même, ce travail nous a surtout appris à penser un projet dans son ensemble. Il ne suffisait pas de développer des fonctionnalités séparées, mais de construire une solution cohérente, capable de répondre à un usage réel. Cette démarche nous a amenés à réfléchir à l'expérience de l'utilisateur, à la fiabilité du système, à la clarté des interfaces et à la continuité entre les différentes étapes du processus de correction.

Ce projet a également été une occasion de mieux comprendre la différence entre un simple exercice technique et une véritable solution logicielle. Chaque choix devait être justifié par un besoin précis : faciliter le travail de l'enseignant, éviter les erreurs, garder une trace des résultats et rendre l'utilisation la plus simple possible. Cette vision nous a permis de prendre davantage conscience de l'importance de l'analyse, de la conception et de l'organisation dans la réussite d'un projet informatique.

QCM Corrector constitue ainsi une première base solide, mais aussi un point de départ vers une solution plus complète. Avec davantage de tests, d'améliorations et de retours utilisateurs, la plateforme pourrait évoluer vers un outil encore plus performant, capable d'accompagner les enseignants dans le suivi des évaluations et l'analyse des résultats.

Cette expérience nous a donc permis de progresser sur le plan technique, mais aussi sur le plan méthodologique et humain. Elle nous a appris à travailler avec rigueur, à résoudre des problèmes concrets et à mieux comprendre les exigences d'un projet destiné à être utilisé dans un contexte réel. Elle représente ainsi une étape importante dans notre parcours de formation et une base encourageante pour nos futurs projets.

Ce projet ne représente pas seulement une application développée, mais aussi une première réponse concrète à un besoin réel d'automatisation et d'efficacité dans le domaine de l'évaluation pédagogique.